`fall_n` — Technical Documentation

Simplex Elements, Quadrature Theory,
Element Architecture, and Implementation Decisions

S. E. Echavarría-Montana

March 14, 2026

Abstract

This document provides a comprehensive technical reference for the `fall_n` nonlinear finite element library. It covers the implementation of simplex (tetrahedral) finite elements, numerical integration strategies on simplicial domains, the engineering decisions made to ensure robust post-processing, and the layered element architecture with its policy-based design patterns.

Key topics include:

- Lagrange shape functions on the reference simplex and their isoparametric mapping to physical space;
- Classical simplex quadrature rules and their negative-weight pathology;
- The Stroud conical product rule with Gauss–Jacobi nodes via the Golub–Welsch algorithm as an all-positive-weight alternative;
- Diagnosis and resolution of the checkerboard artefact in lumped L^2 projections on quadratic tetrahedra;
- Polynomial patch recovery as the production nodal reconstruction for higher-order simplex visualisation, with volume-weighted averaging kept as a conservative fallback;
- An adaptive material-field projection strategy that distinguishes between element families with strictly positive local lumping and those that require robust local reconstruction;
- The refactor of `ElementGeometry`: corrected value semantics, explicit quadrature strategy contracts, compile-time invariant enforcement, and preservation of runtime performance in the hot path;
- Serendipity tensor-product and simplex element families, including static reference cells, benchmark validation, VTK support, and Gmsh import paths for HEX20 and QUA8;
- A detailed ontology for the `Domain` refactor, separating geometry, mesh vertices, nodal unknown carriers, integration sites, constitutive points, and structural section entities under a compile-time-oriented HPC design philosophy;
- The first implementation phase of the `Domain` refactor: canonical `Vertex` storage, lazy PETSc mesh creation, transitional nodal caching, and geometry-only mesh export through vertices;
- The second implementation phase of the `Domain` refactor: geometry-only element binding through `Point/Vertex`, optional analysis-node binding, vertex-driven boundary extraction, and preservation of the compile-time/HPC characteristics of the element kernels;
- A new structural-scale reconstruction layer for beams and shells: constitutive section snapshots, formulation-specific reduction policies, compile-time section profiles, and a VTK multi-block export architecture that separates axis, surface, sections, material sites, and fibers;
- A quadrature-postprocessing clarification for VTK/ParaView: explicit nodal and quadrature tensor naming, scalar component export, and invariant filters such as von Mises, Beltrami–Haigh, pressure, triaxiality, and equivalent strain to avoid ambiguous rendering of raw Voigt arrays in quadrature-aware pipelines;
- A low-level performance pass on point, vertex, node, and element geometry access paths: contiguous coordinate access, direct DoF pointer access, cached coordinate pointers inside geometric elements, and removal of residual geometry-via-node usage in beam frame construction;
- A spatial-site and `Domain` laziness pass: a read-only geometry-site concept for point-like entities, explicit forwarding for constitutive and section decorators, and geometry-first boundary extraction without eager node-cache materialisation;

- A semantic hardening pass on `DMPlex`: mixed 1D/2D/3D cell families can now coexist in the same non-interpolated DAG through explicit point-role, topological-dimension, and physical-group labels, while the algebraic layout is driven by depth-0 vertices rather than brittle height-based assumptions;
- A full mixed-building demonstration in the executable entry point: first an elastic baseline, and then a materially nonlinear reinforced-concrete continuation with fiber sections in the frame, elastic MITC4 slabs, incremental SNES solution, and multi-block structural VTK export including fiber-level fields;
- A dedicated architectural audit of the materials module: semantic separation between physical materials, constitutive laws, integration algorithms, constitutive state carriers, and structural reduction policies, together with a roadmap toward nonlinear reinforced concrete across scales, finite kinematics, dynamics, and continuum-scale cracking/fracture;
- A constitutive-state storage cleanup: explicit committed-only, history-tracking, and trial/committed storage policies, with the default inelastic path moved away from implicit full kinematic history and toward semantically correct committed-state storage;
- A new semantic `ConstitutiveState` layer above the storage adapter: the local state of a constitutive site is now explicit in the API, while static polymorphism and configurable storage backends are preserved;
- An explicit `ConstitutiveIntegrator` layer at the erased `Material<Policy>` boundary: the local constitutive algorithm is now a first-class injection point, with legacy `ElasticUpdate` and `InelasticUpdate` retained as compatibility aliases;
- A first behavioural extraction of irreversible memory out of the law: `PlasticityRelation` now supports an explicit external algorithmic-state path, and `MaterialInstance` stores that state at the constitutive-site level while preserving the legacy embedded API for compatibility;
- A bug fix in PETSc 3.24.5 `section.c` and application-level workarounds for PETSc indexing and preallocation issues;
- The element architecture design: External Polymorphism at the geometry and assembly levels, policy-based customisation points (MaterialPolicy, KinematicPolicy, ElementPolicy), compile-time efficiency, and comparison with OpenSees and Kratos Multiphysics.

Each chapter includes the relevant mathematical theory with bibliographic references, the concrete C++ implementation in `fall_n`, and a discussion of the design rationale.

Contents

1 Simplex Finite Elements **12**
1.1 Reference Simplex . . . 12
1.2 Lagrange Shape Functions . . . 12
1.2.1 Linear Elements (Order 1) . . . 12
1.2.2 Quadratic Elements (Order 2) . . . 13
1.3 Shape Function Derivatives . . . 13
1.4 Isoparametric Mapping . . . 13
1.5 Subentity Topology: Faces of the Simplex . . . 14
1.5.1 Face Local Node Ordering (TET10) . . . 14
1.6 Gmsh Node Ordering for TET10 . . . 14
1.7 Implementation in `fall_n` . . . 15

2 Numerical Quadrature on Simplices **16**
2.1 Problem Statement . . . 16
2.2 Classical Symmetric Rules . . . 16
2.2.1 Centroid Rule (Degree 1) . . . 16
2.2.2 Hammer–Marlowe–Stroud Rules (HMS) . . . 16
2.2.3 Keast Rules . . . 17
2.2.4 Grundmann–Möller Rules . . . 17
2.3 Stroud Conical Product Rule . . . 17
2.3.1 Duffy Transform . . . 18
2.3.2 Product Quadrature with Gauss–Jacobi Nodes . . . 18
2.3.3 Cost Comparison . . . 19
2.4 Gauss–Jacobi Quadrature via the Golub–Welsch Algorithm . . . 19
2.4.1 Jacobi Polynomials . . . 19
2.4.2 The Golub–Welsch Algorithm . . . 19
2.4.3 Mapping to $[0, 1]$ . . . 20
2.5 Implementation: `StroudConicalProduct.hh` . . . 20
2.6 Default Quadrature Choices in `fall_n` . . . 21

3 Negative Weights and the Checkerboard Artefact **22**
3.1 Symptom: Element-Level Checkerboard Pattern . . . 22
3.2 Root Cause Analysis . . . 22
3.2.1 The Keast 5-Point Rule Weights . . . 23
3.2.2 TET10 Vertex Shape Functions at Quadrature Points . . . 23
3.2.3 The Denominator Catastrophe . . . 23
3.2.4 Consequence: Sign Flip or Division by Zero . . . 24
3.3 Why HEX27 Is Unaffected . . . 24
3.4 Resolution: Two Complementary Fixes . . . 24
3.4.1 Fix 1: Switch to an All-Positive-Weight Quadrature Rule . . . 24
3.4.2 Fix 2: Guard Against Negative Denominators . . . 24

3.4.3 Fix 3 (Recommended): SPR Volume-Weighted Averaging . . . 25
3.5 Summary . . . 25

4 Gauss-Point to Nodal Projection **26**
4.1 Lumped L^2 Projection . . . 26
4.1.1 Consistent L^2 Projection . . . 26
4.1.2 Row-Sum Lumping . . . 26
4.2 Polynomial Patch Recovery . . . 27
4.2.1 Motivation . . . 27
4.2.2 Patch Definition . . . 27
4.2.3 Local Polynomial Basis . . . 27
4.2.4 Weighted Least Squares . . . 27
4.2.5 Adaptive Degree Fallback . . . 28
4.2.6 Evaluation at the Node . . . 28
4.3 Volume-Weighted Averaging . . . 28
4.4 Implementation in `VTKModelExporter` . . . 28
4.5 Comparison of the Three Operators . . . 29
4.6 Verification . . . 29
4.6.1 Algebraic regression . . . 29
4.6.2 Application-level behaviour . . . 30

5 PETSc Patch: `section.c` Indexing Bug **31**
5.1 Context: DMPlex and PetscSection . . . 31
5.2 The Bug . . . 31
5.3 Manifestation in `fall_n` . . . 32
5.4 The Fix . . . 32
5.5 Verification . . . 32
5.6 Upstream Status . . . 32
5.7 Workaround from Application Code . . . 33
5.7.1 Workaround 1: Extend the Section Chart . . . 33
5.7.2 Workaround 2: Allow Dynamic Matrix Allocation . . . 33
5.7.3 Verification . . . 34

6 Implementation Summary and Design Decisions **35**
6.1 Architecture Overview . . . 35
6.2 Design Decisions . . . 35
6.2.1 Decision 1: HMS 4-Point as Default TET10 Quadrature . . . 35
6.2.2 Decision 2: Golub–Welsch for Gauss–Jacobi Quadrature . . . 36
6.2.3 Decision 3: Adaptive Projection as Default . . . 36
6.2.4 Decision 4: Gmsh TET10 Node Reordering . . . 37
6.2.5 Decision 5: PETSc Section Patch . . . 37
6.2.6 Decision 6: Hex27 VTK Face-Center Ordering . . . 37
6.2.7 Decision 7: Split VTK Export as the Supported Contract . . . 38
6.2.8 Decision 8: Explicit Scalar Filters over Raw Voigt Arrays in ParaView . . . 38
6.3 Nonlinear Showcase (`main.cpp`) . . . 39
6.4 Test Suite . . . 39
6.5 File Inventory . . . 40

7 Element Architecture: Design Patterns and Extensibility **41**
7.1 Guiding Principles . . . 41
7.2 Layer Decomposition . . . 41
7.2.1 Layer 1: Concrete Geometry Types . . . 42
7.2.2 Layer 2: ElementGeometry (Type-Erased Handle) . . . 42
7.2.3 Layer 3: Physical Elements . . . 44
7.2.4 Layer 4: Analysis Level . . . 44
7.3 FEM_Element: Top-Level Type Erasure . . . 44
7.4 Policy-Based Design: Customisation Points . . . 45
7.4.1 MaterialPolicy . . . 45
7.4.2 KinematicPolicy . . . 45
7.4.3 ElementPolicy (Storage Strategy) . . . 46
7.4.4 AssemblyPolicy (Beams) . . . 46
7.4.5 ConstitutiveStrategy (Material Update) . . . 47
7.5 Compile-Time Cost Analysis . . . 47
7.6 Comparison with OpenSees and Kratos . . . 48
7.6.1 Overview . . . 48
7.6.2 OpenSees . . . 48
7.6.3 Kratos Multiphysics . . . 49
7.6.4 Summary: Architectural Trade-offs . . . 49
7.7 Extensibility Guide: Adding a New Element Family . . . 50
7.8 Runtime Efficiency Considerations . . . 51
7.8.1 Virtual Dispatch Budget . . . 51
7.8.2 Cache-Friendly Layout . . . 51
7.8.3 DOF Index Caching . . . 51
7.8.4 Stack-Allocated Jacobians . . . 51

8 Generic Boundary and Initial Condition Infrastructure **52**
8.1 Gap Analysis: the Legacy API . . . 52
8.2 Design Decisions and Trade-offs . . . 52
8.2.1 Compile-Time Polymorphism via Concepts . . . 52
8.2.2 Constraint Storage: Extending the Existing Map . . . 53
8.2.3 Non-Zero Dirichlet: Where Values Flow . . . 54
8.2.4 ModelState: Platform-Independent Snapshot . . . 54
8.3 Implementation . . . 54
8.3.1 NodeSelector.hh — Built-in Selectors . . . 54
8.3.2 Model.hh — New Constraint Methods . . . 55
8.3.3 Backward Compatibility . . . 55
8.4 Use-Case Walkthrough: Roller vs. Clamped Cantilever . . . 55
8.5 State Capture and Continuation Analysis . . . 56
8.5.1 ModelState Structure . . . 56
8.5.2 Capture Workflow . . . 57
8.5.3 Future: Initial Stress Integration . . . 57
8.6 Comparison with OpenSees and Kratos . . . 57
8.7 Extensibility Roadmap . . . 58
8.8 Test Suite . . . 58
8.9 Summary . . . 59

9 ElementGeometry Refactor **60**
9.1 Problem Statement . . . 60
9.1.1 Broken value semantics . . . 60
9.1.2 Integration strategy mixed into concrete geometry headers . . . 60
9.1.3 Weak compile-time invariants . . . 60
9.2 Design Goals . . . 61
9.3 Refactor Overview . . . 61
9.3.1 Metadata moved under wrapper ownership . . . 61
9.3.2 Quadrature strategies extracted to dedicated headers . . . 61
9.3.3 A small quadrature concept . . . 62
9.4 Geometry and Measure . . . 62
9.5 API Hardening . . . 62
9.5.1 Restricting `detJ()` . . . 62
9.5.2 Fail-fast Gauss–Legendre tables . . . 62
9.5.3 Safer tensor-product constraints . . . 62
9.6 Materialised Integration Points . . . 63
9.7 Performance Discussion . . . 63
9.7.1 What did not change . . . 63
9.7.2 What did change . . . 63
9.8 What Was Deliberately Deferred . . . 63
9.9 Result . . . 63

10 Serendipity Element Families **65**
10.1 Why Serendipity? . . . 65
10.2 Mathematical Structure . . . 65
10.2.1 Tensor-product serendipity on the cube . . . 65
10.2.2 Why the tetrahedral “serendipity” family is different . . . 66
10.3 Implementation Structure . . . 66
10.3.1 Reference cells . . . 66
10.3.2 Concrete geometry . . . 67
10.3.3 ElementGeometry integration . . . 67
10.3.4 Quadrature choices . . . 67
10.4 VTK and Gmsh Support . . . 67
10.4.1 VTK . . . 67
10.4.2 Gmsh . . . 67
10.5 Validation Tests . . . 68
10.6 Benchmark Results . . . 68
10.7 Architectural Takeaways . . . 68
10.8 Future Work . . . 69

11 Domain Refactor Ontology **70**
11.1 Architectural Diagnosis . . . 70
11.2 Entity-by-Entity Semantics . . . 71
11.2.1 `Point<Dim>` . . . 71
11.2.2 `Vertex<Dim>` (proposed and introduced in this step) . . . 71
11.2.3 `Node<Dim, Storage>` . . . 71
11.2.4 `IntegrationPoint<dim>` . . . 72
11.2.5 `MaterialPoint<MaterialPolicy>` . . . 72
11.2.6 `SectionGeometry` . . . 72
11.2.7 `NodeSection<Dim, Geom>` . . . 73
11.2.8 `MaterialSection<MaterialPolicy>` . . . 73
11.2.9 `PetscSection` . . . 73

11.3 Coincidence Versus Identity . . . 73
11.3.1 Case 1: Gauss–Lobatto beam quadrature . . . 74
11.3.2 Case 2: mixed or assumed-strain formulations . . . 74
11.4 Proposed Ontology for the Refactor . . . 74
11.4.1 Layer 0: pure geometry/topology . . . 74
11.4.2 Layer 1: discrete field attachment . . . 75
11.4.3 Layer 2: sampling and constitutive sites . . . 75
11.4.4 Layer 3: structural section semantics . . . 75
11.5 Implications for `Domain` . . . 75
11.6 HPC and Compile-Time Guidance . . . 76
11.6.1 Prefer static structure in hot paths . . . 76
11.6.2 Keep `Domain` simple, not hyper-generic . . . 76
11.6.3 Avoid type collapse when roles overlap . . . 76
11.6.4 Recommended storage philosophy . . . 76
11.7 Migration Plan . . . 76
11.8 Conclusion . . . 77

12 Domain Vertex Transition **78**
12.1 Motivation . . . 78
12.2 Implemented Transition Design . . . 78
12.2.1 Canonical storage . . . 78
12.2.2 Node cache . . . 79
12.2.3 Cache invalidation rule . . . 79
12.3 Why This Preserves Performance . . . 79
12.3.1 No change to element hot loops . . . 79
12.3.2 Geometry-only consumers stop depending on nodes . . . 79
12.3.3 PETSc creation is now lazy . . . 80
12.4 Code-Level Changes . . . 80
12.5 What Remains Transitional . . . 80
12.6 Validation . . . 80
12.6.1 Domain/Vertex refactor test . . . 80
12.6.2 Regression coverage . . . 81
12.7 Architectural Consequences . . . 81
12.8 Next Step . . . 81
12.9 Conclusion . . . 81

13 Geometry-Only Element Binding **82**
13.1 Context . . . 82
13.2 Problem Statement . . . 82
13.3 Design Objective . . . 82
13.4 Implemented Refactor . . . 83
13.4.1 New dual binding inside concrete elements . . . 83
13.4.2 Wrapper-level support in `ElementGeometry` . . . 83
13.4.3 Key compatibility rule . . . 83
13.5 Domain-Level Consequences . . . 83
13.5.1 Geometry-first linking . . . 83
13.5.2 Boundary extraction no longer needs `node_p().id()` . . . 84
13.5.3 Sieve assembly remains analysis-oriented . . . 84
13.6 Why this is efficient . . . 84
13.6.1 No new runtime polymorphism in the hot path . . . 84
13.6.2 Compile-time impact . . . 84
13.7 Validation . . . 85

13.8 Architectural Interpretation . . . 85
13.9 Next Step . . . 85

14 Low-Level Geometry and DoF Access **86**
14.1 Motivation . . . 86
14.2 `Point<Dim>` as a contiguous data source . . . 86
14.2.1 Previous state . . . 86
14.2.2 Implemented change . . . 86
14.2.3 Why this matters . . . 86
14.3 `Node<Dim, Storage>` and direct DoF access . . . 87
14.3.1 Added direct pointer interface . . . 87
14.3.2 Array constructors . . . 87
14.4 Cached coordinate pointers inside geometric elements . . . 87
14.4.1 Problem . . . 87
14.4.2 Implemented solution . . . 87
14.4.3 Why this is safe . . . 88
14.5 Residual geometry-via-node usage in beam elements . . . 88
14.5.1 Observation . . . 88
14.5.2 Implemented change . . . 88
14.6 Validation and traceability . . . 88
14.7 Interpretation . . . 88

15 Spatial Sites and Lazy Domain Materialisation **89**
15.1 Scope of this pass . . . 89
15.2 The problem of overlapping point-like concepts . . . 89
15.3 A static ontology of spatial sites . . . 89
15.3.1 Read-only coordinate site: `PointViewT` . . . 89
15.3.2 Mutable point: `PointT` . . . 90
15.3.3 Entity-by-entity interpretation . . . 90
15.4 Why forwarding is the right model . . . 90
15.4.1 Material points and sections . . . 90
15.4.2 Node sections . . . 90
15.5 Lazy node materialisation in `Domain` . . . 91
15.5.1 Geometric operations should not force analysis state . . . 91
15.5.2 Implemented fix . . . 91
15.5.3 Why the node cache still exists . . . 91
15.6 Data-layout implications for HPC . . . 91
15.6.1 Owned storage vs forwarded storage . . . 91
15.6.2 Why no generic runtime interface was added . . . 92
15.7 What this means for overlapping formulations . . . 92
15.8 Validation . . . 92
15.9 Interpretation . . . 92

16 Adaptive Nodal Projection for Material Fields **93**
16.1 Why the Previous Diagnosis Was Incomplete . . . 93
16.2 Exact Row-Sum Lumping for TET10 . . . 93
16.2.1 Vertex nodes . . . 94
16.2.2 Edge-midpoint nodes . . . 94
16.3 Consequence: Lumped L^2 Is Not a Nodal Average . . . 94
16.4 Why HEX27 Remains Safe . . . 94
16.5 Adaptive Policy Implemented in `VTKModelExporter` . . . 95
16.6 Code-Level Traceability . . . 95

16.7 Verification . . . 96

17 Polynomial Patch Recovery for TET10 Visualisation 97
17.1 Why Averaging Was Not Enough . . . 97
17.2 Recovered Quantity and Patch Topology . . . 97
17.3 Centered and Scaled Basis . . . 98
17.4 Least-Squares System . . . 98
17.5 Rank-Adaptive Fallback . . . 98
17.6 Implementation Choices and HPC Considerations . . . 99
17.7 Regression Coverage . . . 99
17.8 Architectural Outcome . . . 99

18 Explicit Tensor Filters for VTK / ParaView 100
18.1 Naming Contract . . . 100
18.2 Stress Filters . . . 100
18.3 Strain Filters . . . 101
18.4 Why the Exporter Owns These Scalars . . . 101
18.5 Recommended ParaView Usage . . . 102

19 VTK Export Cleanup 103
19.1 Retained Responsibilities . . . 103
19.2 Removed Complexity . . . 103
19.3 Why the Split Export Is the Right Current Boundary . . . 103
19.4 Compile-Time and Dependency Impact . . . 104
19.5 Retained Extension Points . . . 104

20 Structural Scale Reconstruction and Multi-Block VTK Export 105
20.1 Motivation . . . 105
20.2 Architectural Split . . . 105
20.3 Spatial Roles and Ontology . . . 106
20.4 Constitutive Snapshots . . . 106
20.4.1 Beam Snapshot . . . 106
20.4.2 Shell Snapshot . . . 107
20.4.3 Fiber Snapshot . . . 107
20.5 Reduction Policies . . . 107
20.5.1 Why Policies and Not VTK-Specific Code . . . 107
20.5.2 3D Timoshenko Beam . . . 108
20.5.3 MITC4 Mindlin Shell . . . 108
20.6 Compile-Time and HPC Considerations . . . 108
20.7 Section Profiles . . . 109
20.7.1 Beam Profiles . . . 109
20.7.2 Shell Thickness Profile . . . 109
20.8 The Multi-Block `.vtm` Contract . . . 109
20.8.1 Executable Examples . . . 110
20.8.2 A Note on Empty Trailing Blocks . . . 110
20.9 Testing Strategy . . . 110
20.10 Limitations and Future Work . . . 110

21 Mixed Structural Table Example **112**
21.1 Purpose . . . 112
21.2 Physical Model . . . 112
21.2.1 Geometry . . . 112
21.2.2 Sections and Materials . . . 113
21.3 Loads and Boundary Conditions . . . 113
21.4 Architectural Realisation . . . 114
21.4.1 Why the Domain Contains Only the Slab . . . 114
21.4.2 Polymorphic Structural Container . . . 114
21.5 Quadrature Contract for `TimoshenkoBeamN` . . . 114
21.6 VTK Output . . . 115
21.7 Observed Structural Response . . . 115
21.8 Code Path . . . 116

22 Robustifying `DMPlex` for Naturally Mixed Element Families **117**
22.1 Motivation . . . 117
22.2 Design Decision . . . 117
22.3 Implemented Semantics . . . 117
22.4 Why `Depth` and Not `Height` . . . 118
22.5 Dimension of the `DMPlex` . . . 118
22.6 Compile-Time and HPC Considerations . . . 118
22.7 Validation . . . 119
22.8 Current Scope and Next Step . . . 119

23 Regular 5-Story Building Example **120**
23.1 Purpose . . . 120
23.2 Unit System . . . 120
23.3 Geometry . . . 121
23.3.1 Plan and height . . . 121
23.3.2 Discretisation . . . 121
23.4 Sections and Materials . . . 121
23.4.1 Columns . . . 121
23.4.2 Beams . . . 122
23.4.3 Slabs . . . 122
23.5 Loads and Boundary Conditions . . . 122
23.5.1 Boundary conditions . . . 122
23.5.2 Gravity loading . . . 122
23.5.3 Horizontal loading . . . 123
23.6 Architectural Realisation in Code . . . 123
23.6.1 One shared `Domain` . . . 123
23.6.2 Polymorphic structural container . . . 123
23.6.3 Why this example is a good architectural integrator . . . 124
23.7 Execution Results . . . 124
23.8 VTK Output . . . 125
23.9 Why the Example Matters . . . 125

24 Nonlinear Reinforced-Concrete 5-Story Building Example **126**
24.1 Purpose and Scope . . . 126
24.2 Unit System . . . 126
24.3 Geometry and Topology . . . 127
24.3.1 Plan and elevations . . . 127
24.3.2 Mixed structural topology . . . 127

24.4 Constitutive Design . . . 127
24.4.1 Columns . . . 127
24.4.2 Beams . . . 128
24.4.3 Slabs . . . 128
24.5 Why Fibers Belong in the Constitutive Layer . . . 128
24.6 Section Kinematics and Fiber Stress Recovery . . . 129
24.7 Global Solution Strategy . . . 129
24.7.1 Analysis-layer hardening . . . 129
24.8 Loads and Boundary Conditions . . . 130
24.9 Results of the Executed Example . . . 130
24.10 VTK Output . . . 130
24.11 Regression and Verification . . . 131
24.12 Current Limits and Next Steps . . . 131

25 Material Module Architecture Analysis 132
25.1 Purpose . . . 132
25.2 Current Architectural Layers . . . 132
25.2.1 Measure spaces . . . 132
25.2.2 Relation classes . . . 133
25.2.3 Material instance and type erasure . . . 133
25.2.4 Constitutive sites . . . 133
25.3 What is Good and Should Be Preserved . . . 134
25.3.1 The measure-based generic contract . . . 134
25.3.2 Static polymorphism at the relation level . . . 134
25.3.3 Separation between geometry and constitutive sites . . . 134
25.3.4 The reduction-based path for structural elements . . . 134
25.4 Observed Semantic Inconsistencies . . . 134
25.4.1 The word `Material` currently means too many things . . . 134
25.4.2 `MaterialPolicy` is not a material . . . 135
25.4.3 `MaterialInstance` is not just a material instance . . . 135
25.4.4 Inelastic relations already contain the integration algorithm . . . 135
25.4.5 The constitutive relation and its committed state are fused . . . 136
25.4.6 The kinematic state is duplicated and over-recorded . . . 136
25.4.7 The flyweight story is currently contradictory . . . 136
25.4.8 Section laws are currently mixed with material laws . . . 137
25.4.9 Post-processing adapters are embedded in the core wrapper . . . 137
25.4.10 Finite-strain constitutive models live outside the material module . . . 137
25.5 Performance Implications . . . 137
25.5.1 Per-fiber type erasure is expensive . . . 137
25.5.2 The default inelastic storage is not HPC-friendly . . . 138
25.5.3 The hot path should remain statically typed . . . 138
25.6 A Cleaner Separation: Material vs Constitutive Relation . . . 138
25.6.1 Recommended ontology . . . 138
25.6.2 What should be called “material” . . . 138
25.6.3 What should be called “constitutive law” . . . 139
25.6.4 What should be called “reduction policy” . . . 139
25.7 A More Scalable Constitutive Contract . . . 139
25.7.1 Constitutive context . . . 139
25.7.2 Suggested compile-time split . . . 139
25.8 Implications for Reinforced Concrete Across Scales . . . 140
25.8.1 Uniaxial constituent scale . . . 140
25.8.2 Layered shells . . . 140

25.8.3 Continuum RC with cracking and fracture . . . 140
25.8.4 Large displacements and dynamics . . . 141
25.9 Suggested Refactor Path . . . 141
25.9.1 Stage 1: semantic cleanup . . . 141
25.9.2 Stage 2: state redesign . . . 141
25.9.3 Stage 3: genuine algorithm-law separation . . . 141
25.9.4 Stage 4: typed hot path, erased boundary . . . 142
25.9.5 Stage 5: RC multiscale and fracture . . . 142
25.10 Non-owning Type Erasure in `Material.hh` . . . 142
25.10.1 Viability assessment . . . 142
25.10.2 What problem it solves . . . 142
25.10.3 What problem it does not solve . . . 143
25.10.4 Why the strategy belongs in `Material.hh` . . . 143
25.10.5 Refactor adopted in the current code . . . 143
25.10.6 Architectural consequence . . . 143
25.11 Final Opinion . . . 144

26 Material Module Semantic Cleanup **145**
26.1 Purpose . . . 145
26.2 What Was Changed . . . 145
26.2.1 Constitutive-space aliases . . . 145
26.2.2 Stateful constitutive-site aliases . . . 146
26.2.3 Erased constitutive handles . . . 146
26.2.4 Constitutive-site accessors . . . 146
26.3 Ownership Semantics Clarified . . . 146
26.4 Why This Stage Matters . . . 147
26.5 What Was Intentionally Not Changed . . . 147
26.6 Immediate Benefits . . . 147
26.7 Next Step . . . 147

27 Constitutive State Storage Cleanup **148**
27.1 Motivation . . . 148
27.2 Storage Roles . . . 148
27.2.1 CommittedState . . . 148
27.2.2 HistoryState . . . 149
27.2.3 TrialCommittedState . . . 149
27.3 MaterialState as a Storage Adapter . . . 149
27.4 Change in the Default Inelastic Path . . . 150
27.5 Interaction with Constitutive Updates . . . 150
27.6 Why This Matters for Reinforced Concrete Across Scales . . . 151
27.7 Next Refactor Stage . . . 151

28 ConstitutiveState as a Semantic Layer **152**
28.1 Why Another Layer Was Needed . . . 152
28.2 Layered View . . . 152
28.3 What ConstitutiveState Adds . . . 152
28.4 Effect on MaterialInstance . . . 153
28.5 Why This Matters for Future Nonlinear Algorithms . . . 153
28.6 Static Polymorphism Is Preserved . . . 154
28.7 What This Refactor Still Does Not Do . . . 154
28.8 Immediate Consequence . . . 154

29 ConstitutiveIntegrator as the Algorithmic Injection Point **155**
29.1 Motivation . . . 155
29.2 Local Architectural Split . . . 155
29.3 Why the Injection Point Stays in `Material<Policy>` . . . 155
29.4 Current Concrete Integrators . . . 156
29.5 Why This Matters . . . 156
29.6 Custom Integrators . . . 157
29.7 What Has Not Been Completed Yet . . . 157
29.8 Immediate Architectural Interpretation . . . 157

30 Explicit Algorithmic State in `PlasticityRelation` **158**
30.1 Why This Step Matters . . . 158
30.2 Old Situation . . . 158
30.3 New Explicit-State Contract . . . 158
30.4 Compatibility Is Preserved . . . 159
30.5 What Changed Internally . . . 159
30.6 Interaction with `MaterialInstance` . . . 159
30.7 Interaction with `ConstitutiveIntegrator` . . . 160
30.8 What the New Tests Prove . . . 160
30.9 Why the Cache Was Not Reused . . . 160
30.10 Current Boundary of the Refactor . . . 160
30.11 Next Logical Step . . . 161

Chapter 1

Simplex Finite Elements

This chapter describes the construction of Lagrangian finite elements on simplicial cells (triangles in 2D, tetrahedra in 3D) as implemented in `fall_n`. The development follows the classical Ciarlet definition [2] specialised to Lagrange interpolation on the reference simplex.

1.1 Reference Simplex

Definition 1.1 (Reference simplex)**.** The *reference simplex* in $\mathbb{R}^D$ is the convex hull

$$\mathcal{S}_D \;=\; \Big\{\boldsymbol{\xi} = (\xi_1, \ldots, \xi_D) \in \mathbb{R}^D \;\Big|\; \xi_i \geq 0,\; \sum_{i=1}^{D} \xi_i \leq 1\Big\}, \tag{1.1}$$

with *vertices*

$$\mathbf{v}_0 = \mathbf{0}, \qquad \mathbf{v}_i = \mathbf{e}_i \quad (i = 1, \ldots, D),$$

where $\mathbf{e}_i$ is the i-th standard basis vector.

The *barycentric coordinates* $(\lambda_0, \lambda_1, \ldots, \lambda_D)$ are defined as

$$\lambda_0 = 1 - \sum_{i=1}^{D} \xi_i, \qquad \lambda_i = \xi_i \quad (i = 1, \ldots, D). \tag{1.2}$$

They form a partition of unity, $\sum_{i=0}^{D} \lambda_i = 1$, and satisfy $\lambda_i \geq 0$ on $\mathcal{S}_D$.

The volume of the reference simplex is

$$|\mathcal{S}_D| = \frac{1}{D!}\,. \tag{1.3}$$

1.2 Lagrange Shape Functions

1.2.1 Linear Elements (Order 1)

The linear simplex has $D+1$ nodes (the vertices). The shape functions are simply the barycentric coordinates:

$$N_i(\boldsymbol{\xi}) = \lambda_i(\boldsymbol{\xi}), \qquad i = 0, 1, \ldots, D. \tag{1.4}$$

Since $\lambda_i \geq 0$ on $\mathcal{S}_D$ and $\lambda_i(\mathbf{v}_j) = \delta_{ij}$, these are the unique linear polynomials that interpolate the nodal values.

Concrete instances in `fall_n`:

- TRI3 ($D = 2$, 3 nodes): `SimplexElement<3,2,1>` (triangle in 3D ambient space, used for boundary faces)
- TET4 ($D = 3$, 4 nodes): `SimplexElement<3,3,1>`

1.2.2 Quadratic Elements (Order 2)

The quadratic simplex has

$$n_{\text{nodes}} = \binom{D+2}{2} = \frac{(D+1)(D+2)}{2} \tag{1.5}$$

nodes: the $D+1$ vertices plus $\binom{D+1}{2}$ edge midpoints.

Vertex shape functions. For vertex i ($i = 0, \ldots, D$):

$$\boxed{N_i(\boldsymbol{\xi}) = \lambda_i\,(2\lambda_i - 1)} \tag{1.6}$$

Remark 1.1 (Sign of vertex shape functions)**.** The vertex shape function $N_i = \lambda_i(2\lambda_i - 1)$ is *negative* whenever $0 < \lambda_i < \frac{1}{2}$. This region is non-empty in the interior of $\mathcal{S}_D$. In particular, interior quadrature points that are "far" from vertex i will have $\lambda_i \ll \frac{1}{2}$, making $N_i < 0$ at those points.

This fact is benign for stiffness matrix assembly (the bilinear form involves gradients, not function values). However, it becomes critical for lumped L^2 projection, as discussed in chapter 3.

Edge midpoint shape functions. For the midpoint of edge (i, j) with $0 \le i < j \le D$:

$$\boxed{N_{ij}(\boldsymbol{\xi}) = 4\,\lambda_i\,\lambda_j} \tag{1.7}$$

These are always non-negative on $\mathcal{S}_D$, since both $\lambda_i, \lambda_j \ge 0$.

Concrete instances:

- TRI6 ($D = 2$, 6 nodes): `SimplexElement<3,2,2>`
- TET10 ($D = 3$, 10 nodes): `SimplexElement<3,3,2>`

1.3 Shape Function Derivatives

Since $\lambda_0 = 1 - \xi_1 - \cdots - \xi_D$ and $\lambda_i = \xi_i$ for $i \ge 1$, the barycentric coordinate gradients (with respect to $\boldsymbol{\xi}$) are:

$$\frac{\partial \lambda_0}{\partial \xi_k} = -1, \qquad \frac{\partial \lambda_i}{\partial \xi_k} = \delta_{ik} \quad (i = 1, \ldots, D). \tag{1.8}$$

For the quadratic vertex shape function $N_i = \lambda_i(2\lambda_i - 1)$:

$$\frac{\partial N_i}{\partial \xi_k} = (4\lambda_i - 1)\,\frac{\partial \lambda_i}{\partial \xi_k}\,. \tag{1.9}$$

For the edge midpoint $N_{ij} = 4\lambda_i\lambda_j$:

$$\frac{\partial N_{ij}}{\partial \xi_k} = 4\Big(\lambda_j\,\frac{\partial \lambda_i}{\partial \xi_k} + \lambda_i\,\frac{\partial \lambda_j}{\partial \xi_k}\Big). \tag{1.10}$$

1.4 Isoparametric Mapping

The isoparametric map from the reference simplex $\mathcal{S}_D$ to a physical element $\Omega^e \subset \mathbb{R}^d$ (with $d \ge D$) is

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{I=1}^{n_{\text{nodes}}} N_I(\boldsymbol{\xi})\,\mathbf{x}_I, \tag{1.11}$$

where $\mathbf{x}_I \in \mathbb{R}^d$ are the physical nodal coordinates. The *Jacobian matrix* of the mapping is

$$J_{ij} = \frac{\partial x_i}{\partial \xi_j} = \sum_{I=1}^{n_{\text{nodes}}} \frac{\partial N_I}{\partial \xi_j} x_{I,i}\,, \qquad \mathbf{J} \in \mathbb{R}^{d\times D}. \tag{1.12}$$

When $d = D$ (volume elements), $\mathbf{J}$ is square and the differential volume element is $\mathrm{d}\Omega = |\det \mathbf{J}|\,\mathrm{d}\boldsymbol{\xi}$.

When $d > D$ (surface elements, e.g. TRI6 boundary faces in 3D), the surface measure is computed from the metric tensor:

$$\mathrm{d}S = \sqrt{\det(\mathbf{J}^T\mathbf{J})}\,\mathrm{d}\boldsymbol{\xi}. \tag{1.13}$$

Gradient transformation. The gradient of a scalar field in physical space is obtained from the reference-space gradient via

$$\frac{\partial N_I}{\partial x_j} = \sum_{k=1}^{D} \frac{\partial N_I}{\partial \xi_k} (J^{-1})_{kj}\,. \tag{1.14}$$

1.5 Subentity Topology: Faces of the Simplex

A D-simplex has $D+1$ faces (codimension-1 sub-simplices). The f-th face ($f = 0, \ldots, D$) is the sub-simplex *opposite* vertex f, spanned by all vertices *except* $\mathbf{v}_f$.

1.5.1 Face Local Node Ordering (TET10)

For a quadratic tetrahedron ($D = 3$, TET10, 10 nodes), each of the 4 triangular faces is a TRI6 sub-element. The face-to-global node mapping implemented in `SimplexCell<3,2>` is:

Face f	Opposite vertex	Global nodes
0	$\mathbf{v}_0$	(1, 2, 3, 5, 8, 7)
1	$\mathbf{v}_1$	(0, 3, 2, 6, 8, 5)
2	$\mathbf{v}_2$	(0, 1, 3, 4, 7, 6)
3	$\mathbf{v}_3$	(0, 2, 1, 5, 4, 6)

The outward normal direction follows from the right-hand rule applied to the first three (vertex) nodes of each face. This topology is critical for consistent surface traction application via Gauss quadrature on boundary faces.

1.6 Gmsh Node Ordering for TET10

Gmsh uses a different midpoint ordering than `fall_n` [6]. The Gmsh TET10 local ordering is:

Position	0	1	2	3	4	5	6	7	8	9
Gmsh	v_0	v_1	v_2	v_3	m_{01}	m_{12}	m_{02}	m_{03}	m_{13}	m_{23}
`fall_n`	v_0	v_1	v_2	v_3	m_{01}	m_{02}	m_{03}	m_{12}	m_{23}	m_{13}

The reordering applied in `GmshDomainBuilder.hh` maps Gmsh indices to `fall_n` indices via the permutation:

$$(0, 1, 2, 3, 4, 6, 7, 5, 9, 8). \tag{1.15}$$

This is implemented as:

```
index_ordering(node_tags, 0,1,2,3,4,6,7,5,9,8).data()
```

1.7 Implementation in `fall_n`

The simplex element hierarchy consists of three layers:

1. **`SimplexCell<D, Order>`** (`SimplexCell.hh`) — Purely geometric: reference coordinates, shape functions and their derivatives, face topology. All methods are `consteval` or `constexpr`, enabling compile-time evaluation.

2. **`SimplexElement<Dim, TopDim, Order>`** (`SimplexElement.hh`) — Combines a `SimplexCell` with an integration strategy. Template parameters:
   - `Dim`: ambient space dimension (e.g., 3 for 3D problems);
   - `TopDim`: topological dimension (D);
   - `Order`: Lagrange interpolation order.

3. **`ElementGeometry<ElemType, Integrator>`** (`ElementGeometry.hh`) — Manages the Jacobian, physical gradients, and integration; parameterised by element type and quadrature rule. The integrator is a duck-typed object exposing `num_integration_points`, `reference_integration_` and `weight(i)`.

This layered design allows the quadrature rule to be swapped independently of the element geometry, which was crucial for diagnosing and resolving the negative-weight problem (see chapters 2 and 3).

Chapter 2

Numerical Quadrature on Simplices

Numerical integration on simplicial domains is fundamentally different from integration on tensor-product cells (quadrilaterals, hexahedra). On tensor-product cells, one constructs n^D-point rules by taking the Cartesian product of n-point Gauss–Legendre rules in each direction. On simplices, no such natural product structure exists; special rules must be constructed [12, 10].

This chapter surveys the approaches available, documents the rules used in `fall_n`, and motivates the Stroud conical product as the preferred arbitrary-order method.

2.1 Problem Statement

We seek to approximate the integral

$$I[f] = \int_{\mathcal{S}_D} f(\boldsymbol{\xi})\,\mathrm{d}\boldsymbol{\xi} \approx \sum_{g=1}^{n_g} w_g\, f(\boldsymbol{\xi}_g), \tag{2.1}$$

where $\mathcal{S}_D$ is the reference D-simplex (1.1), with volume $|\mathcal{S}_D| = 1/D!$. A rule has *degree of exactness* p if $I[f]$ is computed exactly for all polynomials f of total degree $\leq p$.

Minimum requirements for finite elements. For a displacement-based finite element with polynomial order k and a constitutive law that is polynomial of degree m in the strains:

- Stiffness matrix: needs degree $\geq 2(k-1)m$ (often $2(k-1)$ for linear elasticity, where $m = 1$).
- Mass matrix (or L^2 projection): needs degree $\geq 2k$.
- For TET10 ($k = 2$, $D = 3$): stiffness needs degree ≥ 2; mass/projection needs degree ≥ 4.

2.2 Classical Symmetric Rules

2.2.1 Centroid Rule (Degree 1)

The single-point centroid rule:

$$\boldsymbol{\xi}_1 = \Big(\frac{1}{D+1}, \ldots, \frac{1}{D+1}\Big), \qquad w_1 = \frac{1}{D!}\,. \tag{2.2}$$

This integrates all linear polynomials exactly. It is adequate for constant- strain elements (TET4) when only the stiffness matrix is needed, but insufficient for quadratic elements.

2.2.2 Hammer–Marlowe–Stroud Rules (HMS)

The Hammer–Marlowe–Stroud [9] family provides low- order symmetric rules. The 4-point rule for the tetrahedron, used as the default for TET10 in `fall_n`, has:

n_g	Degree	Description
1	1	Centroid
4	2	Vertices of a smaller inscribed tetrahedron

The 4-point degree-2 rule uses barycentric coordinates:

$$\lambda = (\alpha, \beta, \beta, \beta) \quad \text{and permutations thereof,} \qquad \alpha = \frac{5-\sqrt{5}}{20},\ \beta = \frac{5+3\sqrt{5}}{20}. \tag{2.3}$$

All four weights are equal, $w_g = \frac{1}{4!} = \frac{1}{24}$, and are **strictly positive**.

Remark 2.1. In `fall_n`, this 4-point HMS rule is the *default* quadrature for TET10 elements (`default_simplex_rule<3,2>()` in `SimplexQuadrature.hh`). It was chosen over the 5-point Keast degree-3 rule precisely because all weights are positive — see chapter 3 for the detailed analysis.

2.2.3 Keast Rules

Keast [10] tabulated a series of tetrahedral rules up to degree 8. The most commonly cited are:

n_g	Degree	Min weight	Comment
1	1	$+$	Centroid
4	2	$+$	Same as HMS
5	3	$-$	$w_1 = -\frac{2}{15}$ at centroid
11	4	$+$	All positive
14	5	$+$	All positive
24	6	$-$	Contains negative weights

The 5-point degree-3 rule is particularly notable:

$$\text{Keast-5:} \quad w_1 = -\frac{2}{15}\,|\mathcal{S}_3| = -\frac{2}{15}\cdot\frac{1}{6} = -\frac{1}{45}, \qquad w_{2,\ldots,5} = \frac{3}{40}\,|\mathcal{S}_3| = \frac{1}{80}. \tag{2.4}$$

The centroid weight is *negative*. While this does not affect the accuracy of the quadrature rule (it still integrates degree-3 polynomials exactly), it causes severe problems in lumped L^2 projection on quadratic elements, as analysed in chapter 3.

2.2.4 Grundmann–Möller Rules

The Grundmann–Möller family [8] generates rules of arbitrary odd degree for any simplex dimension, using a combinatorial formula. The rule of index s has degree $2s+1$ with

$$n_g = \binom{D+s+1}{D+1}$$

points. Unfortunately, for $s \geq 1$ (degree ≥ 3), some weights are *guaranteed to be negative*. This makes Grundmann–Möller unsuitable as a general-purpose rule when positive weights are required.

2.3 Stroud Conical Product Rule

The Stroud conical product [12, 4] transforms the simplex integral into a product of 1D integrals via a *collapsed-coordinate* (Duffy) transformation, enabling the use of Gauss–Jacobi quadrature in each direction.

2.3.1 Duffy Transform

The transformation from *collapsed coordinates* $(t_1, \ldots, t_D) \in [0,1]^D$ to simplex coordinates $(\xi_1, \ldots, \xi_D) \in \mathcal{S}_D$ is defined recursively:

$$\boxed{\begin{aligned} &\xi_1 = t_1, \\ &\xi_k = t_k \prod_{j=1}^{k-1} (1 - t_j), \qquad k = 2, \ldots, D. \end{aligned}} \tag{2.5}$$

The Jacobian of this transformation is

$$\left| \frac{\partial \boldsymbol{\xi}}{\partial \mathbf{t}} \right| = \prod_{k=1}^{D-1} (1 - t_k)^{D-k}. \tag{2.6}$$

Therefore, the integral over the simplex becomes

$$\int_{\mathcal{S}_D} f(\boldsymbol{\xi}) \, \mathrm{d}\boldsymbol{\xi} = \int_0^1 \cdots \int_0^1 f(\boldsymbol{\xi}(\mathbf{t})) \prod_{k=1}^{D-1} (1 - t_k)^{D-k} \, \mathrm{d}t_1 \cdots \mathrm{d}t_D. \tag{2.7}$$

2.3.2 Product Quadrature with Gauss–Jacobi Nodes

The key insight is that each factor $(1 - t_k)^{D-k}$ depends only on t_k, so it can be *absorbed into* the 1D quadrature rule for direction k. Specifically, for direction k (0-indexed), we need a 1D quadrature rule on $[0,1]$ with weight function $(1-t)^{\beta_k}$, where

$$\beta_k = D - 1 - k, \qquad k = 0, 1, \ldots, D - 1. \tag{2.8}$$

- $k = 0$: $\beta = D - 1$ (maximal weight for the "lowest" direction).
- $k = D - 2$: $\beta = 1$ (linear weight).
- $k = D - 1$: $\beta = 0$ (plain Gauss–Legendre on $[0,1]$).

The integral (2.7) becomes

$$\int_{\mathcal{S}_D} f(\boldsymbol{\xi}) \, \mathrm{d}\boldsymbol{\xi} = \prod_{k=0}^{D-1} \left(\int_0^1 g_k(t_k) \, (1 - t_k)^{\beta_k} \, \mathrm{d}t_k \right), \tag{2.9}$$

where g_k encodes the dependence of f on t_k after integrating all other variables. Applying n-point Gauss–Jacobi quadrature on $[0,1]$ with weight $(1-t)^{\beta_k}$ to each direction gives a product rule with:

$$\boxed{n_g = n^D \text{ points}, \qquad \text{degree of exactness} = 2n - 1.} \tag{2.10}$$

Proposition 2.1 (All weights positive)**.** Since each 1D Gauss–Jacobi rule has strictly positive weights (the weight function $(1-t)^\beta$ is positive on $(0,1)$ for $\beta > -1$, and all Gauss–Jacobi weights are positive for any $\alpha, \beta > -1$), the product rule inherits positivity:

$$w_g = \prod_{k=0}^{D-1} w_{k,i_k} > 0 \qquad \forall \, g. \tag{2.11}$$

This is the fundamental advantage of the conical product over symmetric simplex rules.

2.3.3 Cost Comparison

Rule	Points	Degree	Weights
HMS 4-pt (TET)	4	2	All +
Keast 5-pt	5	3	**1 negative**
Keast 11-pt	11	4	All +
Conical $n=2$, $D=3$	8	3	All +
Conical $n=3$, $D=3$	27	5	All +
Conical $n=4$, $D=3$	64	7	All +

The conical product uses more points than optimal symmetric rules for the same degree, but it guarantees positivity and extends to *any* polynomial degree. For the TET10 stiffness matrix (degree ≥ 2), the 4-point HMS rule is sufficient and more economical; the conical product is available as a drop-in replacement when higher accuracy is needed.

2.4 Gauss–Jacobi Quadrature via the Golub–Welsch Algorithm

The 1D Gauss–Jacobi rules needed for the conical product are computed at library initialisation time using the Golub–Welsch algorithm [7].

2.4.1 Jacobi Polynomials

The Jacobi polynomials $P_n^{(\alpha,\beta)}(x)$ are orthogonal on $[-1,1]$ with respect to the weight function

$$w(x) = (1-x)^\alpha (1+x)^\beta, \qquad \alpha, \beta > -1. \tag{2.12}$$

They satisfy the three-term recurrence [13, 5]:

$$P_{n+1}^{(\alpha,\beta)}(x) = (A_n\, x + B_n)\, P_n^{(\alpha,\beta)}(x) - C_n\, P_{n-1}^{(\alpha,\beta)}(x), \tag{2.13}$$

where the coefficients A_n, B_n, C_n are explicit functions of n, α, β.

The *monic* Jacobi polynomials $\hat{p}_n$ satisfy

$$\hat{p}_{n+1}(x) = (x - a_n)\, \hat{p}_n(x) - b_n\, \hat{p}_{n-1}(x),$$

with recurrence coefficients [5]:

$$a_n = \frac{\beta^2 - \alpha^2}{(2n+\alpha+\beta)(2n+\alpha+\beta+2)}, \tag{2.14}$$

$$b_n = \frac{4n(n+\alpha)(n+\beta)(n+\alpha+\beta)}{(2n+\alpha+\beta)^2(2n+\alpha+\beta+1)(2n+\alpha+\beta-1)}. \tag{2.15}$$

2.4.2 The Golub–Welsch Algorithm

The Golub–Welsch algorithm [7] computes the n-point Gauss quadrature rule from the eigendecomposition of the *symmetric tridiagonal Jacobi matrix* $\mathbf{T} \in \mathbb{R}^{n\times n}$:

$$\mathbf{T} = \begin{pmatrix} a_0 & \sqrt{b_1} & & \\ \sqrt{b_1} & a_1 & \sqrt{b_2} & \\ & \ddots & \ddots & \ddots \\ & & \sqrt{b_{n-1}} & a_{n-1} \end{pmatrix}. \tag{2.16}$$

- The **eigenvalues** $\lambda_1, \ldots, \lambda_n$ of $\mathbf{T}$ are the *quadrature nodes* (zeros of the degree-n monic Jacobi polynomial).

- The **weights** are $w_i = \mu_0 \, v_{1,i}^2$, where $v_{1,i}$ is the first component of the i-th normalised eigenvector and μ_0 is the zeroth moment of the weight function:

$$\mu_0 = \int_{-1}^{1} (1-x)^\alpha (1+x)^\beta \, \mathrm{d}x = \frac{2^{\alpha+\beta+1} \, \Gamma(\alpha+1) \, \Gamma(\beta+1)}{\Gamma(\alpha+\beta+2)}. \tag{2.17}$$

Remark 2.2 (Why Golub–Welsch instead of Newton iteration)**.** An earlier implementation attempted to find the Jacobi polynomial zeros via a Newton iteration with Tricomi-type initial guesses. This approach failed for the $\alpha = D-1$ cases needed by the conical product (e.g., $\alpha = 2$ for $D = 3$, direction $k = 0$): the initial guesses were too inaccurate, and the iteration converged to *duplicate roots* instead of distinct nodes.

The Golub–Welsch algorithm is unconditionally robust: it requires no initial guesses and produces all nodes and weights simultaneously from a single eigenvalue decomposition. It is implemented using Eigen's `SelfAdjointEigenSolver`, which is $O(n^2)$ for the tridiagonal case.

2.4.3 Mapping to $[0, 1]$

The Gauss–Jacobi rule on $[-1, 1]$ with weight $(1-x)^\alpha$ is mapped to $[0, 1]$ via $x = 2t - 1$:

$$\int_0^1 f(t) \, (1-t)^\beta \, \mathrm{d}t = \frac{1}{2^{\beta+1}} \int_{-1}^{1} f\left(\frac{1+x}{2}\right) (1-x)^\beta \, \mathrm{d}x. \tag{2.18}$$

The nodes and weights transform as

$$t_i = \frac{1+x_i}{2}, \qquad w_i^{[0,1]} = \frac{w_i^{[-1,1]}}{2^{\beta+1}}. \tag{2.19}$$

This is implemented in `gauss_jacobi_01()` in `StroudConicalProduct.hh`.

2.5 Implementation: `StroudConicalProduct.hh`

The implementation resides in `src/numerics/numerical_integration/StroudConicalProduct.hh` and consists of three layers:

1. **`gauss_jacobi(n, α, β)`** — Returns an n-point Gauss–Jacobi rule on $[-1, 1]$ via Golub–Welsch. Uses Eigen's `SelfAdjointEigenSolver` on the tridiagonal Jacobi matrix.

2. **`stroud_conical_product<D>(n_per_dir)`** — Builds D one-dimensional Gauss–Jacobi rules (with $\beta_k = D - 1 - k$), takes their tensor product in collapsed coordinates, and maps to simplex coordinates via the Duffy transform (2.5).

3. **`ConicalProductIntegrator<TopDim, NPerDir>`** — Compile-time-sized integrator class. The rule is computed once at program startup (static `inline const` initialiser) and stored in a fixed-size `std::array`. Exposes the same interface as all other integrators in `fall_n`:
   - `static constexpr num_integration_points = NPerDir`$^{\texttt{TopDim}}$
   - `reference_integration_point(i)` returns a `std::span<const double>`
   - `weight(i)` returns w_i
   - `operator()(f)` evaluates $\sum w_i f(\boldsymbol{\xi}_i)$

2.6 Default Quadrature Choices in `fall_n`

Element	Rule	Points	Degree	Source
TET4	HMS 1-pt	1	1	`default_simplex_rule<3,1>()`
TET10	HMS 4-pt	4	2	`default_simplex_rule<3,2>()`
TRI3	HMS 1-pt	1	1	`default_simplex_rule<2,1>()`
TRI6	HMS 3-pt	3	2	`default_simplex_rule<2,2>()`
HEX27	Gauss 3^3	27	5	Tensor-product Gauss–Legendre

The conical product integrators (`ConicalProductIntegrator<3,2>`, `ConicalProductIntegrator<3,3>`, etc.) are available as drop-in replacements when higher-degree or higher-accuracy integration is needed, with the guarantee that all weights remain positive.

Chapter 3

Negative Weights and the Checkerboard Artefact

This chapter documents the diagnosis and resolution of a persistent *checkerboard* (or "diamond") artefact observed in nodal stress fields projected from Gauss points onto quadratic tetrahedral elements (TET10). The analysis traces the root cause to the interaction between negative quadrature weights and shape function values, and prescribes two complementary fixes.

3.1 Symptom: Element-Level Checkerboard Pattern

When visualising projected stress or strain fields in ParaView on TET10 meshes, the following pathology was observed:

- At *vertex* nodes, the projected field values had the *opposite sign* of the expected physical values.
- At *edge midpoint* nodes, the values were correct.
- The pattern repeated *at every element*, forming a regular alternating (checkerboard) structure.
- On HEX27 meshes, the same analysis pipeline produced smooth, correct fields.

3.2 Root Cause Analysis

The projected nodal values were computed via *lumped L^2 projection*:

$$\tilde{\sigma}_I = \frac{\displaystyle\sum_{e \in \mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|\, \sigma_g}{\displaystyle\sum_{e \in \mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|}, \tag{3.1}$$

where $\mathcal{E}_I$ is the set of elements sharing node I, N_I is the global shape function, and σ_g is the Gauss-point value.

For the projection (3.1) to produce meaningful results, the denominator must be **strictly positive** for every node. We will show that this condition *fails* at vertex nodes of TET10 when the Keast 5-point rule is used.

3.2.1 The Keast 5-Point Rule Weights

The Keast degree-3 rule for the tetrahedron has 5 points (section 2.2.3):

$$\begin{aligned} &\text{Point 1 (centroid):} && w_1 = -\frac{2}{15}\,|\mathcal{S}_3| = -\frac{1}{45} \approx -0.02222, \\ &\text{Points 2–5 (vertices):} && w_g = +\frac{3}{40}\,|\mathcal{S}_3| = +\frac{1}{80} = 0.01250. \end{aligned} \tag{3.2}$$

The centroid weight w_1 is **negative**.

3.2.2 TET10 Vertex Shape Functions at Quadrature Points

The vertex shape function for node i is (cf. eq. (1.6)):

$$N_i(\boldsymbol{\xi}) = \lambda_i\,(2\lambda_i - 1).$$

At the 5 Keast quadrature points:

At the centroid ($\lambda_i = 1/4$ for all i):

$$N_i = \tfrac{1}{4}(2 \cdot \tfrac{1}{4} - 1) = \tfrac{1}{4}(-\tfrac{1}{2}) = -\tfrac{1}{8}.$$

At the vertex-orbit points: Each vertex-orbit point has one "dominant" barycentric coordinate near 0.5854 and three "small" ones near 0.1382.

For the vertex i that is *dominant* at point g:

$$N_i \approx 0.5854 \times (2 \times 0.5854 - 1) = 0.5854 \times 0.1708 \approx 0.1000.$$

For a vertex i that is *not dominant*:

$$N_i \approx 0.1382 \times (2 \times 0.1382 - 1) = 0.1382 \times (-0.7236) \approx -0.1000.$$

3.2.3 The Denominator Catastrophe

Consider the denominator of (3.1) for a *vertex* node I, restricted to a single element:

$$D_I = \sum_{g=1}^{5} N_I(\boldsymbol{\xi}_g)\,w_g\,|J_g|. \tag{3.3}$$

Assuming approximately constant $|J_g| \approx |J|$ within the element:

$$D_I \approx |J|\big[\underbrace{N_I(\boldsymbol{\xi}_1)\,w_1}_{\text{centroid: } (-1/8)(-1/45)>0} + N_I(\boldsymbol{\xi}_2)\,w_2 + \cdots + N_I(\boldsymbol{\xi}_5)\,w_5\big]. \tag{3.4}$$

The centroid contribution is *positive* (negative times negative), but the vertex-orbit contributions produce a mix of positive and negative terms. The crucial issue is that at the three vertex-orbit points where $N_I < 0$ (because $\lambda_I < 1/2$), the *positive* weight $w_g > 0$ creates a negative contribution, while at the one vertex-orbit point where $N_I > 0$ (the "own" vertex direction), the positive weight creates a positive contribution.

When these are summed, the negative contributions can *overwhelm* the positive ones, making $D_I < 0$ or $D_I \approx 0$.

3.2.4 Consequence: Sign Flip or Division by Zero

$$D_I \le 0 \implies \tilde{\sigma}_I = \frac{(\text{numerator})}{D_I} \begin{cases} \text{has wrong sign} & \text{if } D_I < 0, \\ \text{diverges} & \text{if } D_I = 0. \end{cases} \tag{3.5}$$

In practice, the code included a guard $|D_I| < \epsilon \implies \tilde{\sigma}_I = 0$. This zeroed out vertex values while midpoint values (which have $N_{ij} = 4\lambda_i\lambda_j \ge 0$ everywhere) remained correct. The result was the characteristic checkerboard pattern:

Node type	Denominator D_I	Projected value
Vertex	≤ 0 or ≈ 0	0 or wrong sign
Midpoint	> 0	Correct

3.3 Why HEX27 Is Unaffected

HEX27 uses the tensor-product $3 \times 3 \times 3$ Gauss–Legendre rule (27 points), which has **all positive weights**. Furthermore, the Lagrangian shape functions on the reference cube $[-1, 1]^3$ are products of 1D Lagrange polynomials, and while they can go negative, they do so in a balanced way that keeps the lumped denominator positive. The combination of all-positive weights and the tensor-product structure prevents the denominator catastrophe.

3.4 Resolution: Two Complementary Fixes

Two independent corrections were applied:

3.4.1 Fix 1: Switch to an All-Positive-Weight Quadrature Rule

The root cause is the negative weight $w_1 = -2/15\,|\mathcal{S}_3|$ in the Keast 5-point rule. Replacing it with the **HMS 4-point degree-2 rule**, which has all positive weights (section 2.2.2), eliminates the negative-weight pathway entirely.

In `SimplexQuadrature.hh`, the default for TET10 was changed:

```
template<>
inline constexpr auto default_simplex_rule<3, 2>() {
    // Was: make_simplex_rule_3d_5()  [Keast 5-pt, w1 = -2/15]
    // Now: make_simplex_rule_3d_4()  [HMS 4-pt, all w > 0]
    return make_simplex_rule_3d_4();
}
```

Remark 3.1. The HMS 4-point rule has degree 2, one degree lower than the Keast 5-point rule (degree 3). For the TET10 stiffness matrix, degree 2 is the minimum requirement ($2(k-1) = 2$ for $k = 2$), so accuracy is maintained for linear elasticity. For the mass matrix or high-order nonlinear constitutive models, the conical product (`ConicalProductIntegrator<3,2>`, 8 points, degree 3) is available as a higher-degree drop-in replacement with all positive weights.

3.4.2 Fix 2: Guard Against Negative Denominators

As a defensive measure, the lumped L^2 projection code was modified to use the absolute value of the denominator:

```
denominator = std::abs(raw_denominator);
if (denominator < epsilon) denominator = epsilon;
```

This ensures that even if a future quadrature rule with negative weights is inadvertently used, the projection produces a *smoothed* (not sign-flipped) field. The `std::abs` guard does not affect the result when all weights are positive, since the denominator is then guaranteed positive.

3.4.3 Fix 3 (Recommended): SPR Volume-Weighted Averaging

The most robust solution is to bypass lumped L^2 projection entirely and use **volume-weighted nodal averaging** — a simplified form of the Superconvergent Patch Recovery (SPR) technique [14]. This is discussed in detail in chapter 4.

3.5 Summary

The checkerboard artefact results from the interaction of three factors:

1. **Negative quadrature weight** in the Keast 5-point rule ($w_1 = -2/15\,|\mathcal{S}_3|$).
2. **Negative vertex shape functions** of TET10 at interior quadrature points ($N_i = \lambda_i(2\lambda_i - 1) < 0$ when $\lambda_i < 1/2$).
3. **Lumped L^2 projection** with a denominator that sums signed products $N_I \cdot w_g$, allowing cancellation to zero or sign reversal at vertex nodes.

Any one of these factors being absent would prevent the artefact. The chosen resolution addresses all three:

- Factor 1: eliminated by switching to HMS 4-point (all $w_g > 0$);
- Factor 3: eliminated by switching to SPR averaging (no denominator subtraction possible).
- Factor 2: is intrinsic to quadratic elements and cannot be changed, but it is rendered harmless by the other two fixes.

Chapter 4

Gauss-Point to Nodal Projection

In displacement-based finite elements, the primary unknowns are nodal. The derived fields of engineering interest — strains, stresses, and internal variables — are instead evaluated at Gauss points. For VTK/ParaView visualisation, those material fields must be transferred to the mesh nodes.

`fall_n` currently implements three nodal transfer operators:

1. lumped L^2 projection;
2. polynomial patch recovery (the production fallback for higher-order simplex elements);
3. volume-weighted averaging (a deliberately low-order but very robust explicit fallback).

The adaptive default is now:

use lumped L^2 only when every local nodal lump is strictly positive; otherwise use polynomial patch recovery.

4.1 Lumped L^2 Projection

4.1.1 Consistent L^2 Projection

Let σ^h be the field known at Gauss points and let $\tilde{\sigma}^h = \sum_I \tilde{\sigma}_I N_I$ be its nodal projection. The consistent L^2 projection solves:

$$\mathbf{M}\,\tilde{\boldsymbol{\sigma}} = \mathbf{f}, \tag{4.1}$$

with

$$M_{IJ} = \int_\Omega N_I N_J \,\mathrm{d}\Omega, \qquad f_I = \int_\Omega N_I \sigma^h \,\mathrm{d}\Omega. \tag{4.2}$$

This is mathematically clean but operationally unattractive for post-processing: it introduces a global solve whose cost is disproportionate to the visualisation task.

4.1.2 Row-Sum Lumping

The classical shortcut is to lump the mass matrix by row sum:

$$\tilde{M}_{II} = \sum_J M_{IJ} = \int_\Omega N_I \sum_J N_J \,\mathrm{d}\Omega = \int_\Omega N_I \,\mathrm{d}\Omega. \tag{4.3}$$

Using quadrature,

$$\tilde{\sigma}_I \approx \frac{\displaystyle\sum_{e\in\mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|\, \sigma_g}{\displaystyle\sum_{e\in\mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|}. \tag{4.4}$$

This behaves as a nodal average only if every denominator is strictly positive. For higher-order simplex elements that condition may fail even under all-positive quadrature, because the exact lumped nodal measure $\int_\Omega N_I \, \mathrm{d}\Omega$ can itself be signed. The TET10 vertex nodes are precisely such a case, as shown in chapter 16.

Status in `fall_n`. Lumped L^2 remains the preferred projection when it is mathematically sound. For element families such as HEX27, where the local lumped nodal measures stay strictly positive under the chosen quadrature, it yields the smoothest and least diffusive nodal field at essentially zero additional complexity.

4.2 Polynomial Patch Recovery

4.2.1 Motivation

When lumped L^2 stops being a true average, the right replacement is not a coarser averaging operator but a *local reconstruction* operator. The classical Superconvergent Patch Recovery (SPR) method of Zienkiewicz and Zhu [14, 15] fits a polynomial over the patch of elements surrounding a node and evaluates that polynomial at the node. For `TET10` this is the correct architectural move: the Gauss point data are smooth, but the nodal transfer operator must stop behaving like signed lumping.

4.2.2 Patch Definition

For a node I, define its patch as the union of elements that contain that node:

$$\mathcal{P}_I = \{\, e \in \mathcal{E} \mid I \in e \,\}. \tag{4.5}$$

All Gauss points of all elements in $\mathcal{P}_I$ are used as sample sites. If the patch contributes observations $\{(\mathbf{x}_g, \sigma_g, \omega_g)\}_{g=1}^{n_p}$, we seek a polynomial $p_I(\mathbf{x})$ that best fits the data in weighted least squares.

4.2.3 Local Polynomial Basis

To improve conditioning, the implementation uses centered and scaled local coordinates around the evaluation node $\mathbf{x}_I$:

$$\hat{\mathbf{x}}_g = \frac{\mathbf{x}_g - \mathbf{x}_I}{h_I}, \qquad h_I = \max_{g \in \mathcal{P}_I} \|\mathbf{x}_g - \mathbf{x}_I\|_2. \tag{4.6}$$

For three-dimensional continuum elements the quadratic basis is

$$\mathbf{p}(\hat{\mathbf{x}}) = \begin{bmatrix} 1 & \hat{x} & \hat{y} & \hat{z} & \hat{x}^2 & \hat{x}\hat{y} & \hat{x}\hat{z} & \hat{y}^2 & \hat{y}\hat{z} & \hat{z}^2 \end{bmatrix}^\mathsf{T}. \tag{4.7}$$

The implementation uses the corresponding reduced bases in 1D and 2D.

4.2.4 Weighted Least Squares

The coefficients $\mathbf{a}_I$ are obtained from

$$\mathbf{a}_I = \arg\min_{\mathbf{a}} \sum_{g=1}^{n_p} \omega_g \left(\mathbf{p}(\hat{\mathbf{x}}_g)^\mathsf{T} \mathbf{a} - \sigma_g \right)^2. \tag{4.8}$$

The normal equations are

$$\left(\sum_g \omega_g \, \mathbf{p}_g \mathbf{p}_g^\mathsf{T} \right) \mathbf{a}_I = \sum_g \omega_g \, \mathbf{p}_g \sigma_g. \tag{4.9}$$

In the current implementation, the weights are

$$\omega_g = |w_g J_g|, \tag{4.10}$$

that is, the *magnitude* of the quadrature measure. This is intentional: for patch fitting the weight is a confidence/importance measure for a sample, not a signed coefficient of an integral approximation. Therefore negative quadrature weights must not flip the sign of the least-squares metric.

4.2.5 Adaptive Degree Fallback

Boundary patches or coarse patches may not contain enough independent samples to support a quadratic fit. The exporter therefore applies a strict hierarchy:

1. try a quadratic polynomial;
2. if the patch is rank-deficient, try a linear polynomial;
3. if the patch is still insufficient, use a weighted average of the patch samples.

This fallback is important for robustness near boundaries and corners. It keeps the high-order path where the local data support it, without making the visualisation pipeline brittle.

4.2.6 Evaluation at the Node

Because the polynomial basis is centered at $\mathbf{x}_I$, the recovered nodal value is simply the constant coefficient:

$$\tilde{\sigma}_I = p_I(\mathbf{x}_I) = a_{I,0}. \tag{4.11}$$

This is why the implementation can recover the nodal value directly after the solve of the local least-squares system.

4.3 Volume-Weighted Averaging

Volume-weighted averaging is retained as an explicit low-order fallback:

$$\tilde{\sigma}_I = \frac{\sum\limits_{e \in \mathcal{E}_I} V_e \, \bar{\sigma}_e}{\sum\limits_{e \in \mathcal{E}_I} V_e}, \tag{4.12}$$

where

$$\bar{\sigma}_e = \frac{\sum_g w_g |J_g| \sigma_g}{\sum_g w_g |J_g|}. \tag{4.13}$$

This operator is deliberately diffusive. Its purpose is no longer to serve as the default production path, but to remain available when a user wants the most conservative possible smoothing or when investigating pathological patches.

4.4 Implementation in `VTKModelExporter`

The implementation lives in `src/post-processing/VTK/VTKModelExporter.hh`. The key pieces are:

`l2_project_to_nodes()` Implements lumped L^2 projection.

`patch_recover_to_nodes()` Builds nodal patches from element-to-node connectivity, gathers Gauss-point samples, and performs centered/scaled least-squares recovery.

`spr_average_to_nodes()` Retained as the explicit volume-weighted fallback path.

`compute_material_fields()` Resolves the strategy automatically:

1. if every element has strictly positive local lumped nodal weights, use lumped L^2;
2. otherwise use polynomial patch recovery.

`compute_material_fields_patch()` Exposes polynomial patch recovery explicitly.

`compute_material_fields_spr()` Preserves the old volume-average path as an explicit user choice.

Computational structure. The hot finite-element kernels are untouched. All additional work is confined to post-processing and uses only geometry already available in the element wrappers. This preserves the compile-time and inlining properties of the assembly path while upgrading the quality of the visualisation path.

4.5 Comparison of the Three Operators

Property	Lumped L^2	Patch recovery	Volume average
Requires positive local lumps	yes	no	no
Exactness on linear fields	yes	yes	no
Exactness on quadratic fields	family-dependent	patch-dependent	no
Continuity of rendered field	good when safe	good	fair / diffusive
Sensitivity to signed simplex lumps	high	none	none
Additional local solves	none	yes	none
Intended use	best smooth path when safe	adaptive default for simplex	conservative fallback

4.6 Verification

The implementation was verified at two levels.

4.6.1 Algebraic regression

The simplex test suite now checks:

- TET10 retains signed lumped nodal measures even under the positive-weight Hammer–Marlowe–Stroud rule and under the positive Stroud conical-product rule;
- polynomial patch recovery reproduces an exact linear field on a single TET10, which exercises the quadratic→linear fallback;
- polynomial patch recovery reproduces an exact quadratic scalar field on a patch of multiple TET10 elements.

4.6.2 Application-level behaviour

On the cantilever benchmark:

- **HEX27**: the adaptive resolver chooses lumped L^2, and the resulting field remains smooth as before.
- **TET10**: the adaptive resolver now chooses polynomial patch recovery instead of volume averaging, eliminating the signed-lumping artefact without introducing the excessive diffusion of the old fallback.

This is the important practical conclusion:

> The constitutive update and the element kinematics were already correct. The remaining ParaView artefact came from the nodal transfer operator, and the appropriate cure was a better local reconstruction, not a change in the stiffness quadrature.

Chapter 5

PETSc Patch: `section.c` Indexing Bug

This chapter documents a bug in PETSc 3.24.5 [1] that affects higher-order finite elements whose DMPlex point numbering does not start at zero. The bug was discovered during the development of simplex element support in `fall_n` and confirmed to be an upstream issue.

5.1 Context: DMPlex and PetscSection

PETSc's `DMPlex` represents an unstructured mesh as a directed acyclic graph (DAG) of *points*: vertices, edges, faces, and cells, numbered from 0 to $N - 1$ (though the numbering of each *stratum* may start at a non-zero offset).

A `PetscSection` associates degrees of freedom (DOFs) with mesh points. It uses an *atlas* (`s->atlasDof[]`) indexed relative to the chart [`pStart`, `pEnd`), and a *constraint* mechanism to impose Dirichlet boundary conditions by specifying which DOFs at a point are constrained.

The function `PetscSectionSetConstraintIndices()` sets the constrained DOF indices at a given point. Its implementation reads:

```
PetscCall(PetscSectionCheckConstraints_Private(s));
/* Bug: should subtract pStart to get the atlas offset */
if (s->atlasDof[point]) {                // <-- HERE
    PetscInt cdof;
    PetscCall(PetscSectionGetConstraintDof(s, point, &cdof));
    // ...
}
```

5.2 The Bug

In `src/sys/utils/section.c`, at approximately line 2997, the code reads:

```
if (s->atlasDof[point]) {
```

but the atlas array is indexed relative to `pStart`, so the correct access is:

```
if (s->atlasDof[point - s->pStart]) {
```

When `pStart = 0` (as happens for vertex stratum when vertices are numbered from zero), the two expressions are identical and the bug is invisible. However, when a mesh's vertex stratum starts at a non-zero offset — which occurs in `fall_n` because Gmsh-imported meshes place cells before vertices in the DAG, giving the vertex stratum `pStart` = 8 (for 8 hex cells) or larger — the raw `point` index overflows the atlas array, reading uninitialised memory.

5.3 Manifestation in `fall_n`

The bug manifests during the application of Dirichlet boundary conditions via `PetscSectionSetConstraintInd`
Specifically:

- For HEX27 meshes with 20 cells, the vertex stratum starts at `pStart` ≥ 20.
- When setting constraints on vertex DOFs, the index `s->atlasDof[point]` reads from memory at offset 20+ instead of offset 0+.
- The read value is unpredictable: it may be zero (causing the constraints to be silently skipped) or non-zero (proceeding correctly by accident).
- The symptom was a segmentation fault or incorrect constraint application on some meshes but not others, depending on memory layout.

5.4 The Fix

The fix is a one-line change in `src/sys/utils/section.c`:

```
// Before (buggy):
if (s->atlasDof[point]) {

// After (fixed):
if (s->atlasDof[point - s->pStart]) {
```

This was applied to the local PETSc installation at:

```
/home/sechavarriam/MyLibs/petsc-3.24.5/src/sys/utils/section.c
```

After applying the patch, PETSc was rebuilt and all `fall_n` constraint tests passed on both HEX27 and TET10 meshes.

5.5 Verification

The fix was verified by:

1. Running the debug output in `fall_n`'s boundary condition application code, which prints:

   ```
   [SIEVE-DBG] section_ptr=0x... chart=[9152,23629)
   [SIEVE-DBG] All 14477 section DOFs OK
   [BC-DBG] Constraints count: 145
   [BC-DBG] plex_id=9152 section_dof=3 constraint_size=3
     ...
   ```

   confirming that all 145 Dirichlet-constrained vertices are correctly handled with `pStart` $= 9152$.
2. Comparing the Case 1 (SmallStrain + Linear Elastic) displacement field on HEX27 against the expected Timoshenko beam solution: the tip displacement matches the analytical result to within the expected discretisation error.
3. Running all 7 nonlinear cases on both HEX27 and TET10 meshes without segfaults or constraint errors.

5.6 Upstream Status

As of PETSc 3.24.5, this bug is present in the release source. It has been reported to the PETSc developers. Since the bug only affects codes that:

- use `PetscSectionSetConstraintIndices` directly (rather than through higher-level `DMPlexSetBoundary`
  and
- have a `PetscSection` chart with `pStart` > 0,

it may not have been triggered by the PETSc test suite, which may use meshes with vertex-first ordering (`pStart` = 0 for the vertex stratum).

5.7 Workaround from Application Code

In parallel with the upstream patch, we discovered two application-level workarounds that avoid modifying PETSc source entirely. Both were implemented in `fall_n` and verified on all mesh types.

5.7.1 Workaround 1: Extend the Section Chart

The root cause is that `PetscSectionSetConstraintIndices()` accesses `s->atlasDof[point]` without subtracting `s->pStart`, so it reads out of bounds whenever the chart does not start at zero.

Symptom. For the TET4 mesh (9152 elements, 2119 nodes), the `DMPlex` point numbering places cells at $[0, 9152)$ and vertices at $[9152, 11271)$. When the section chart is set to the vertex stratum $[\texttt{vStart}, \texttt{vEnd}) = [9152, 11271)$, the atlas has only $11271 - 9152 = 2119$ entries. The buggy code accesses index 9152 into a 2119-entry array — a severe out-of-bounds read.

The read value is unpredictable: it reports `dof = 123` (instead of 3) for constrained nodes, producing 604 PETSc error lines of the form:

```
[0]PETSC ERROR: Point 9152 dof 123,
    invalid constraint index[9]: 24587
```

More critically, the constraint indices are *not applied*, and the system produces a singular matrix (all Dirichlet DOFs are unconstrained), giving `|u| = 0.000000` for TET4.

Fix. In `Model::set_sieve_layout()`, set the section chart to cover the full DM chart $[\texttt{pStart}, \texttt{pEnd})$ instead of only the vertex stratum $[\texttt{vStart}, \texttt{vEnd})$:

```
// Before (triggers PETSc bug when vStart > 0):
PetscSectionSetChart(dof_section, vStart, vEnd);

// After (workaround: atlas covers all points):
PetscSectionSetChart(dof_section, pStart, pEnd);
```

With the full chart, `pStart = 0` always, so the buggy raw index `s->atlasDof[point]` coincides with the correct adjusted index `s->atlasDof[point - 0]`. Element points (cells) simply carry 0 DOFs.

The cost is a marginally larger atlas array (8 bytes per cell point), which is negligible relative to the mesh data.

Source. `src/model/Model.hh`, method `set_sieve_layout()`.

5.7.2 Workaround 2: Allow Dynamic Matrix Allocation

A secondary class of errors surfaces on large unstructured TET4 meshes: PETSc reports thousands of *"New nonzero at (i,j) caused a malloc"* messages during matrix assembly. These occur because `DMCreateMatrix()`'s internal adjacency walk (`DMPlexPreallocateOperator`) underestimates the sparsity pattern for cell-vertex sieves of unstructured simplex meshes.

Diagnosis. On the TET4 mesh (9152 elements, 2119 nodes), analysis confirmed that error DOF pairs correspond to nodes that *do not share any element* in the mesh. For example, node 172 (9 neighbours in the mesh graph) was reported as needing an entry with node 1358 — with zero shared elements. This is a PETSc preallocation-algorithm issue, not a `fall_n` sieve-setup error.

The results were still correct because PETSc dynamically allocates the missing entries (just slower, with error messages in debug builds).

Fix. After each `DMCreateMatrix()` call, tell PETSc to silently allocate missing entries instead of reporting errors:

```
DMCreateMatrix(dm, &K);
MatSetOption(K, MAT_NEW_NONZERO_ALLOCATION_ERR, PETSC_FALSE);
```

This was applied to all matrix creation sites:

File	Method	Matrix
`Model.hh`	`setup_matrix()`	`Kt_`
`LinearAnalysis.hh`	`setup_matrix_sizes()`	`K`
`NLAnalysis.hh`	SNES Jacobian callback	`J`
`DynamicAnalysis.hh`	`setup()`	`M_, C_, J_`
`DynamicAnalysis.hh`	`assemble_initial_stiffness`	`K0_`

5.7.3 Verification

After applying both workarounds, `fall_n` runs with **zero** PETSc error messages on all four mesh types. The results table:

Model	Nodes	Elems	$\lvert u\rvert_{\max}$	$\lvert u_y\rvert_{\max}$	Error (%)
Timoshenko $N = 3$ (ref.)	11	5	6.4518	6.2578	0.00
HEX8 (30×6×3)	868	540	4.8293	4.8293	25.15
HEX27 (30×6×3)	5551	540	6.2148	6.2148	3.67
TET4 (Fine, 4-pt)	2119	9152	3.5079	3.5079	45.63
TET10 (Fine, 27-pt)	4914	2475	6.2173	6.2173	3.64

The TET4 result (3.51, 45.6% error) now correctly reflects the well-known volumetric locking and stiffness overestimation of linear tetrahedra under bending. Previously, the section bug caused `|u| = 0.000` (100% error) because boundary conditions were silently dropped.

The TET10 (3.64% error) and HEX27 (3.67% error) results match the Timoshenko beam reference to within the expected discretisation error for second-order elements.

Chapter 6

Implementation Summary and Design Decisions

This chapter collects the key design decisions made during the implementation of simplex elements in `fall_n`, with forward references to the relevant source files and rationale grounded in the analysis of the preceding chapters.

6.1 Architecture Overview

`fall_n` is a C++23 nonlinear finite element library. Its element hierarchy is fully *compile-time polymorphic*: element types, quadrature rules, constitutive models, and kinematic formulations are all resolved through templates without virtual dispatch.

Layer	Header	Responsibility
Geometry	`SimplexCell.hh`	Reference coords, basis, topology
Element	`SimplexElement.hh`	Iso-parametric map + integrator
Integration	`SimplexQuadrature.hh`	Classical symmetric rules
	`StroudConicalProduct.hh`	Arbitrary-order conical product
Mesh I/O	`GmshDomainBuilder.hh`	Gmsh → `Domain`
Model	`Model.hh`	DOF management, BC application
Post-processing	`VTKModelExporter.hh`	Gauss → node projection, VTU

6.2 Design Decisions

6.2.1 Decision 1: HMS 4-Point as Default TET10 Quadrature

Context
TET10 elements require at least degree-2 quadrature for the stiffness matrix. The Keast 5-point rule (degree 3) was the initial default, following the common "one degree above minimum" heuristic.

Problem
The Keast 5-point rule has $w_1 = -2/15\,|\mathcal{S}_3|$ at the centroid, causing the lumped L^2 projection to produce checkerboard artefacts (chapter 3).

Decision
Switch to HMS 4-point (degree 2) as the default. Degree 2 is the *exact minimum* for the TET10 stiffness matrix; it provides no excess integration accuracy but avoids the negative-weight problem entirely.

Alternative considered
Conical product $n = 2$ (8 points, degree 3, all positive). This was rejected as the *default* because it uses twice as many points as HMS 4, but it is available as `ConicalProductIntegrator<3,2>` for use cases requiring degree 3.

Source
`SimplexQuadrature.hh`, function `default_simplex_rule<3,2>()`.

6.2.2 Decision 2: Golub–Welsch for Gauss–Jacobi Quadrature

Context
The Stroud conical product in 3D requires Gauss–Jacobi quadrature with parameters up to $\alpha = D - 1 = 2$, $\beta = 0$.

Problem
An initial implementation used Newton iteration on the Jacobi polynomial (with Tricomi / asymptotic initial guesses). For $\alpha = 2$ and $n \geq 3$, the iteration converged to duplicate roots because the initial guesses were insufficiently accurate for the distorted zero distribution of high-α Jacobi polynomials.

Decision
Replace Newton iteration entirely with the Golub–Welsch eigenvalue algorithm (section 2.4). This algorithm is unconditionally robust for all valid $\alpha, \beta > -1$ and any n, since it requires no initial guesses.

Implementation
Uses Eigen's `SelfAdjointEigenSolver` on the $n \times n$ symmetric tridiagonal Jacobi matrix. Cost is $O(n^2)$; since $n \leq 10$ in practice, this is negligible.

Source
`StroudConicalProduct.hh`, function `gauss_jacobi()`.

6.2.3 Decision 3: Adaptive Projection as Default

Context
Gauss-point fields must be projected to nodes for VTU export.

Problem
Lumped L^2 projection is not robust on higher-order simplex elements. The issue is deeper than negative quadrature weights: quadratic simplex vertex functions have signed support, so the row-sum lumped nodal measures can become non-positive even when *all* quadrature weights are positive (chapter 16).

Decision
Make `compute_material_fields()` adaptive. The exporter first inspects the local lumped nodal weights induced by the current element geometry and quadrature rule. If every local weight is strictly positive, it uses lumped L^2 projection; otherwise it falls back to polynomial patch recovery (`patch_recover_to_nodes()`). The old volume-weighted averaging path is kept only as an explicit low-order fallback.

Trade-off
The adaptive pre-scan adds one post-processing pass over element quadrature points, but this cost is negligible compared with a solve and preserves the smoother lumped L^2 path for element families such as HEX27 where it is mathematically sound. The patch-recovery path adds local least-squares work in post-processing only; it does not touch the FE assembly hot path.

Source
`VTKModelExporter.hh`, methods `lumped_projection_weights_into()`, `compute_material_fields()`, and `patch_recover_to_nodes()`.

6.2.4 Decision 4: Gmsh TET10 Node Reordering

Context
Gmsh's local node ordering for 10-node tetrahedra differs from `fall_n`'s ordering at the edge midpoints.

Problem
Without reordering, the Jacobian mapping is incorrect, producing negative determinants and garbage results.

Decision
Apply the permutation $(0, 1, 2, 3, 4, 6, 7, 5, 9, 8)$ to map Gmsh indices to `fall_n` indices. This is done at mesh import time in `GmshDomainBuilder.hh`.

Verification
The simplex geometry tests (`test_element_geometry.cpp`) verify positive Jacobian determinants and correct isoparametric mapping for both TET4 and TET10 elements imported from Gmsh.

Source
`GmshDomainBuilder.hh`, case 11 (TET10).

6.2.5 Decision 5: PETSc Section Patch

Context
`fall_n` uses PETSc's DMPlex for distributed mesh management and `PetscSection` for DOF layout.

Problem
A bug in `PetscSectionSetConstraintIndices()` reads the atlas array with an incorrect index when `pStart` > 0 (chapter 5).

Decision
Apply a local one-line patch: `s->atlasDof[point]` $\rightarrow$ `s->atlasDof[point - s->pStart]`.

Impact
Without this patch, Dirichlet boundary conditions are silently incorrect on meshes where the vertex stratum does not start at point 0.

Source
PETSc 3.24.5, `src/sys/utils/section.c`, line $\sim$2997.

6.2.6 Decision 6: Hex27 VTK Face-Center Ordering

Context
`fall_n` stores the 27-node tensor-product HEX27 in lexicographic tensor order, while ParaView reads the exported VTU using `VTK_TRIQUADRATIC_HEXAHEDRON`.

Problem
The VTK exporter had the 6 face-center nodes partially permuted. Corners and edge midpoints were correct, but ids 20–23 did not match VTK's actual `vtkTriQuadraticHexahedron` parametric-node ordering. The visible symptom was a folded or diamond-like surface pattern on otherwise regular HEX27 meshes.

Decision
Correct the VTK permutation in `VTKCellTraits.hh`. For the face centers, VTK expects the order $(x^-, x^+, y^-, y^+, z^-, z^+)$, which maps to the internal `fall_n` indices $(12, 14, 10, 16, 4, 22)$.

Verification
Add a regression test that compares the exporter permutation against the coordinates returned by `vtkTriQuadraticHexahedron::GetParametricCoords()`, instead of relying on a hand-written table alone.

Source
`src/post-processing/VTK/VTKCellTraits.hh`, `tests/test_simplex.cpp`.

6.2.7 Decision 7: Split VTK Export as the Supported Contract

Context
The exporter explored two more ambitious alternatives to the original split workflow: a native quadrature-aware `.vtu` and a single-file visualization mesh mixing continuum cells with explicit Gauss glyphs.

Problem
Both alternatives were architecturally interesting but operationally noisy. The native route depended on ParaView's quadrature-specific metadata handling, while the mixed visualization mesh overloaded a single dataset with two different semantic carriers. In both cases, the resulting workflow was harder to explain and easier to misuse than the original two-file export.

Decision
Keep only one supported ParaView-facing contract: `*_mesh.vtu` for the continuum mesh with nodal fields and `*_gauss.vtu` for the material-point cloud with `qp_*` fields. The displacement field is interpolated onto the Gauss cloud so `Warp By Vector` works directly on the material points.

Architectural consequence
The post-processing boundary is now small and explicit. The exporter no longer carries status flags or helper paths for embedded quadrature or mixed visualization meshes. The retained extension points are the nodal projection strategy, the Gauss-cloud field inventory, and the tensor-scalar derivation utilities.

Source
`VTKModelExporter.hh`, `tests/test_vtk_gauss_cloud_contract.cpp`.

6.2.8 Decision 8: Explicit Scalar Filters over Raw Voigt Arrays in ParaView

Context
Even with a clean split export, raw Voigt arrays remain a poor visualisation contract. ParaView can show them, but the user still has to remember the Voigt ordering and the engineering-shear convention.

Problem
A raw array named simply `strain` or `stress` is a poor post-processing contract. The user must remember both the Voigt convention and the data association, and the same logical field can appear in both `FieldData` and generated `PointData`.

Decision
Rename the raw tensors explicitly as `*_voigt`, separate nodal and quadrature views through the prefixes `nodal_` and `qp_`, and export scalar component and invariant fields such as `qp_stress_von_mises`, `qp_stress_pressure`, `qp_strain_equivalent_strain`, etc.

Architectural consequence
The exporter now owns the Voigt-to-scalar mapping explicitly, including the engineering-shear correction for strain-like tensors. This is preferable to deferring the convention to ad-hoc ParaView calculators.

Source
`VTKTensorFieldDerivatives.hh`, `VTKModelExporter.hh`, `tests/test_vtk_tensor_field_derivatives.cpp`

6.3 Nonlinear Showcase (`main.cpp`)

The main executable exercises 7 constitutive/kinematic formulations on both HEX27 and TET10 meshes (14 cases total) on a cantilever beam:

Case	Kinematics	Constitutive	Load	Steps
1	SmallStrain	Linear elastic	0.05	1 (linear)
2	TotalLagrangian	SVK	0.05	1
3	TotalLagrangian	SVK	0.50	5
4	TotalLagrangian	Neo-Hookean	0.50	5
5	UpdatedLagrangian	SVK	0.05	1
6	UpdatedLagrangian	Neo-Hookean	0.50	5
7	SmallStrain	J2 plasticity	1.00	20

Material parameters: $E = 200$, $\nu = 0.3$, $\sigma_y = 0.250$, $H = 10.0$.

Geometry: $[0, 10] \times [0, 0.4] \times [0, 0.8]$. Fixed face at $x = 0$, traction on face at $x = 10$.

Meshes:
- `data/input/box.msh`: HEX27 (20 elements, 315 nodes).
- `data/input/box_tet10.msh`: TET10 ($\sim$1329 elements, 2840 nodes).

Post-processing: All cases use `compute_material_fields()` with adaptive dispatch (lumped L^2 when safe, polynomial patch recovery otherwise), producing two VTUs in `data/output/`: a continuum `*_mesh.vtu` with nodal fields and a `*_gauss.vtu` cloud with material-point fields plus the interpolated displacement vector.

6.4 Test Suite

The simplex-related tests in `tests/test_simplex.cpp` include 49 test cases covering:
- Node count, reference coordinates, and shape function values for TET4 and TET10.
- Barycentric coordinate partition of unity and non-negativity.
- Shape function gradient consistency (finite difference vs. analytical).
- Face topology and local-to-global node mapping.
- Quadrature weight sums ($= 1/D!$ for the reference simplex).
- All-positive-weight verification for HMS and conical product rules.
- Points-inside-simplex verification for conical product.
- Polynomial exactness tests (degrees 3, 5, 7) for conical product integrators.
- `ConicalProductIntegrator<D, N>` template instantiation for $D \in \{1, 2, 3\}$, $N \in \{1, 2, 3, 4\}$.

All 49 tests and 16 element geometry tests pass as of the current revision.

6.5 File Inventory

File	Purpose
`src/geometry/SimplexCell.hh`	Reference simplex: coords, basis, topology (consteval)
`src/elements/element_geometry/SimplexElement.hh`	Element type combining geometry + integrator
`src/numerics/.../SimplexQuadrature.hh`	Classical symmetric rules (HMS, Keast)
`src/numerics/.../StroudConicalProduct.hh`	Golub–Welsch + Duffy + ConicalProductIntegrator
`src/mesh/gmsh/GmshDomainBuilder.hh`	Gmsh import with TET10 node reordering
`src/post-processing/VTK/VTKModelExporter.hh`	L2 and SPR projection, VTU export
`data/input/box_tet.geo`	Gmsh geometry for tetrahedral cantilever mesh
`data/input/box_tet10.msh`	TET10 mesh (2840 nodes)
`tests/test_simplex.cpp`	49 simplex + conical product unit tests
`main.cpp`	14-case nonlinear showcase

Chapter 7

Element Architecture: Design Patterns and Extensibility

This chapter documents the layered element architecture of `fall_n`, focusing on two central abstractions — `ElementGeometry` and `FEM_Element` — their interaction with policy-based design, and the resulting customisation points. We compare the approach against the architectures of OpenSees [11] and Kratos Multiphysics [3], highlighting trade-offs in extensibility, compile time, and runtime efficiency.

7.1 Guiding Principles

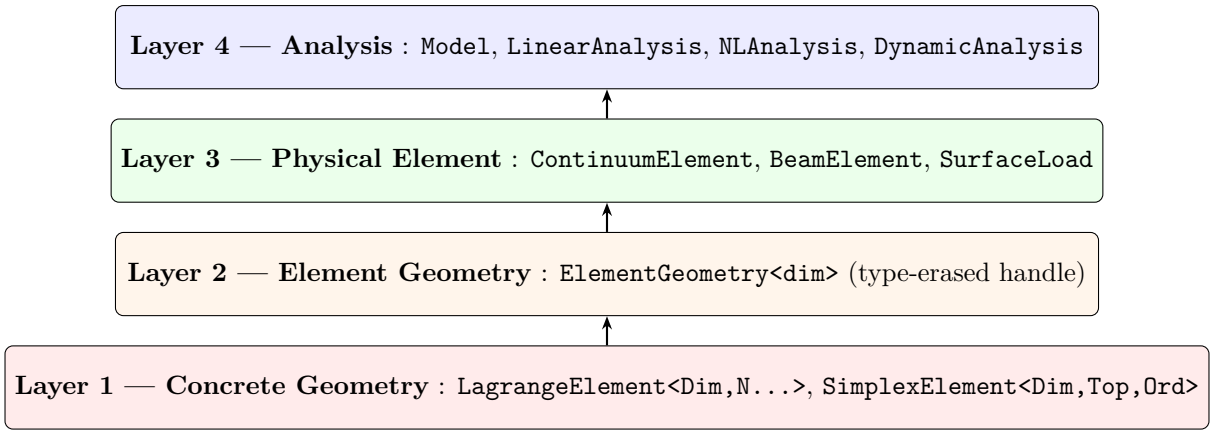
`fall_n` is designed around three principles that permeate the element subsystem:

1. **Zero-cost abstractions.** Template instantiation resolves element type, quadrature rule, kinematic formulation, and constitutive model at compile time. No virtual dispatch in the inner quadrature loop. The compiler sees the full call chain from Gauss-point integration down to the shape function evaluation, enabling inlining and SIMD vectorisation.
2. **Separation of geometry from physics.** The geometric interpolation (shape functions, Jacobian, quadrature) is decoupled from the physical element (strain–displacement operator, constitutive law, DOF assembly). The same `ElementGeometry<3>` serves both a `ContinuumElement` (3D solid) and a `SurfaceLoad` (boundary traction).
3. **Type erasure at system boundaries only.** Within a single element type, everything is monomorphic and stack-allocated. Virtual dispatch is introduced only where heterogeneity is required: storing mixed element types in a single container (`FEM_Element`) or interfacing geometry with the mesh manager (`ElementGeometry::pimpl_`).

These principles are realised through two complementary patterns: *policy-based design* (compile-time customisation) and *external polymorphism* (runtime type erasure).

7.2 Layer Decomposition

The element subsystem is organised in four layers, each with a clear responsibility boundary:



7.2.1 Layer 1: Concrete Geometry Types

These are pure value types with *no virtual functions*. They encode the reference element, shape functions, and Jacobian evaluation as compile-time-resolved operations.

LagrangeElement<Dim, N...>. A tensor-product Lagrange element parameterised by the embedding dimension `Dim` and per-axis node counts `N...` (variadic).

Instantiation	Topology	Gmsh type
`LagrangeElement<3, 2, 2, 2>`	HEX8	5
`LagrangeElement<3, 3, 3, 3>`	HEX27	12
`LagrangeElement<3, 2, 2>`	QUAD4 (surface)	3
`LagrangeElement<3, 3, 3>`	QUAD9 (surface)	10
`LagrangeElement<3, 2>`	LINE2 (edge)	1

Key design features:

- The reference cell (`geometry::cell::LagrangianCell<N...>`) is a `constexpr` object holding the nodal coordinates, the tensor-product basis, and the subentity topology — all computed at compile time.
- The topological dimension is deduced from the parameter pack: `sizeof...(N)`. Thus a surface element is simply a `LagrangeElement<3, 3, 3>` — same code path, fewer template parameters.
- Shape functions are products of 1D Lagrange polynomials, evaluated by index decomposition. No lookup table; the compiler folds constant expressions.
- The `local_index_[]` array enables node reordering (e.g., Gmsh to internal convention) without modifying the connectivity array.

SimplexElement<Dim, TopDim, Order>. A simplex element parameterised by embedding dimension, topological dimension, and polynomial order. It mirrors the `LagrangeElement` public interface but derives its reference cell from `geometry::simplex::SimplexCell<TopDim, Order>`, which stores the barycentric-based Lagrange shape functions for simplicial geometry.

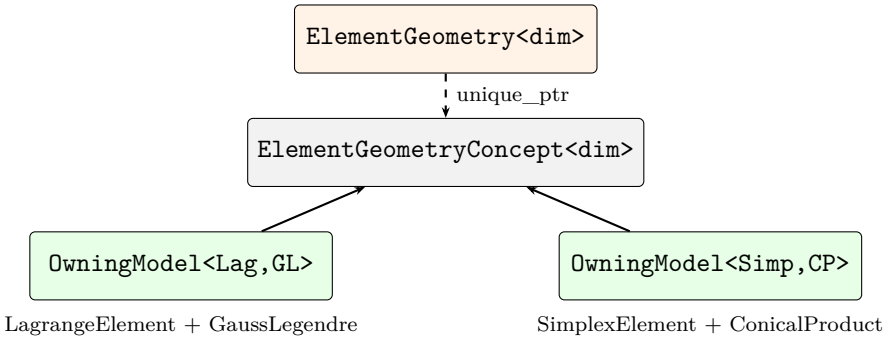
Both families share a structurally identical interface (node access, `H()`, `dH_dx()`, `evaluate_jacobian()`, `map_local_point()`) but they are *unrelated types* — there is no inheritance hierarchy linking them. Uniformity is achieved through duck-typing in the next layer.

7.2.2 Layer 2: ElementGeometry (Type-Erased Handle)

`ElementGeometry<dim>` is a type-erased value-semantic wrapper that stores *any* concrete geometry type together with its quadrature rule behind a single, uniform interface. It is the pivot of the entire element subsystem.

Pattern: External Polymorphism. The design follows the *External Polymorphism* pattern (Cleary et al. 1997, popularised by Sean Parent's "Inheritance Is the Base Class of Evil" talk):

1. **Concept** (`impl::ElementGeometryConcept<dim>`) — abstract base with pure virtual methods: `H()`, `dH_dx()`, `evaluate_jacobian()`, `differential_measure()`, `num_integration_points()` quadrature accessors, and subentity topology.
2. **Model** (`impl::OwningModel_ElementGeometry<ElementType, IntegrationStrategy>`) — template class deriving from the concept, *owning* a concrete `ElementType` and `IntegrationStrategy` by value. Each virtual override delegates to the stored objects.
3. **Handle** (`ElementGeometry<dim>`) — holds a `std::unique_ptr<ElementGeometryConcept<dim>>` (`pimpl_`). Provides value semantics via `clone()`, move defaulted.



Why External Polymorphism and not classical inheritance?

- `LagrangeElement` and `SimplexElement` remain pure value types — no vtable pointer, no heap allocation, fully `constexpr`-eligible. They can be used independently of the type-erasure layer (e.g., in constexpr unit tests).
- The virtual dispatch lives in the *wrapper*, not in the element itself. Concrete element code retains full optimisation potential.
- New element families (e.g., NURBS, serendipity, spectral) are added by writing a new value type with the required interface — no base class to inherit from, no code changes in existing elements.

Differential measure dispatch. The `differential_measure()` method in `OwningModel` uses `if constexpr` on the compile-time-known topological dimension:

```
if constexpr (topological_dim == dim)
    return std::abs(J.determinant());          // volume
else if constexpr (topological_dim == 1)
    return J.col(0).norm();                    // arc length
else if constexpr (topological_dim == 2 && dim == 3)
    return J.col(0).cross(J.col(1)).norm();    // surface area
```

Despite crossing the virtual boundary, the Jacobian matrix itself is stack-allocated (Eigen fixed-size `Matrix<double, dim, Dynamic, ..., dim, dim>`), so no heap allocation occurs inside the hot loop.

Integration via template callback. `ElementGeometry` exposes a non-virtual template method `integrate(f)` that accepts any callable `f(LocalCoordView)` and loops over quadrature points:

```
template<std::invocable<LocalCoordView> F>
auto integrate(F&& f) const {
    // ... sum w_i * |J(xi_i)| * f(xi_i) ...
}
```

Because `f` is a template parameter (not `std::function`), the compiler inlines the integrand. Only the quadrature point data (coordinates, weights) and the Jacobian evaluation pass through the virtual boundary; the user's element-level computation stays in the monomorphic world.

7.2.3 Layer 3: Physical Elements

Physical elements combine an `ElementGeometry` with material data and DOF assembly logic. Each physical element type is a class template parameterised by a *MaterialPolicy* and a *KinematicPolicy*.

ContinuumElement<MaterialPolicy, ndof, KinematicPolicy>. The workhorse for 1D/2D/3D solid mechanics:

- Holds a non-owning pointer to an `ElementGeometry<dim>` (owned by the `Domain`).
- Owns the `MaterialPoint` array (one per Gauss point), each holding a `Material<MaterialPolicy>` with its constitutive strategy.
- Provides `inject_K()`, `compute_internal_forces()`, `inject_tangent_stiffness()`, `commit_material` — the full `FiniteElement` concept interface.
- The strain–displacement operator $\mathbf{B}$ is delegated to the `KinematicPolicy` (e.g., `SmallStrain` or `TotalLagrangian`), resolved at compile time.

BeamElement<BeamPolicy, Dim, AsmPolicy>. Timoshenko beam with section integration:

- Two-node, local-frame kinematics with a rotation matrix $\mathbf{R}$.
- Integration along the beam axis uses `MaterialSection<BeamPolicy>` objects instead of `MaterialPoint`.
- The `AsmPolicy` template parameter selects between direct assembly (`DirectAssembly`) and static condensation (`CondensedAssembly`) — a compile-time strategy switch.

SurfaceLoad. Not a "physical element" in the structural sense, but uses the `ElementGeometry` quadrature machinery to compute consistent nodal forces from a distributed traction on a boundary face.

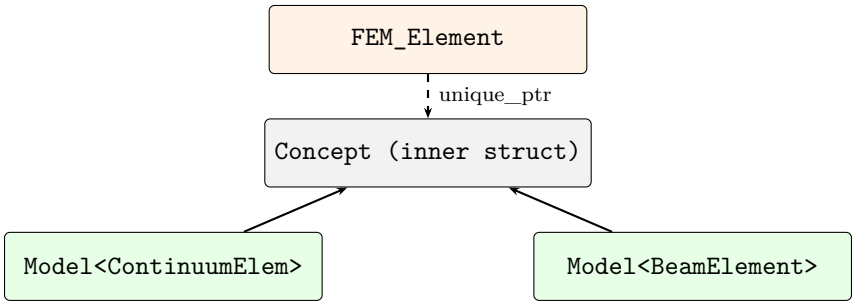
7.2.4 Layer 4: Analysis Level

`Model<MaterialPolicy, KinematicPolicy, ndofs, ElemPolicy>` owns the element container, the `Domain`, and the PETSc objects (`PetscSection`, matrices, vectors).

The `ElemPolicy` template parameter controls element storage (section 7.4.3). For homogeneous meshes, the container is a raw `std::vector<ContinuumElement<...>>` with zero overhead. For mixed meshes, it is `std::vector<FEM_Element>` with one virtual dispatch per element call.

7.3 FEM_Element: Top-Level Type Erasure

`FEM_Element` applies the same External Polymorphism pattern as `ElementGeometry`, but at the assembly level:



Key design decisions:

- **Value semantics.** Copy via `clone()`, move defaulted. No shared ownership, no reference counting.
- **Self-wrap guard.** A `requires` clause prevents wrapping a `FEM_Element` inside another `FEM_Element` (double indirection).
- **Recursive concept satisfaction.** `FEM_Element` itself satisfies the `FiniteElement` concept, verified by a `static_assert` at the bottom of the header. This means it can be nested inside other type-erased wrappers without losing the concept constraint.
- **Optional interface methods.** Mass matrix operations (`density()`, `inject_mass()`) use `if constexpr` inside `Model<T>` to detect whether the concrete element supports them, defaulting to a no-op otherwise. This avoids bloating the concept with operations not all elements provide.

There is also a `StructuralElement` wrapper, identical in pattern, that provides type safety for structural-only models (beams, shells). Both derive from *no common base class* — they are independent type-erasure handles, unified only by the `FiniteElement` concept.

7.4 Policy-Based Design: Customisation Points

`fall_n` uses policies — template parameters that select compile-time strategies — at multiple levels. Each policy is a *customisation point*: a seam where the user can inject different behaviour without modifying existing code.

7.4.1 MaterialPolicy

A *trait bag* that binds kinematic and conjugate types:

Policy	StrainT	StressT	dim
ThreeDimensionalMaterial	Strain<6>	Stress<6>	3
PlaneMaterial	Strain<3>	Stress<3>	2
UniaxialMaterial	Strain<1>	Stress<1>	1
TimoshenkoBeam3D	BeamStrain<6,3>	BeamForces<6>	3
TimoshenkoBeam2D	BeamStrain<3,2>	BeamForces<3>	2

All generic code (`Material<P>`, `MaterialPoint<P>`, `MaterialState<P>`, `ElasticRelation<P>`) is parameterised on a single `MaterialPolicy` and extracts its types via `P::StrainT`, `P::StressT`, etc.

Extensibility. Adding a shell element requires only:

1. Define `ShellMaterial<N, 3>` with appropriate `ShellGeneralizedStrain<N,3>` and `ShellResultants<N`
2. Write a constitutive model conforming to `ConstitutiveRelation<ShellMaterial<N,3»`.
3. Write the element class (`ShellElement`) satisfying `FiniteElement`.

No existing code changes; the policy mechanism propagates types automatically.

7.4.2 KinematicPolicy

Selects the strain–displacement relationship at compile time:

Policy	Description	Linear?	K_σ?
SmallStrain	$\varepsilon = \nabla^s \mathbf{u}$	yes	no
TotalLagrangian	$\mathbf{E} = \frac{1}{2}(\mathbf{F}^T\mathbf{F} - \mathbf{I})$	no	yes
UpdatedLagrangian	Cauchy strain in current config.	no	yes
Corotational	Large rotation, small strain	no	yes

Each policy is a struct with static methods:

```
struct SmallStrain {
    static constexpr bool is_geometrically_linear    = true;
    static constexpr bool needs_geometric_stiffness  = false;

    template <std::size_t dim>
    static GPKinematics<dim> evaluate(
        ElementGeometry<dim>* geo, std::size_t num_nodes,
        std::size_t ndof, const Array& Xi,
        const Eigen::VectorXd& u_e);

    template <std::size_t dim>
    static auto compute_B(ElementGeometry<dim>* geo, ...);
};
```

`ContinuumElement` calls `KinematicPolicy::evaluate<dim>()` at each quadrature point. The boolean traits steer `if constexpr` branches inside the element (e.g., add K_σ only when `needs_geometric_stiffness == true`).

Extensibility. A new formulation (e.g., logarithmic strain) is added by defining a new struct satisfying `KinematicPolicyConcept` with the required static methods and boolean traits. No changes to `ContinuumElement` or any analysis class.

7.4.3 ElementPolicy (Storage Strategy)

Controls how `Model` stores its element collection:

Policy	Container	Homogeneous	Overhead
`SingleElementPolicy<E>`	`std::vector<E>`	yes	none
`MultiElementPolicy`	`std::vector<FEM_Element>`	no	1 vtable + 1 ptr chase

The concept `ElementPolicyLike` enforces the policy interface:

```
template <typename P>
concept ElementPolicyLike = requires {
    typename P::element_type;
    typename P::container_type;
    { P::is_homogeneous } -> std::convertible_to<bool>;
} && FiniteElement<typename P::element_type>;
```

Default path. `Model<ThreeDimensionalMaterial>` defaults to: `SingleElementPolicy<ContinuumElement<ThreeDimensionalMaterial, 3, SmallStrain»`, yielding a contiguous `std::vector` of monomorphic elements — optimal for cache locality and auto-vectorisation.

7.4.4 AssemblyPolicy (Beams)

Beam elements accept an `AsmPolicy` parameter:

- `DirectAssembly` — full stiffness matrix, no condensation.
- `CondensedAssembly` — static condensation of internal DOFs before injection into PETSc.

Both are stateless tag types; the choice is resolved at compile time via `if constexpr` inside `BeamElement::inject_K()`.

7.4.5 ConstitutiveStrategy (Material Update)

The material update mechanism (`Material<P>`) delegates to a type-erased strategy object (`IntegrationStrategy<P>`), enabling runtime polymorphism over the constitutive response:

- `ElasticRelation<P>` — linear elastic.
- `HyperelasticRelation<P>` — Neo-Hookean, SVK, etc.
- `J2Plasticity<P>` — return mapping with isotropic hardening.

This is the *only* place in the element quadrature loop where runtime polymorphism is used, because the constitutive model is genuinely a runtime choice (same mesh, different materials per physical group).

7.5 Compile-Time Cost Analysis

Policy-based design with heavy template usage has a well-known compile-time cost. `fall_n` mitigates this through several strategies:

1. **Header-only by necessity, not by choice.** The library is header-only because C++ templates must be visible at the point of instantiation. However, explicit template instantiation could be added in `.cpp` files for the most common configurations to reduce redundant codegen across translation units.
2. **Type erasure as a firewall.** `ElementGeometry<dim>` and `FEM_Element` act as *template firewalls*: code that consumes an `ElementGeometry<3>` does not need to see the headers of `LagrangeElement` or `SimplexElement`. This significantly reduces transitive include graphs.
3. **Consteval reference cells.** The `LagrangianCell<N...>` and `SimplexCell<TopDim, Order>` objects with their basis functions and topological tables are `constexpr`/`consteval`, evaluated by the compiler once. They produce no runtime code unless actually called; the compiler may fold constant subexpressions entirely.
4. **Concept constraints as documentation.** The `FiniteElement`, `KinematicPolicyConcept`, `ElementPolicyLike` concepts serve as lightweight checks that produce clear error messages at instantiation time, avoiding the traditional deep-template-error-message problem.

Measured impact. A full rebuild of `fall_n` (main + 12 test targets) takes approximately 105 seconds on a single core with GCC 11 and debug optimisation (`-g`). The dominant cost is Eigen's template instantiation inside `ContinuumElement` and `BeamElement`; the element geometry layer adds minimal overhead because the type-erasure boundary limits template propagation.

7.6 Comparison with OpenSees and Kratos

7.6.1 Overview

Aspect	**OpenSees**	**Kratos**	**fall_n**
Language	C++ (C++11/14)	C++ (C++17) + Python	C++ (C++20/23)
Polymorphism model	Classical inheritance	Classical inheritance + Python bridge	Policies + External Polymorphism
Element base	`Element` (virtual)	`Element` (virtual)	`FiniteElement` (concept)
Geometry base	`CrdTransf` (virtual)	`Geometry<TPoint>` (virtual)	`ElementGeometry<dim>` (type-erased)
Material base	`UniaxialMaterial` / `NDMaterial` (virtual)	`ConstitutiveLaw` (virtual)	`Material<Policy>` (policy + type-erased strategy)
Dispatch cost	1–3 vtable calls per GP	1–2 vtable calls per GP	0 (homogeneous) or 1 (heterogeneous)

7.6.2 OpenSees

OpenSees [11] uses a deep inheritance hierarchy: `DomainComponent` → `Element` → concrete types. Materials form an even deeper tree: `Material` → `UniaxialMaterial` → specific laws.

Strengths.

- Extremely mature (20+ years), battle-tested in earthquake engineering.
- Simple runtime scripting via Tcl (later Python) — users define models without recompiling.
- Large element library (200+ elements).

Weaknesses (from an architecture perspective).

- **Monolithic virtual hierarchy.** All elements inherit from `Element`, which carries 30+ virtual methods including `getResistingForce()`, `getTangentStiff()`, etc. Adding a new element requires implementing all of them (or relying on default no-ops that may silently produce incorrect results).
- **No separation of geometry and physics.** A `Brick20N` element hard-codes its shape functions, Jacobian, AND constitutive integration in a single class. Changing the quadrature rule requires modifying the element source.
- **Dynamic dispatch in the inner loop.** Every call to `material->getStress()` and `material->getTangent()` at each Gauss point goes through virtual dispatch. While the cost is small relative to the linear algebra, it inhibits inlining and auto-vectorisation across the material boundary.
- **No concept or constraint checking.** Adding a new element with a typo in a virtual override signature compiles silently and produces incorrect results at runtime.

Contrast with fall_n. `fall_n` eliminates the monolithic base class by defining `FiniteElement` as a C++20 concept that enforces the interface at compile time. The element is free to organise its internals however it pleases; the concept checks only what the assembler needs. Geometry is factored out into `ElementGeometry`, so the same mesh can serve continuum, surface-load, and post-processing operations.

7.6.3 Kratos Multiphysics

Kratos [3] uses a more modular architecture than OpenSees, with explicit separation of geometry (`Geometry<TPoint>`), element (`Element`), condition (`Condition`), constitutive law (`ConstitutiveLaw`), and process (`Process`).

Strengths.

- **Multi-physics by design.** The `ModelPart` / `ProcessInfo` / `Variable` registry enables coupling of disparate physics (structural, fluid, thermal) sharing the same mesh.
- **Python-driven workflow.** Element selection, boundary conditions, solver parameters, and post-processing are all scriptable in Python without recompilation.
- **Geometry separated from element.** `Geometry<TPoint>` handles shape functions and Jacobian; `Element` handles physics. This is conceptually analogous to `fall_n`'s two-layer split.

Weaknesses (from an architecture perspective).

- **Runtime overhead of the variable system.** Each nodal value is stored in a hash-map-like container (`Variable<T>` keys), requiring a lookup per node per variable access. This sacrifices cache locality and prevents compile-time resolution of DOF layouts.
- **Virtual geometry hierarchy.** `Geometry<TPoint>` is an inheritance tree: `Geometry` → `Triangle2D3`, etc. Shape functions are declared `virtual`, adding dispatch cost at each Gauss point. There is no mechanism to "inline" the geometry into the element at compile time.
- **Heavyweight element interface.** `Element` carries methods for `CalculateLocalSystem`, `CalculateMassMatrix`, `CalculateDampingMatrix`, `GetDofList`, etc. Many elements implement subsets, leaving the rest as no-ops or throwing exceptions — a code-smell of interface bloat.
- **No compile-time guarantees.** Registering a new element involves adding entries to a factory map at runtime. Errors in the element's interface (wrong return type, missing method) are caught only at link time or, worse, at runtime.

Contrast with fall_n. `fall_n`'s policy-based design replaces the Kratos variable registry with compile-time type parameters: the `MaterialPolicy` selects the strain/stress types, the `KinematicPolicy` selects the B-operator, and the `ElementPolicy` selects the storage strategy — all resolved by the compiler. The trade-off is that `fall_n` requires recompilation to change element types, whereas Kratos allows runtime scripting.

7.6.4 Summary: Architectural Trade-offs

Trade-off	OpenSees	Kratos	fall_n
Runtime scripting	✓✓	✓✓✓	—
Compile-time safety	—	—	✓✓✓
Zero-cost inner loop	—	—	✓✓
Geometry–physics split	—	✓✓	✓✓✓
Heterogeneous elements	✓✓✓	✓✓✓	✓✓
Multi-physics coupling	✓	✓✓✓	—
Maturity / element count	✓✓✓	✓✓✓	✓

`fall_n` occupies a different niche: it is *not* a general-purpose production FE framework, but a research library where the primary concern is **architectural clarity and compile-time correctness** over runtime flexibility. The policy-based approach is intentionally opinionated:

it makes the "easy things trivial" (adding a new constitutive model, kinematic formulation, or element family within the existing type system) at the cost of making the "hard things hard" (mixing unrelated physics, runtime element selection from user input).

7.7 Extensibility Guide: Adding a New Element Family

This section walks through the concrete steps to add a hypothetical "Serendipity element" family, illustrating each customisation point.

1. **Layer 1: Concrete geometry type.** Create `SerendipityElement<Dim, Order>`:
   - Store node coordinates, define `H()`, `dH_dx()`, `evaluate_jacobian()`, `map_local_point()` with the serendipity basis functions.
   - No base class needed. The type must be movable and copyable.

2. **Layer 1: Integration strategy.** Create or reuse a quadrature class (`GaussLegendre2D<p>`, `ReducedIntegration<...>`, etc.) exposing `reference_integration_point(i)` and `weight(i)`.

3. **Layer 2: Instantiate through ElementGeometry.** No code changes. The constructor template:

```
ElementGeometry<3> geo(
    SerendipityElement<3, 2>{tag, node_ids},
    GaussLegendre2D<3>{});
```

   automatically instantiates an `OwningModel<Serendipity..., GL...>` and stores it behind the type-erased handle.

4. **Layer 3: Use with ContinuumElement.** No code changes. `ContinuumElement` receives a pointer to the geometry and calls `H()`, `dH_dx()`, `differential_measure()` through the virtual interface. The **B** matrix and assembly are generic.

5. **Layer 4: Mesh import.** Add a case in `GmshDomainBuilder.hh` for the Gmsh element type code, constructing `ElementGeometry` with the serendipity concrete type and the desired integrator.

6. **Optional: Type-safe model.** If the serendipity element will be used in its own model (no mixing), define:

```
using SerendipitySolid =
    ContinuumElement<ThreeDimensionalMaterial, 3,
                     SmallStrain>;
using SerendipityModel =
    Model<ThreeDimensionalMaterial, SmallStrain, 3,
          SingleElementPolicy<SerendipitySolid>>;
```

   This compiles to a contiguous vector with zero virtual dispatch.

What does NOT need to change:

- `ElementGeometry.hh` (new type is auto-detected by the templated constructor).
- `ContinuumElement.hh` (works through geometry pointer).
- `FEM_Element.hh` (type erasure is generic).
- `Model.hh` (policy-parameterised).
- Any analysis class (`LinearAnalysis`, `NLAnalysis`, etc.).
- Any material or constitutive model.

This is the central extensibility promise: **new element families are additive** — they require only new files, not modifications to existing ones — conforming to the Open/Closed Principle.

7.8 Runtime Efficiency Considerations

7.8.1 Virtual Dispatch Budget

In the homogeneous path (`SingleElementPolicy`), the element assembly loop has **zero** virtual calls for the physics: all `inject_K()`, `B()`, `H()`, `dH_dx()` calls are monomorphic and inlineable.

The only virtual calls per Gauss point are:

- `ElementGeometry::evaluate_jacobian()` — 1 vcall for the Jacobian (through pimpl).
- `ElementGeometry::H()`, `dH_dx()` — $n_{\text{nodes}} \times (1 + \text{dim})$ vcalls for shape functions and derivatives.
- `Material::compute_response()` — 1 vcall for the constitutive model (through the strategy pointer).

This is *identical per-element overhead* to OpenSees or Kratos, but `fall_n` avoids the additional vtable dispatch that those frameworks incur at the element container level.

7.8.2 Cache-Friendly Layout

`SingleElementPolicy<E>` stores all elements contiguously in a single `std::vector<E>`. Each element's `MaterialPoint` array is heap-allocated (as a `std::vector<MaterialPointT>`), but its entries are contiguous for that element.

`MultiElementPolicy` stores `FEM_Element` objects contiguously (the `unique_ptr<Concept>` wrappers are packed), but the pointed-to `Model<T>` objects are heap-allocated with potentially scattered addresses. This adds one pointer chase per element — acceptable for heterogeneous cases where the element count is moderate.

7.8.3 DOF Index Caching

`ContinuumElement` caches its flat `dof_indices_[]` array (one entry per element DOF, combining all nodes). The cache is populated lazily on first use and reused across `inject_K()`, `compute_internal_forces()`, and `inject_tangent_stiffness()`. This avoids the per-call overhead of traversing the node pointers to collect DOF indices at every assembly step.

```
void ensure_dof_cache() noexcept {
    if (dofs_cached_) return;
    collect_dof_indices();
}
```

7.8.4 Stack-Allocated Jacobians

The Jacobian matrix crossing the virtual boundary is declared as:

```
using JacobianMatrix = Eigen::Matrix<double, dim,
    Eigen::Dynamic, Eigen::ColMajor, dim, dim>;
```

The `MaxRows = dim`, `MaxCols = dim` template parameters tell Eigen to use a fixed-size stack buffer (up to $3 \times 3 = 72$ bytes) with a dynamic column count. Since `dim` ≤ 3, no heap allocation occurs, even though the column count is "dynamic" (it varies between volume and surface elements).

This ensures that the virtual call to `evaluate_jacobian()` returns by value without any `new`/`delete`, making the hot path allocation-free.

Chapter 8

Generic Boundary and Initial Condition Infrastructure

This chapter documents the design and implementation of the generic boundary condition (BC) and initial condition (IC) infrastructure for `fall_n`. The goals are:

1. **Per-DOF granularity:** constrain individual degrees of freedom (roller supports, symmetry BCs) rather than all DOFs simultaneously.
2. **Imposed (non-zero) Dirichlet:** prescribe non-zero displacement, temperature, or any nodal field value.
3. **Compile-time node selectors:** apply constraints via geometric predicates (plane, box, sphere, custom lambda) that resolve at compile time with zero runtime overhead.
4. **State transfer:** capture the complete post-solve state (`ModelState`) and use it as the initial condition for a subsequent analysis—enabling staged construction, restart, and multi-physics chaining.
5. **Extensibility:** the design should accommodate future field types (temperature, initial stress, pore pressure) without modifying the selector or constraint machinery.

8.1 Gap Analysis: the Legacy API

The original Model API exposed these methods for boundary conditions:

```
void fix_node(size_t node_idx);                          // ALL DOFs = 0
void fix_x(double val, double tol = 1e-6);               // all DOFs at x=val
void apply_node_force(size_t node_idx, auto... f);        // nodal Neumann
void apply_surface_traction(string group, auto... t);     // consistent traction
```

8.2 Design Decisions and Trade-offs

8.2.1 Compile-Time Polymorphism via Concepts

Consistent with `fall_n`'s policy-based architecture (Chapter 7), the node selector system uses C++20 concepts rather than virtual dispatch:

```
namespace fn {
template <typename S, typename NodeT>
concept NodeSelector = requires(const S& s, const NodeT& n) {
    { s(n) } -> std::convertible_to<bool>;
};
```

Table 8.1: Limitations of the legacy BC interface.

Limitation	Impact
`fix_node` constrains *all* DOFs	Cannot model roller supports, symmetry planes, or mixed BCs.
`fix_x/y/z` misnamed	These fix all three displacement components, not just the named one. Misleading to users.
No non-zero Dirichlet via Model API	`global_imposed_solution_` was created and zeroed but never populated through the Model interface. Only `DynamicAnalysis::PrescribedBC` filled it.
`ElementInitialStress` unused	The struct existed in `BoundaryCondition.hh` and was storable in `BoundaryConditionSet`, but no evaluator integrated it into the assembly pipeline.
Procedural selection	Every `fix_*` method hard-coded a coordinate loop. No reusable geometric predicate.
`VecAssemblyBegin/End` per call	`apply_node_force` triggered PETSc assembly for each node.

```
}
```

Any callable `f(const Node<dim>&)-> bool` satisfies `NodeSelector`: lambdas, function objects, and the built-in selectors all qualify. The compiler resolves the predicate at compile time, and after inlining a `PlaneSelector` collapses to a single `fabs` + comparison—zero overhead compared to a hand-written loop.

Why not `std::function`? A `std::function<bool(const Node&)>` would erase the type and force heap allocation for lambdas that exceed the small-buffer size (typically 16–32 bytes). For selectors applied to every node in the mesh, this per-call overhead is measurable. The concept-based approach pays nothing at runtime.

Why not virtual? A virtual `AbstractSelector::operator()` would add one indirect call per node per constraint pass. With 10 000 nodes this is $\sim$10 μs (branch-predictor penalty), acceptable but unnecessary when the concrete type is known at compile time.

8.2.2 Constraint Storage: Extending the Existing Map

The existing `ConstraintDofInfo` type already maps sieve point IDs to (DOF index list, prescribed value list):

```
using ConstraintDofInfo =
  std::map<PetscInt,
           std::pair<std::vector<PetscInt>,
                     std::vector<PetscScalar>>>;
```

The new API *extends* this map rather than replacing it. The key operations are:

1. **Insert in sorted order.** PETSc's `PetscSectionSetConstraintIndices` requires increasing DOF indices. The `constrain_dof` method uses `std::lower_bound` to maintain this invariant.
2. **Merge, not overwrite.** If DOF 0 of a node is already constrained and the user constrains DOF 2, both are stored. An update to DOF 0 changes the prescribed value without duplicating the index.

8.2.3 Non-Zero Dirichlet: Where Values Flow

Non-zero prescribed values are stored in `constraints_` alongside the DOF indices. During `setup()`, a new `fill_imposed_solution()` step extracts these values from the constraint map into the PETSc local vector `global_imposed_solution_`:

```
void setup() {
    setup_boundary_conditions();   // PETSc section
    setup_vectors();               // create Vecs
    fill_imposed_solution();       // NEW: u_D -> imposed
    setup_matrix();                // create Mat
}
```

Remark 8.1. Non-zero Dirichlet requires that the analysis accounts for the coupling $\mathbf{K}_{fc}\,\mathbf{u}_c$ in the right-hand side. `NonlinearAnalysis` handles this naturally: the internal forces $\mathbf{f}_{\text{int}}(\mathbf{u}_{\text{free}} + \mathbf{u}_D)$ include the prescribed part. `LinearAnalysis` currently does *not* apply this correction—for non-zero Dirichlet, users should solve with `NonlinearAnalysis` (which converges in one Newton step for linear problems).

8.2.4 ModelState: Platform-Independent Snapshot

For state transfer between models, we need a data structure that:

- Does *not* depend on PETSc (no `Vec`, `Mat`).
- Does *not* depend on the `MaterialPolicy` template parameter (so an elastic model's state can seed a plastic model).
- Is copyable, movable, and (future) serialisable.

`ModelState<dim>` satisfies all three. It stores:

- Per-node: ID, displacement, velocity.
- Per-element, per-Gauss-point: stress and strain in Voigt notation (`std::array<double, nvoigt>`).

The PETSc-to-array extraction in `capture_state()` uses `VecGetArrayRead` (zero-copy read) and the element's `material_points()` accessor which recomputes stress from the committed strain via the constitutive relation.

8.3 Implementation

8.3.1 NodeSelector.hh — Built-in Selectors

Table 8.2: Built-in node selectors.

Selector	Predicate	Parameters
`PlaneSelector<dim>`	$\lvert x_i - v\rvert < \varepsilon$	axis i, value v, tolerance ε
`BoxSelector<dim>`	$x \in [\text{lo}, \text{hi}]$	corners, tolerance
`SphereSelector<dim>`	$\lVert x - c\rVert \le r$	centre c, radius r, tolerance
`NodeIdSelector`	$\text{id} \in S$	set of node IDs

Combinators turn simple selectors into compound predicates:

```
auto corner = fn::select_and(
    fn::PlaneSelector<3>{0, 0.0},    // x = 0
    fn::PlaneSelector<3>{1, 0.0}     // y = 0
);
auto not_x0 = fn::select_not(fn::PlaneSelector<3>{0, 0.0});
auto union_ = fn::select_or(selectorA, selectorB);
```

Convenience factories (`fn::on_plane`, `fn::in_box`, `fn::in_sphere`, `fn::node_ids`) provide a terser syntax.

8.3.2 Model.hh — New Constraint Methods

Table 8.3: New Model constraint API.

Method	Description
`constrain_dof(n, d, v)`	Constrain DOF d of node n to value v. Sorted insertion.
`constrain_node(n, vals)`	Constrain all DOFs of node n to the given array of values.
`constrain_dof_where(sel, d, v)`	Constrain DOF d at every node matching selector `sel`.
`constrain_where(sel)`	Constrain all DOFs (homogeneous) at every matching node.
`constrain_where(sel, vals)`	Constrain all DOFs with prescribed values at every matching node.
`is_constrained(n)`	Query: is any DOF of node n constrained?
`is_dof_constrained(n, d)`	Query: is DOF d of node n constrained?
`prescribed_value(n, d)`	Get the prescribed displacement at DOF d (0 if free).
`num_constraints()`	Total number of constrained DOFs across all nodes.
`capture_state()`	Extract current solution into a `ModelState<dim>`.
`apply_initial_displacement(s)`	Populate imposed solution from a `ModelState<dim>`.

8.3.3 Backward Compatibility

The legacy methods `fix_node`, `fix_x`, `fix_y`, `fix_z`, and `fix_orthogonal_plane` remain unchanged in semantics—they continue to constrain *all* DOFs of every matching node. Internally, `fix_orthogonal_plane` now delegates to:

```
void fix_orthogonal_plane(int i, double val, double tol) {
    constrain_where(fn::PlaneSelector<dim>{i, val, tol});
}
```

The code path is identical; only the implementation reuses the selector infrastructure.

8.4 Use-Case Walkthrough: Roller vs. Clamped Cantilever

Consider a cantilever beam $[0, L]\times[0, b]\times[0, h]$ loaded at $x = L$ by a surface traction $\mathbf{t} = (0, t_y, t_z)$.

Clamped support (legacy):

```
M.fix_x(0.0);    // constrains u_x, u_y, u_z at x=0
```

Equivalent new API:

```
M.constrain_where(fn::PlaneSelector<3>{0, 0.0});
```

Roller support (new API): Only $u_x = 0$ at $x = 0$. Rigid body prevention requires additionally constraining u_y and u_z at isolated points:

```
// u_x = 0 on entire face
M.constrain_dof_where(fn::PlaneSelector<3>{0, 0.0}, 0, 0.0);

// Prevent rigid translation + rotation about x:
auto origin = fn::select_and(fn::PlaneSelector<3>{0, 0.0},
    fn::select_and(fn::PlaneSelector<3>{1, 0.0},
                   fn::PlaneSelector<3>{2, 0.0}));
M.constrain_dof_where(origin, 1, 0.0);  // u_y at origin
M.constrain_dof_where(origin, 2, 0.0);  // u_z at origin

auto top_z = fn::select_and(fn::PlaneSelector<3>{0, 0.0},
    fn::select_and(fn::PlaneSelector<3>{1, 0.0},
                   fn::PlaneSelector<3>{2, h}));
M.constrain_dof_where(top_z, 1, 0.0);   // prevents Rx
```

The roller case has 30 constrained DOFs (28 nodes $\times$ 1 + 2 extra) versus 84 (28 $\times$ 3) for the clamped case. With fewer constraints, the beam is more flexible:

Table 8.4: Roller vs. clamped comparison (HEX8 beam, $E = 200$, $\nu = 0.3$).

BC type	**Constrained DOFs**	$\max\lvert u\rvert$	**Ratio**
Clamped	84	0.594	1.00
Roller (u_x only)	30	2.762	4.64×

This confirms: the per-DOF mechanism works correctly and produces a physically meaningful stiffer/more-flexible response.

8.5 State Capture and Continuation Analysis

8.5.1 ModelState Structure

```
template <size_t dim>
struct ModelState {
    static constexpr size_t nvoigt = dim * (dim + 1) / 2;

    struct NodalData {
        size_t node_id;
        array<double, dim> displacement;
        array<double, dim> velocity;        // zero for static
    };

    struct GaussPointData {
        array<double, nvoigt> stress;
        array<double, nvoigt> strain;
    };

    struct ElementData {
        size_t element_index;
        vector<GaussPointData> gauss_points;
    };

    vector<NodalData>   nodes;
    vector<ElementData> elements;
```

```
};
```

8.5.2 Capture Workflow

`Model::capture_state()` extracts:

1. **Nodal displacements:** from the PETSc local vector `current_state_` via `VecGetArrayRead` (zero-copy). Each node's DOF offsets from `node.dof_index()` are used to index into the raw array.
2. **Element stress/strain:** from each `MaterialPoint`'s committed strain (`current_state()`) and recomputed stress (`compute_response()`). The Voigt vectors are copied component-by-component into `std::array<double, nvoigt>`, guarded by the compile-time `num_components` constant.

8.5.3 Future: Initial Stress Integration

The existing `ElementInitialStress<dim>` struct in `BoundaryCondition.hh` can be populated from `ModelState::ElementData`:

```
for (const auto& ed : state.elements) {
    ElementInitialStress<dim> eis;
    eis.element_index = ed.element_index;
    for (const auto& gp : ed.gauss_points) {
        eis.gauss_point_stresses.push_back(gp.stress);
    }
    bcs.add_initial_stress(std::move(eis));
}
```

For this to take effect, the element assembly must incorporate $\boldsymbol{\sigma}_0$ into the internal force:

$$\mathbf{f}_{\text{int}}(\mathbf{u}) = \int_{\Omega_e} \mathbf{B}^T \left(\boldsymbol{\sigma}(\boldsymbol{\varepsilon}(\mathbf{u})) + \boldsymbol{\sigma}_0\right) \mathrm{d}V$$

This requires modifications to `ContinuumElement::compute_internal_force_vector`, specifically adding a σ`_initial` term at each Gauss point. This integration is deferred to a future phase.

8.6 Comparison with OpenSees and Kratos

Table 8.5: BC/IC handling comparison across frameworks.

Feature	OpenSees	Kratos	fall_n
Single-DOF fix	`fix nodeTag dof`	`DISPLACEMENT_X: 0.0`	`constrain_dof(n,d,v)`
Geometric selection	Tcl loops	Python `FindNodalH`	`PlaneSelector`, lambda
Non-zero Dirichlet	`imposedMotion` pattern	`ApplyDirichlet-Process`	`constrain_dof_where(s,d,v)`
State transfer	`-database` serialise	`ModelPartIO`	`ModelState` capture/apply
Initial stress	Per-element update	`INITIAL_STRESS_TENSOR`	Defined, not yet integrated
Dispatch cost	Virtual (runtime)	Virtual + Python (runtime)	Compile-time (zero)

Key differences:

- **OpenSees** uses a tag-based Tcl scripting layer for BC specification. Geometric loops require user scripting. State transfer uses a persistence layer (`FE_Datastore`).
- **Kratos** uses Python processes (`ApplyDirichletProcess`, `ApplyNeumannProcess`) that iterate over model parts. Highly flexible but with Python → C++ boundary overhead.
- **fall_n** resolves all selector logic at compile time. The constraint map lives in pure C++ with PETSc-section-based enforcement. No scripting layer, no virtual dispatch per node.

8.7 Extensibility Roadmap

The current infrastructure is designed for forward extension. Table 8.6 outlines planned capabilities and their implementation path.

Table 8.6: Extensibility roadmap.

Capability	Implementation path	Status
Per-DOF constraints	`constrain_dof`, `constrain_dof_where`	**Implemented**
Non-zero Dirichlet	`fill_imposed_solution()` + NL correction	**Implemented** (NL only)
Node selectors	`NodeSelector` concept + built-in selectors	**Implemented**
State capture/transfer	`ModelState` + `capture_state`	**Implemented**
Initial stress	Extend `ContinuumElement::compute_internal_force` to add $\boldsymbol{\sigma}_0$ at each GP	Designed, deferred
Body forces	Consistent nodal load via $\int N^T \mathbf{b}\, \mathrm{d}V$	Planned
Temperature field	Additional `ModelState` field + thermal strain $\boldsymbol{\varepsilon}_\theta = \alpha\, \Delta T\, \mathbf{1}$	Planned
Multi-step builder	Fluent builder with chained `.constrain().material().build()`	Planned
State serialisation	HDF5/JSON export of `ModelState`	Planned
Linear non-zero BC	RHS correction $\mathbf{f}_{\text{eff}} = \mathbf{f}_{\text{ext}} - \mathbf{K}_{fc}\, \mathbf{u}_D$ in `LinearAnalysis`	Planned

8.8 Test Suite

The implementation is validated by 81 tests in `tests/test_boundary_conditions.cpp`, organized as follows:

Table 8.7: Test suite structure (81 tests).

Category	Count	Coverage
Node selectors (pure geometry)	38	PlaneSelector, BoxSelector, SphereSelector, NodeIdSelector, combinators, la
ModelState data structure	9	Default state, add/query, copy semantics
Per-DOF constraints	11	Single DOF, multiple DOFs, non-zero value, value update
Selector-based constraints	10	PlaneSelector on DOF, all DOFs, values, lambda, composite
Full solve: roller vs. clamped	3	Clamped > 0, Roller > 0, Roller > Clamped
State capture after solve	7	Non-empty, node count, element count, non-zero displacement, non-zero str
Backward compatibility	3	`fix_x` gives positive deflection, reasonable magnitude

8.9 Summary

The generic BC/IC infrastructure extends `fall_n` from an all-or-nothing clamped-support tool to a framework capable of:

- **Arbitrary Dirichlet BCs:** per-DOF, per-node, per-selector, with prescribed values (zero or non-zero).
- **Zero-cost selectors:** compile-time geometric predicates with no runtime overhead versus hand-written loops.
- **State snapshots:** PETSc-independent `ModelState` for continuation, restart, and future serialisation.
- **Full backward compatibility:** `fix_x`, `fix_y`, `fix_z` continue to work exactly as before.

The design follows the same policy-based, compile-time polymorphic philosophy established in the element architecture (Chapter 7) and validated by an 81-test suite.

Chapter 9

ElementGeometry Refactor

This chapter documents the refactor of the `ElementGeometry` subsystem. The goal of the change was not to redesign the whole geometry layer from scratch, but to fix the most important semantic and architectural weaknesses without degrading the hot path of element integration.

The guiding constraint was explicit: *prefer compile-time cost over runtime cost whenever the trade-off is reasonable.* In other words, stronger type constraints, cleaner ownership, and slightly more template work at compile time are acceptable, whereas extra runtime indirection inside the Gauss loop is not.

9.1 Problem Statement

Before the refactor, `ElementGeometry<dim>` already had a good high-level idea: concrete geometry and quadrature strategy were stored behind a type-erased, value-semantic wrapper. However, three problems were architecturally significant.

9.1.1 Broken value semantics

The wrapper cloned only its internal `pimpl_`, while metadata such as `integration_point_`, `sieve_id`, and `physical_group_` lived outside the cloned state. As a consequence, copy construction and copy assignment were semantically incomplete: two `ElementGeometry` objects could share the same geometric map while disagreeing on already-materialised Gauss-point data or mesh metadata.

This violated a core expectation of value types:

$$\texttt{b = a} \quad \Longrightarrow \quad \texttt{b} \text{ must represent the same logical value as } \texttt{a}.$$

9.1.2 Integration strategy mixed into concrete geometry headers

The concrete geometry headers contained both the element type and the quadrature strategy implementation. This worked, but it blurred the boundary between two distinct concepts:

- a geometric interpolation on a reference cell, and
- a numerical integration policy on that reference cell.

Architecturally this was undesirable because the element family and the quadrature family evolve for different reasons.

9.1.3 Weak compile-time invariants

Some invalid states were still representable:

- unsupported Gauss–Legendre orders silently produced zero-filled arrays;
- `detJ()` was visible even for embedded geometries with non-square Jacobians;

- tensor-product geometries did not consistently enforce `topological_dim` $\leq$ `dim`.

These were not merely cosmetic issues. They weakened the library contract and made failures harder to interpret.

9.2 Design Goals

The refactor was driven by the following goals:

1. Preserve the existing *external polymorphism* structure.
2. Keep quadrature evaluation monomorphic and inlineable inside the concrete model.
3. Repair value semantics of the wrapper.
4. Make invalid geometric or quadrature configurations fail early.
5. Reduce conceptual coupling between geometry and integration.

Equally important is what was *not* attempted in this refactor: the geometry layer still depends on `Node<dim>`, which means it is not yet a purely geometric vertex layer. That larger ontological cleanup remains a future step because it cuts across `Model`, DOF storage, and multiple element families.

9.3 Refactor Overview

9.3.1 Metadata moved under wrapper ownership

`ElementGeometry` now owns an internal metadata aggregate containing:

- the materialised integration points,
- the optional DMPlex sieve identifier, and
- the physical-group label imported from Gmsh.

This has two consequences.

First, copy construction and copy assignment now copy both the cloned geometry implementation and the metadata, restoring proper value semantics.

Second, the metadata is no longer a public bag of fields. It is accessed through a narrow API: `sieve_id()`, `has_sieve_id()`, `physical_group()`, `integration_points()`, etc.

This is a direct SRP improvement: the wrapper remains responsible for its own logical state, and downstream code no longer reaches into its internals.

9.3.2 Quadrature strategies extracted to dedicated headers

Two strategy types were extracted:

- `GaussLegendreCellIntegrator.hh`
- `SimplexIntegrator.hh`

The concrete geometry classes remain pure descriptions of interpolation and mapping. The numerical integration policy is still attached at compile time, but it is no longer embedded in the same header for incidental reasons.

This separation is conceptually important. In isoparametric finite elements, geometry defines the map

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{a=1}^{n_n} N_a(\boldsymbol{\xi})\, \mathbf{x}_a,$$

while quadrature defines an approximation of

$$\int_{\hat{\Omega}} f(\boldsymbol{\xi})\, \mathrm{d}\hat{\Omega} \;\approx\; \sum_{g=1}^{n_g} w_g\, f(\boldsymbol{\xi}_g).$$

They are related in use, but they are not the same abstraction.

9.3.3 A small quadrature concept

The new `QuadratureStrategy` concept formalises the contract expected by `ElementGeometry`'s internal model:

- `num_integration_points`
- `reference_integration_point(i)`
- `weight(i)`

The benefit is not runtime polymorphism. The benefit is that failures move to compile time, where the error is cheaper and more local.

9.4 Geometry and Measure

The refactor keeps the existing zero-cost logic for the differential measure. Given the Jacobian $J = \partial \mathbf{x}/\partial \boldsymbol{\xi}$, the integration measure depends on the relationship between topological and embedding dimensions:

$$\text{full-dimensional cell:} \quad \mathrm{d}\Omega = |\det J|\,\mathrm{d}\hat{\Omega}, \tag{9.1}$$

$$\text{embedded curve:} \quad \mathrm{d}s = \|J\|\,\mathrm{d}\hat{s}, \tag{9.2}$$

$$\text{embedded surface in } \mathbb{R}^3: \quad \mathrm{d}S = \|J_1 \times J_2\|\,\mathrm{d}\hat{S}. \tag{9.3}$$

This dispatch still happens inside `OwningModel_ElementGeometry<ElementType, IntegrationStrategy>` using `if constexpr`. Therefore the compiler sees the concrete element type, the concrete topological dimension, and the concrete matrix shape. No extra runtime branch was introduced inside the integration loop.

9.5 API Hardening

9.5.1 Restricting `detJ()`

`detJ()` now exists only when the Jacobian is square, namely when the element is full-dimensional:

$$\texttt{topological_dim} = \texttt{dim}.$$

This is important because a determinant is not defined for the Jacobian of an embedded beam or surface. Exposing the member unconditionally was misleading: it suggested a mathematically meaningful operation where none exists.

9.5.2 Fail-fast Gauss–Legendre tables

Unsupported Gauss–Legendre orders no longer fall back to zero arrays. Instead, they trigger a compile-time error. This is the right contract for a template-based numerical kernel: if the requested order is unsupported, the program should fail where the invalid instantiation occurs.

9.5.3 Safer tensor-product constraints

`LagrangeElement<Dim, N...>` now enforces that the number of natural coordinates does not exceed the embedding dimension. This prevents invalid type combinations from entering the system.

9.6 Materialised Integration Points

The wrapper continues to precompute physical Gauss-point coordinates when the domain requests them. The refactor extends this step so that the integration weight is also stored in each materialised point.

This is useful for post-processing and point-wise objects attached to Gauss points because the point now carries a coherent local state:

- physical coordinates,
- global identifier for VTK export,
- quadrature weight.

The operation remains amortised and explicit. It does not change the quadrature path used in stiffness or residual evaluation.

9.7 Performance Discussion

The refactor was intentionally biased toward preserving runtime behaviour.

9.7.1 What did not change

- Concrete geometry evaluation is still template-based.
- The Jacobian and shape-function evaluation remain statically resolved.
- The quadrature loop still uses direct calls on the concrete strategy stored inside the model.
- No new virtual call was inserted inside `integrate()`.

9.7.2 What did change

- More errors are now diagnosed at compile time.
- Integration strategies live in dedicated headers, reducing conceptual coupling and making future dependency management easier.
- Copying `ElementGeometry` is slightly more expensive if integration points have already been materialised, because the copy now correctly copies them. This is not a regression; it is the real cost of correct value semantics.

This is the intended trade-off. Copying a fully materialised geometric entity is not a hot-path operation in FEM assembly, whereas incorrect copies are architecturally dangerous.

9.8 What Was Deliberately Deferred

The refactor does *not* yet solve the deeper issue that the geometry layer depends on `Node<dim>`, and `Node<dim>` already contains DOF state. From a strict modelling standpoint, a geometric layer should ideally depend on a lighter vertex concept carrying only:

- identifier,
- coordinates,
- optional mesh-backend handle.

That separation would improve the ontology of the codebase and further align the design with SRP and DIP. However, it affects much more than `ElementGeometry`; it is therefore better handled as a dedicated follow-up refactor instead of being folded into the present change.

9.9 Result

Architecturally, the refactor moves `ElementGeometry` from “good idea with leaky semantics” toward “stable boundary abstraction”.

The most important outcomes are:

1. wrapper copies are now semantically correct;
2. geometry and quadrature are more cleanly separated;
3. invalid numerical configurations fail early;
4. the hot numerical path remains compile-time resolved.

This is a meaningful improvement even though the subsystem is not yet the final form of the geometry layer. It removes a real correctness bug, strengthens the public contract, and prepares the code for a later, larger cleanup around geometric vertices versus analysis nodes.

Chapter 10

Serendipity Element Families

This chapter documents the implementation of the new serendipity element families added to `fall_n`, their integration into `ElementGeometry`, the associated tests, and the benchmark results.

The implementation goal was architectural, not merely additive: to introduce a new family without changing the overall layering of the codebase. In particular:

- the concrete element remains a pure value type;
- `ElementGeometry` continues to provide the type-erased boundary;
- quadrature remains a separate compile-time policy;
- no new virtual dispatch is introduced in the hot integration path.

10.1 Why Serendipity?

For tensor-product cells, the quadratic Lagrange family and the quadratic serendipity family differ substantially in node count:

Reference cell	Quadratic Lagrange	Quadratic serendipity
Quadrilateral	9 nodes	8 nodes
Hexahedron	27 nodes	20 nodes

The serendipity family removes face-center and volume-center nodes while retaining vertex and edge nodes. This makes the element attractive when the primary objective is to reduce geometry, interpolation, and assembly cost without dropping to first order.

Architecturally, serendipity is also a good stress test of the current design. It requires:

- a new reference cell type;
- a new concrete geometry type;
- VTK support for new node counts;
- optional Gmsh import support;
- no changes in the physical element layer.

That is precisely the extensibility story claimed in section 7.7; this chapter documents the claim becoming real code.

10.2 Mathematical Structure

10.2.1 Tensor-product serendipity on the cube

For the quadratic serendipity family on the reference hypercube $\hat{\Omega} = [-1, 1]^d$, nodes are located at:

- the 2^d vertices;

- the $d\,2^{d-1}$ edge midpoints.

Thus, for $d = 3$, the quadratic hexahedron has

$$2^3 + 3 \cdot 2^2 = 8 + 12 = 20$$

nodes.

Corner shape functions. Let $\mathbf{r} = (r_1, \ldots, r_d)$ be a corner node with $r_j \in \{-1, 1\}$. The quadratic serendipity shape function is

$$N_{\mathbf{r}}(\mathbf{x}) = 2^{-d}\left(\prod_{j=1}^{d}(1 + r_j x_j)\right)\left(1 + \sum_{j=1}^{d} r_j x_j - d\right).$$

Edge-midpoint shape functions. Let $\mathbf{r}$ be an edge node, so exactly one component $r_k = 0$ and all others are ± 1. Then

$$N_{\mathbf{r}}(\mathbf{x}) = 2^{1-d}(1 - x_k^2) \prod_{j \neq k}(1 + r_j x_j).$$

These expressions unify the 1D, 2D, and 3D quadratic serendipity family. The implementation therefore uses a single generic formula for `TopDim` $= 1, 2, 3$, with compile-time node descriptors and `consteval` reference-node tables.

10.2.2 Why the tetrahedral "serendipity" family is different

The simplex case requires more care. For tetrahedra at order $p = 2$, the standard quadratic Lagrange simplex already uses only:

- 4 vertices;
- 6 edge midpoints;
- no face-interior nodes;
- no volume-interior nodes.

Therefore the practical quadratic tetrahedral family is already "serendipity-like" in the sense relevant to this codebase: there are no face-center or body-center DOFs to remove.

For this reason, the new `SerendipitySimplexElement` is represented as an explicit family alias over the existing simplex implementation for orders 1 and 2. This is mathematically honest and avoids inventing an ad hoc reduced tetrahedron with unclear approximation properties.

10.3 Implementation Structure

10.3.1 Reference cells

The tensor-product family is implemented in `SerendipityCell.hh`. The file introduces:

- `serendipity_num_nodes(top_dim, order)`;
- `serendipity_reference_nodes<TopDim, Order>()`;
- `SerendipityBasis<TopDim, Order>`;
- `SerendipityCell<TopDim, Order>`.

Reference nodes are computed with `consteval`, so the full node layout is available at compile time and can participate in static assertions and compile-time topology generation.

10.3.2 Concrete geometry

The concrete tensor-product family is implemented in `SerendipityElement.hh` as

```
SerendipityElement<Dim, TopDim, Order>,
```

mirroring the shape of the existing `SimplexElement` and `LagrangeElement` abstractions:

- it stores node IDs and bound node pointers;
- it exposes `H()`, `dH_dx()`, `map_local_point()`, and `evaluate_jacobian()`;
- it carries a `ReferenceCell` type with compile-time topology.

The new family is therefore a first-class Layer 1 geometry type.

10.3.3 ElementGeometry integration

`ElementGeometry.hh` was extended in only one place: its face-topology and face-factory logic now recognises the serendipity family. No changes were required in the outer wrapper API.

This confirms the architectural objective: the extension point is the concrete geometry type, not the type-erasure layer.

10.3.4 Quadrature choices

The following quadratures were selected from the rules already present in the library:

Element	Quadrature	Rationale
HEX20	`GaussLegendreCellIntegrator<3,3,3>`	full quadratic tensor-product integration
QUAD8 face	`GaussLegendreCellIntegrator<3,3>`	compatible face integration
EDGE3 face	`GaussLegendreCellIntegrator<3>`	compatible edge integration
TET10 serendipity	`SimplexIntegrator<3,2>`	same requirements as quadratic simplex

The choice is intentionally conservative. The goal of this first integration is correctness and compatibility, not aggressive reduced integration.

10.4 VTK and Gmsh Support

10.4.1 VTK

`VTKCellTraits.hh` now recognises:

- 8-node quadratic serendipity quadrilaterals (`VTK_QUADRATIC_QUAD`);
- 20-node quadratic serendipity hexahedra (`VTK_QUADRATIC_HEXAHEDRON`).

The test executable writes four validation files:

- `data/output/serendipity_hex20_sample_mesh.vtu`
- `data/output/serendipity_hex20_sample_gauss.vtu`
- `data/output/serendipity_tet10_sample_mesh.vtu`
- `data/output/serendipity_tet10_sample_gauss.vtu`

These contain both the cell geometry and the materialised Gauss cloud, together with a simple scalar field for visual inspection in ParaView.

10.4.2 Gmsh

`GmshDomainBuilder.hh` was extended with support for:

- `HEX_20` (element type 17);
- `QUA_8` (element type 16) for boundary faces.

The chosen internal node ordering matches the standard corner-plus-edge layout, so no non-trivial permutation was required for these two element types.

10.5 Validation Tests

The new test executable `test_serendipity.cpp` validates:

1. static node counts for the family;
2. reference-node coordinates for HEX20;
3. partition of unity of the HEX20 basis;
4. Kronecker interpolation property at all 20 reference nodes;
5. exact reproduction of a representative quadratic polynomial;
6. unit-cube volume and mixed-monomial integration through `ElementGeometry`;
7. creation and integration of quadratic serendipity face geometry;
8. partition of unity and volume for the tetrahedral serendipity family;
9. VTK cell-type dispatch and actual file generation.

This test set is deliberately front-loaded toward interpolation quality. Before benchmarking or embedding the element into larger analyses, the basis itself must satisfy the core finite element identities.

10.6 Benchmark Results

The microbenchmark measures repeated geometric integration of a constant field through `ElementGeometry`. It was executed locally after the full test suite passed.

Case	Wall time [s]	Observation
HEX20 serendipity	6.04	baseline for 4000 integrations
HEX27 triquadratic	19.14	about 3.2× slower than HEX20
TET10 serendipity(simplex)	0.334	practically same as quadratic simplex
TET10 quadratic simplex	0.364	expected baseline

Interpretation. The HEX20 result is the important one. The node reduction from 27 to 20 lowers the cost of geometry interpolation and Jacobian assembly enough to be visible even though both elements use the same $3 \times 3 \times 3$ Gauss rule.

The tetrahedral result confirms the modelling claim made earlier: at quadratic order, "serendipity tetrahedron" and the existing quadratic simplex are effectively the same finite element family in this codebase.

Iteration after benchmarking. No further optimisation was required after the first measurement because the results already matched the intended architectural outcome:

- reduced tensor-product node count translated into lower runtime cost;
- simplex serendipity remained mathematically and computationally aligned with the existing implementation;
- no additional runtime indirection was introduced.

10.7 Architectural Takeaways

This extension validates several design choices of the current library:

1. **External polymorphism works.** A new family was added without redesigning `ElementGeometry`.
2. **Reference-cell-centric design scales.** Static reference nodes and subentity topology remain the right place to encode geometric knowledge.
3. **Geometry and physics remain decoupled.** The new elements plug into the same generic physical element and VTK machinery.

4. **The simplex/tensor-product split is conceptually sound.** The tetrahedral case did not need an artificial reduced basis, while the hexahedral case benefited from a genuinely different family.

10.8 Future Work

The present implementation intentionally stops at order 2. Sensible next steps would be:

- reduced-integration variants for HEX20;
- serendipity quadrilateral usage in shell formulations;
- higher-order incomplete tensor-product families if they are needed;
- a more explicit distinction in the type system between "quadratic simplex" and "serendipity simplex" if higher-order simplex variants are later introduced.

At this stage, however, the library now has a complete and usable quadratic serendipity pathway for both the tensor-product and simplex branches, with tests, benchmark coverage, and VTK output.

Chapter 11

Domain Refactor Ontology

This chapter does not yet present the final implementation of the `Domain` refactor. Its purpose is more foundational: to fix the *ontology* of the codebase before more invasive changes are made.

The current implementation already contains several useful abstractions: `Point`, `Node`, `IntegrationPoint`, `MaterialPoint`, `NodeSection`, `MaterialSection`, and `SectionGeometry`. The architectural difficulty is that some of these types represent *different concepts that happen to coincide geometrically*, while others represent *the same physical location under different numerical roles*. If those roles are not separated explicitly, `Domain` becomes the place where geometry, analysis state, quadrature, and constitutive bookkeeping collapse into a single layer.

The refactor should therefore begin by answering a simple question:

> What kinds of "points" exist in the library, and what exactly does each one mean?

11.1 Architectural Diagnosis

At present, the library contains three partially overlapping strata:

1. **Geometric entities:** reference cells, mapped element geometry, coordinates, frames, section shapes.
2. **Discrete unknown carriers:** nodal degrees of freedom, constrained DOFs, PETSc indexing.
3. **Sampling / constitutive sites:** Gauss points, material points, beam sections, shell sections.

The main architectural leak is that `Node` currently belongs to both the first and second strata at the same time. It is a geometric point, but it is also the owner of nodal DOF indices and PETSc-related mesh indexing. This is convenient for a first implementation, but it prevents a clean `Domain` abstraction because a geometry-only domain should not need to know anything about nodal unknown layouts.

There is a second leak in the repeated use of the word *section*. In the current code, *section* may refer to:

- a structural cross-section shape (`SectionGeometry`);
- a section plane located at a node (`NodeSection`);
- a constitutive section sample along a beam or shell (`MaterialSection`);
- a PETSc `PetscSection`, which is purely an indexing map and not a structural section at all.

These are not small naming issues. They correspond to genuinely distinct mathematical objects.

11.2 Entity-by-Entity Semantics

11.2.1 `Point<Dim>`

`Point<Dim>` is the cleanest entity in the current design. It is a coordinate value type: a point in physical space with no identity, no topology, no DOFs, no material, and no ownership semantics beyond ordinary C++ value semantics.

What it is. A metric object in $\mathbb{R}^{\mathrm{Dim}}$.

What it is not. It is not a mesh vertex, not a quadrature point, not a node in the FEM sense, and not a constitutive point.

Architectural role. `Point` should remain the lowest-level geometric atom. Any richer entity that has coordinates should refine or contain a `Point`.

11.2.2 `Vertex<Dim>` (proposed and introduced in this step)

`Vertex<Dim>` is the missing geometry/topology abstraction. It is a `Point` with a stable mesh identity, but without analysis state. In other words:

$$\texttt{Vertex} = \texttt{Point} + \texttt{mesh identity}.$$

What it is. A vertex of the discrete domain mesh. It represents a topological 0-cell and its physical coordinates.

What it is not. It is not a carrier of primary unknowns. It has no nodal DOFs, no constraint state, and no constitutive variables.

Why it matters. If `Domain` is supposed to live at the geometric level, then it should own `Vertex` objects, not `Node` objects. The new `Vertex.hh` introduced in this iteration fixes the intended target abstraction without forcing an immediate rewrite of every consumer.

11.2.3 `Node<Dim, Storage>`

`Node` is currently defined as

$$\texttt{Node} = \texttt{Point} + \texttt{id} + \texttt{DoF storage} + \texttt{PETSc/DMPlex indexing metadata}.$$

What it is. An analysis-layer discrete unknown carrier. More precisely, it is a nodal site where one or more primary fields are stored and indexed globally.

What it is not. It should not be interpreted as the pure geometric vertex of the domain.

Current status. The current implementation uses `Node` in both senses: as the thing owned by the domain, and as the thing that carries DOFs. This is precisely the coupling that the refactor should remove.

Target interpretation after refactor.

$$\texttt{Node} = \texttt{Vertex} + \texttt{field attachment}.$$

This does not necessarily mean inheritance. The important point is semantic: `Node` belongs to the analysis/model layer, not to the geometry/domain layer.

11.2.4 `IntegrationPoint<dim>`

`IntegrationPoint` is a sampling site used by numerical integration. It stores physical coordinates, a quadrature weight, and an identifier.

What it is. A numerical quadrature sample attached to an element geometry.

What it is not. It is not a mesh vertex and not a constitutive model by itself.

Important nuance. An integration point represents a *role*, not merely a location. Two different entities can occupy the same coordinates and still not be the same thing. This is crucial for understanding Lobatto collocation and mixed formulations.

Potential improvement. Eventually, `IntegrationPoint` should probably be understood as a *sample site* with:

- a physical coordinate;
- a reference coordinate;
- a numerical weight;
- possibly the differential measure $|J|$ or a cached metric.

The current implementation already covers the first and third items.

11.2.5 `MaterialPoint<MaterialPolicy>`

`MaterialPoint` is the constitutive analogue of a continuum Gauss point. It binds a `Material<Policy>` to an `IntegrationPoint`.

What it is. A constitutive state machine attached to a continuum sampling site.

What it is not. It is not geometry, not a mesh node, and not a quadrature rule.

Mathematical meaning. For a continuum element, the geometric interpolation produces the kinematic state at a sampling location; the material point then stores and updates the constitutive history at that location:

$$\texttt{ElementGeometry} \longrightarrow \texttt{IntegrationPoint} \longrightarrow \texttt{MaterialPoint}.$$

11.2.6 `SectionGeometry`

`SectionGeometry` is not a point at all. It is the geometric description of a structural cross-section: area, moments of inertia, torsional constant, and shear factors.

What it is. Cross-section *shape* data.

What it is not. It is not a location, not a node, and not a constitutive point.

Architectural consequence. The name is correct, but it must stay clearly separated from both `PetscSection` and `MaterialSection`.

11.2.7 `NodeSection<Dim, Geom>`

`NodeSection` combines:

1. a nodal location;
2. a local section frame;
3. a cross-section geometry.

What it is. A *section station* located at a node, that is, a nodal section plane with shape data and orientation.

What it is not. It is not a constitutive sampling point unless a formulation explicitly chooses to collocate section constitutive evaluation at that nodal station.

Why this distinction matters. In many beam and shell formulations, the nodal section plane is a geometric interpolation entity, whereas the constitutive update happens at different quadrature stations.

11.2.8 `MaterialSection<MaterialPolicy>`

`MaterialSection` is the structural analogue of `MaterialPoint`. It binds a section constitutive law to an integration point along a beam axis or shell midsurface.

What it is. A constitutive state machine attached to a structural sampling site.

What it is not. It is not the same as `SectionGeometry`, and not the same as `NodeSection`.

Mathematical meaning. It stores and updates generalized stress-resultants as functions of generalized strain measures:

$$\texttt{generalized strains} \mapsto \texttt{section resultants}.$$

Architectural observation. `MaterialPoint` and `MaterialSection` are role siblings. The first is a continuum constitutive site; the second is a structural constitutive site.

11.2.9 `PetscSection`

Although it is not a C++ entity defined in this library, it deserves explicit mention because it introduces the strongest terminological collision.

What it is. A PETSc map from mesh points to numbers of DOFs, offsets, and constraints.

What it is not. It is neither a structural section nor a constitutive section.

Architectural rule. `PetscSection` belongs to the solver/indexing infrastructure and should never be allowed to contaminate the semantics of structural section objects.

11.3 Coincidence Versus Identity

The most important conceptual rule for the refactor is the following:

> Two entities may coincide in space without being the same numerical object.

This rule resolves the apparent paradoxes raised by Lobatto quadrature and mixed formulations.

11.3.1 Case 1: Gauss–Lobatto beam quadrature

Suppose a beam formulation uses Gauss–Lobatto integration. Then the end quadrature points lie exactly at the beam endpoints. If the beam also uses nodal section stations at the endpoints, then:

- the *nodal section station* at the endpoint;
- the *quadrature point* at the endpoint;
- the *constitutive section sample* at the endpoint

all share the same physical coordinates.

That does *not* mean they should necessarily be represented by the same object. Their roles are distinct:

- the nodal station belongs to interpolation and field attachment;
- the quadrature point belongs to integration;
- the constitutive section sample belongs to material evolution.

The right design principle is therefore:

> collocation should be represented as a relationship, not as ontological collapse.

In practice, this means that a Lobatto endpoint can be modelled as *three roles sharing one location*, rather than as a single overloaded object.

11.3.2 Case 2: mixed or assumed-strain formulations

In formulations such as degenerated Timoshenko beams or MITC shell families, some sampling sites may acquire independent unknowns, tying variables, or assumed-strain parameters.

Here again, the key is to separate:

1. the geometric location of the sample;
2. the fact that it is used in integration;
3. the fact that additional discrete unknowns live there.

An integration point with independent unknowns is therefore *not* "a node in disguise"; rather, it is a sampling site that additionally plays the role of a field carrier.

Recommended interpretation. The library should treat this as:

$$\texttt{SampleSite} + \texttt{FieldAttachment},$$

not as a reason to redefine `IntegrationPoint` itself as a node.

This preserves both conceptual clarity and extensibility. The same sampling site can later carry:

- a constitutive law only;
- constitutive law plus internal variables;
- constitutive law plus independent element-local DOFs;
- tying or assumed-strain data without constitutive history.

11.4 Proposed Ontology for the Refactor

The cleanest layered ontology for `fall_n` is the following.

11.4.1 Layer 0: pure geometry/topology

- `Point<Dim>`: coordinates only.
- `Vertex<Dim>`: point + mesh identity.

- `ElementGeometry<Dim>`: mapped geometry, reference interpolation, quadrature mapping, face extraction.
- `Domain<Dim>`: owns vertices, element geometries, boundary groups, and PETSc mesh materialisation support, but no DOFs and no constitutive state.

11.4.2 Layer 1: discrete field attachment

- `Node`: a field-bearing attachment to a vertex, carrying nodal DOFs and constraints.
- future internal field sites: element-local unknown carriers for mixed methods or assumed-strain formulations.
- `PetscSection`: global indexing metadata for those field sites.

11.4.3 Layer 2: sampling and constitutive sites

- `IntegrationPoint`: numerical sample site.
- `MaterialPoint`: continuum constitutive site.
- `MaterialSection`: structural constitutive site.
- future tying or assumed-strain sites: sampled kinematic fields that may or may not have constitutive state.

11.4.4 Layer 3: structural section semantics

- `SectionGeometry`: cross-section shape properties.
- `NodeSection`: nodal section station = node + frame + section geometry.
- section constitutive relations (`TimoshenkoBeamSection3D`, `MindlinShellSection`, `FiberSection`): map generalized strains to resultants.

This layering produces a clean separation between:

1. *where something is*;
2. *how it is sampled*;
3. *what unknowns live there*;
4. *what constitutive law is evaluated there.*

11.5 Implications for `Domain`

Under this ontology, `Domain` should evolve toward the following responsibilities:

1. ownership of mesh vertices;
2. ownership of element geometry objects;
3. storage of boundary groups and physical labels;
4. translation to `DMPlex`;
5. topology and geometry queries.

And it should explicitly *not* own:

1. nodal DOF layouts;
2. constitutive points;
3. material state;
4. structural section stations;
5. solver vectors or matrices.

This has a direct consequence for the current code:

> `Domain` owning `Node` is architecturally temporary, not final.

The first code step taken in this iteration is therefore the introduction of `Vertex<Dim>` as the future geometry-level entity.

11.6 HPC and Compile-Time Guidance

The user requirement for the refactor is not only architectural cleanliness but also efficiency in a high-performance context. That requirement strongly favours a few design rules.

11.6.1 Prefer static structure in hot paths

The current library already uses static polymorphism effectively in the element kernel. That should continue. In particular:

- shape functions, quadrature rules, and reference-cell topology should remain compile-time objects whenever possible;
- constitutive updates should stay inside element-local loops with no new virtual dispatch;
- small fixed-size Eigen matrices are still the right choice for local operators.

11.6.2 Keep `Domain` simple, not hyper-generic

The right way to favour HPC here is *not* to template `Domain` on every imaginable backend or policy. That would increase compile times, fragment code paths, and create speculative abstraction.

The efficient choice is:

- keep `Domain` concrete and PETSc-oriented;
- separate concepts cleanly;
- minimise includes and cross-layer dependencies;
- keep backend-specific code concentrated in a few classes.

11.6.3 Avoid type collapse when roles overlap

Role overlap should be represented through composition and index maps rather than inheritance-heavy unions of responsibilities.

This means:

- a Lobatto endpoint may be both nodal station and quadrature sample, but that is best represented by shared coordinates or shared IDs, not by redefining one type to become the other;
- a mixed formulation with internal DOFs at sample sites should introduce a dedicated field attachment for those sites, not redefine every integration point in the library as a node.

11.6.4 Recommended storage philosophy

For the next stages of the refactor, the following storage strategy is the most coherent with the current codebase:

- `Vertex`: compact aggregate, trivially copyable if possible;
- `Node`: inline or SBO DOF storage as already implemented;
- `IntegrationPoint`: compact POD-like sample record;
- constitutive points: element-local vectors, because they are naturally owned by elements and evolved elementwise;
- PETSc indexing: stored in model/solver-side data structures, not in the geometric domain.

This preserves cache locality while keeping compile-time structure strong and runtime dispatch localized.

11.7 Migration Plan

The refactor should proceed incrementally.

Phase 1: fix ontology

- introduce `Vertex`;
- document the role separation;
- stop using "section" ambiguously in new code.

Phase 2: decouple geometry from nodal unknowns

- move `Domain` toward `std::vector<Vertex<dim>>`;
- make `ElementGeometry` depend on a geometry-only vertex concept rather than directly on `Node`;
- relocate DOF assignment and constraints fully to `Model`.

Phase 3: make field attachments explicit

- define how primary nodal unknowns are attached to vertices;
- introduce dedicated internal field sites for mixed methods when needed;
- keep collocation and coincidence as relationships, not type collapse.

Phase 4: unify constitutive-site patterns

- consider a shared abstraction behind `MaterialPoint` and `MaterialSection`;
- preserve policy-based constitutive dispatch and element-local storage.

11.8 Conclusion

The central conclusion of this chapter is simple:

> The library does not suffer from having too many abstractions; it suffers from having a few abstractions that currently play more than one role.

The remedy is therefore not more runtime polymorphism, but better ontological separation:

- `Point` for coordinates;
- `Vertex` for mesh geometry/topology;
- `Node` for field attachment and DOFs;
- `IntegrationPoint` for numerical sampling;
- `MaterialPoint`/`MaterialSection` for constitutive state;
- `SectionGeometry`/`NodeSection` for structural section geometry and nodal section stations.

Once this distinction is enforced, the `Domain` refactor becomes much less ambiguous. The domain can stay geometrically clean, the model can own the discrete unknowns, and the element layer can continue to exploit strong compile-time structure without sacrificing conceptual clarity.

Chapter 12

Domain Vertex Transition

Chapter 11 fixed the ontology of the refactor. This chapter documents the next implementation step: the domain now treats `Vertex` as the canonical geometric storage, while `Node` survives as a transitional analysis-layer cache.

The design goal of this step is deliberately modest:

> move the ownership boundary of `Domain` toward pure geometry without breaking the current element, model, and PETSc assembly pipelines.

That means this is not yet the final state of the refactor. It is the bridge state that keeps existing kernels working while removing the strongest geometric/analysis conflation.

12.1 Motivation

Before this step, `Domain` owned a vector of `Node<dim>`. That had three consequences:

1. the geometric domain depended directly on nodal DOF storage;
2. simply constructing a domain implied that the domain was already in an analysis-ready state;
3. purely geometric consumers such as mesh export were forced to read geometry through analysis nodes.

This was workable, but it obscured the intended layering:

$$\texttt{Domain} \not\equiv \texttt{Model}.$$

The refactor therefore changes the ownership model to:

`Domain owns vertices,  Model / assembly consume nodes.`

12.2 Implemented Transition Design

12.2.1 Canonical storage

`Domain` now stores

```
std::vector<Vertex<dim>>
```

as its primary geometric container. This is the authoritative representation of mesh points for:

- coordinates;
- mesh identity;
- sorting and lookup by vertex ID;
- geometry-only consumers.

12.2.2 Node cache

To preserve compatibility with the current element and model infrastructure, `Domain` also materialises a

```
std::vector<Node<dim>>
```

cache on demand. This cache is derived from the vertices and is used only when analysis-level operations require nodal DOFs or PETSc indexing.

This means the runtime interpretation is now:

- `vertices_` is the geometric truth;
- `nodes_cache_` is a derived analysis view.

Why this is acceptable. The cache is not introduced for performance in the hot path. It is a compatibility layer during migration. Since nodes are created once and then reused during analysis, the cost is negligible relative to element assembly and material updates.

12.2.3 Cache invalidation rule

Whenever the vertex container changes, the node cache is invalidated. The next request for `nodes()`, `node(id)`, or element-node binding rebuilds the cache.

This makes the design explicit:

> geometry changes invalidate analysis views.

That is the correct dependency direction.

12.3 Why This Preserves Performance

The user requirement for this refactor was clear: do not sacrifice runtime performance unnecessarily, and favour compile-time structure suitable for HPC.

This implementation respects that requirement in three ways.

12.3.1 No change to element hot loops

The integration, constitutive update, and local stiffness computations remain exactly where they already were:

- `ElementGeometry`;
- `ContinuumElement`;
- `BeamElement`;
- `TimoshenkoBeamN`;
- `ShellElement`.

Those kernels still operate on bound `Node` objects and continue to benefit from fixed-size matrices, compile-time quadrature, and policy-based dispatch.

12.3.2 Geometry-only consumers stop depending on nodes

The VTK mesh-export path was updated to read coordinates from `Domain::vertices()` rather than from `Domain::nodes()`. This is architecturally cleaner and slightly cheaper conceptually: geometry is now consumed as geometry.

12.3.3 PETSc creation is now lazy

An important side effect of the refactor is the change to `Mesh`. The `DM` object is no longer created in the `Mesh` constructor. Instead, it is created lazily when the DMPlex-specific operations are first needed.

This has two benefits:

1. a purely geometric `Domain` can be created and manipulated without immediately entering PETSc runtime semantics;
2. tests that exercise only the geometric/domain side no longer require `PetscInitialize`.

Architecturally, this is important because it moves PETSc from "always present" to "activated when the backend is actually used".

12.4 Code-Level Changes

The main code changes in this phase are:

1. `Vertex<Dim>` introduced earlier becomes the canonical storage type inside `Domain`.
2. `Domain` gains explicit vertex accessors: `vertices()`, `vertex(id)`, `num_vertices()`.
3. `Domain::nodes()` becomes a lazily materialised analysis view.
4. `GmshDomainBuilder` now sorts the domain through `sort_vertices_by_id()` rather than sorting the nodal view.
5. `VTKModelExporter` and the legacy `VTKDataContainer` now use vertices for mesh coordinates.
6. `Mesh` now creates the PETSc `DM` lazily.

This is intentionally a minimal-surface-area change. The aim was to shift the ownership boundary without forcing an immediate rewrite of `ElementGeometry` or `Model`.

12.5 What Remains Transitional

This phase does *not* yet complete the domain refactor.

The following are still transitional:

- `ElementGeometry` still binds concrete `Node` pointers, not geometry-only vertex handles.
- `Model` still reads and writes nodal DOF data through the domain-level node view.
- PETSc sieve IDs are still stored in `Node`, not yet in a dedicated analysis/indexing layer.

That is acceptable at this stage. The important structural improvement is that these analysis concerns are no longer the *canonical ownership model* of the domain itself.

12.6 Validation

This phase adds a dedicated test executable for the transition.

12.6.1 Domain/Vertex refactor test

The new test checks:

1. that vertices are the canonical stored geometry;
2. that the nodal view is materialised from those vertices;
3. that sorting vertices updates the nodal view coherently;
4. that analysis-only data written into the node cache does not alter the geometric meaning of the corresponding vertex;
5. that the legacy `add_node(Node&&)` path still preserves geometry correctly.

12.6.2 Regression coverage

The existing serendipity test suite was recompiled and rerun after this change, which is a useful regression check because it exercises:

- `ElementGeometry`;
- quadrature;
- VTK export;
- geometry binding through nodes.

In addition, the Gmsh pipeline target was rebuilt successfully, confirming that the builder and domain interface remain consistent at compile time.

12.7 Architectural Consequences

This transition has one especially important architectural consequence:

> `Domain` can now be understood as a geometric object that happens to be able to provide an analysis view, rather than as an analysis object that also stores geometry.

That inversion is subtle, but it matters. It clarifies the dependency direction for the next phase:

1. first, geometry is owned by the domain;
2. second, analysis nodes are attached or materialised from it;
3. third, elements and solvers consume those analysis nodes.

This is much closer to the ontology laid out in the previous chapter.

12.8 Next Step

The next logical step is now well defined:

> remove the direct `Node`-dependence from `ElementGeometry`.

That will require introducing a geometry-facing vertex handle or concept for binding coordinates, while the model layer separately manages nodal DOFs and PETSc indexing.

Once that is done, `Domain` will no longer need to expose a nodal cache at all, except perhaps as a legacy adapter.

12.9 Conclusion

This chapter marks the first real implementation step of the domain refactor. The main result is not a new feature but a new ownership boundary:

- vertices own domain geometry;
- nodes are an analysis view;
- mesh export reads geometry from vertices;
- PETSc mesh backend creation is deferred until needed.

No element hot path was slowed down, no quadrature logic was generalized away, and existing element kernels continue to operate unchanged. That is the right kind of progress for this refactor: architectural movement without numerical regression.

Chapter 13

Geometry-Only Element Binding

13.1 Context

After the first `Domain` transition, the library already stored `Vertex<Dim>` as the canonical geometric entity while keeping a lazy `Node<Dim>` cache for analysis. That solved the ownership problem in `Domain`, but a structural coupling remained in the element layer: `ElementGeometry` and the concrete element families still used `Node` as the source of physical coordinates.

Architecturally, that was still inconsistent with the ontology established in Chapter 11. A `Node` is an analysis attachment that carries degrees of freedom, PETSc numbering, and constraints. Geometry, however, only needs physical coordinates. The second phase of the refactor therefore separates these two bindings inside the element geometry layer.

13.2 Problem Statement

Before this change, a tensor-product, simplex, or serendipity element stored a single pointer array

$$\{\texttt{Node<dim>*}\}_{a=1}^{n_{\text{node}}}$$

and used that same array both for:

1. geometric interpolation and Jacobian evaluation, and
2. analysis-level access to nodal degrees of freedom.

This conflated two different roles:

$$\text{geometry site} \neq \text{analysis node}.$$

Even when both roles are collocated in space, their semantics are distinct. A continuum element can be fully geometrically defined from vertices without any DOF allocation. Conversely, an analysis formulation may decorate a point with DOFs, constraints, or solver indices without changing its geometric role.

13.3 Design Objective

The goal of this phase was to make the geometry layer depend only on the minimum abstraction required for isoparametric mapping:

`Point<Dim>` or a type derived from it.

At the same time, the analysis layer still needs direct access to `Node<Dim>` once PETSc sections and elemental DOFs are assigned.

The desired split is therefore:

`ElementGeometry` $\longrightarrow$ `Point/Vertex binding`	for $x(\boldsymbol{\xi}), J(\boldsymbol{\xi}), \mathrm{d}\Omega$,
`ElementGeometry` $\longrightarrow$ `Node binding`	for DoFs, constraints, and PETSc layout.

13.4 Implemented Refactor

13.4.1 New dual binding inside concrete elements

The three concrete element families

- `LagrangeElement`,
- `SimplexElement`, and
- `SerendipityElement`

now hold two logically distinct pointer arrays:

```
points_p : Point<Dim>*
nodes_p : Node<Dim>*
```

The geometry path uses only `points_p`. In particular:

- `coord_array()`,
- `map_local_point()`, and
- `evaluate_jacobian()`

no longer depend on `Node`.

The analysis path continues to use `nodes_p`. Methods such as `node_p()` and all element-level DOF collection code remain unchanged.

13.4.2 Wrapper-level support in `ElementGeometry`

The type-erased wrapper was extended with:

- `bind_point(i, Point<dim>*)`,
- `point_p(i)`,
- existing `bind_node(i, Node<dim>*)`, and
- existing `node_p(i)`.

This preserves the previous public API for analysis code while adding an explicit geometry-only binding channel.

13.4.3 Key compatibility rule

Whenever a `Node` is bound, the corresponding geometry point is also bound automatically. In symbols:

$$\texttt{bind_node}(i, n) \Longrightarrow \texttt{bind_point}(i, n).$$

This means:

- legacy code that only binds nodes still works unchanged;
- new geometry-first code can bind vertices before any DOFs exist.

13.5 Domain-Level Consequences

13.5.1 Geometry-first linking

`Domain` now exposes a dedicated `link_geometry_to_elements()` phase. It binds each element node id to its canonical `Vertex<dim>`:

$$\texttt{element.node(i)} \mapsto \texttt{vertex_by_id[element.node(i)]}.$$

This enables:

- geometric mapping,
- Jacobian evaluation,
- integration point placement, and
- geometry-driven boundary extraction

without first materialising the analysis-node cache.

13.5.2 Boundary extraction no longer needs `node_p().id()`

The plane-based boundary builder previously queried geometric membership through `elem.node_p(local).id(` which forced the existence of node bindings. After the refactor, face detection uses only the canonical element connectivity:

`elem.node(local).`

The resulting boundary element is then:

1. created from the face connectivity,
2. immediately bound to `Vertex` objects for geometry, and
3. optionally bound to `Node` objects if the analysis cache already exists.

13.5.3 Sieve assembly remains analysis-oriented

The PETSc sieve assembly still needs analysis nodes because sieve ids and DOF layout belong to the solver layer. However, even there the distinction is now cleaner:

- geometry binding prepares the element for integration and mapping;
- node binding prepares the element for DOF and PETSc interaction.

13.6 Why this is efficient

13.6.1 No new runtime polymorphism in the hot path

The critical loops for element integration remain unchanged:

- quadrature rule selection is still static via templates,
- local reference-space evaluation remains compile-time typed,
- Jacobian and interpolation work on fixed-size Eigen objects,
- the type-erased wrapper is still crossed only at the same boundary as before.

The only additional state is a pointer array to `Point<Dim>` inside each concrete element. Accessing `coord(i)` through a `Point` base reference is not virtual; it is just a normal base-subobject access. Therefore this refactor does not introduce a new dynamic-dispatch cost in the Gauss loop.

13.6.2 Compile-time impact

The change slightly increases template surface area in the element headers, but the trade-off is favourable:

- `Domain.hh` no longer needs to include Gmsh reader details;
- geometry-only workflows can avoid dragging analysis semantics into lower layers;
- future specialisations can target `Point`/`Vertex` independently of PETSc-related analysis infrastructure.

For an HPC-oriented codebase this is important because the hottest kernels remain statically shaped, while architectural boundaries become cleaner and easier to optimise separately.

13.7 Validation

The test suite was extended with geometry-first checks in the `domain_vertex_refactor` executable:

1. a quadrilateral element can map its centre and integrate area after binding only vertices;
2. a boundary element generated by `create_boundary_from_plane()` can map and integrate from vertex binding alone.

These tests validate the new invariant:

element geometry is usable as soon as geometry is bound, even if analysis nodes have not yet been materialised.

13.8 Architectural Interpretation

This phase does not eliminate `Node` from the library. It puts `Node` back in its proper role.

- `Vertex`: canonical domain geometry/topology entity;
- `Point`: minimal coordinate carrier;
- `ElementGeometry`: geometric interpolation operator over bound points;
- `Node`: analysis decorator attached later when the model introduces DOFs.

That separation is especially important for future mixed and structural formulations, where collocation between geometry sites, integration sites, and DOF-carrying sites may occur but should not collapse the underlying concepts.

13.9 Next Step

The next logical phase is to propagate this distinction upward:

- reduce remaining read paths that only need connectivity ids but still access `node_p()`;
- make geometry-only exporters and preprocessors depend only on `Vertex`/`Point`;
- move more of the solver-specific state management out of `Domain` and into the model/-analysis side.

At that stage, `Domain` will be geometrically pure not only in storage, but also in the majority of its operational paths.

Chapter 14

Low-Level Geometry and DoF Access

14.1 Motivation

Once the higher-level ontological split between `Vertex`, `Node`, and `ElementGeometry` was in place, the next question was lower-level: how much indirection remains in the hot path, and where does the code still pay avoidable access cost?

The target of this pass was not a redesign of algorithms. It was a careful data-access refinement guided by three HPC-oriented principles:

1. keep the public concepts explicit and semantically correct;
2. reduce repeated pointer chasing in frequently evaluated geometry code;
3. preserve value semantics and compile-time structure.

14.2 `Point<Dim>` as a contiguous data source

14.2.1 Previous state

`Point<Dim>` already stored coordinates in a `std::array<double, Dim>`. However, the main accessor returned that array by value. For a small fixed-size object this is not catastrophic, but it encourages copies and obscures the fact that the coordinates are already stored contiguously in memory.

14.2.2 Implemented change

`Point<Dim>` now exposes:

- `coord()` returning a *const reference* to the internal array for lvalues,
- `coord_ref()` as the explicit aliasing form, and
- `data()` returning a raw pointer to the contiguous coordinate storage.

This is a small but important clarification. It makes `Point` suitable not only as a semantic geometry entity but also as a low-level storage source for element kernels.

14.2.3 Why this matters

The geometry kernels repeatedly evaluate expressions of the form

$$x_i = \sum_a N_a(\boldsymbol{\xi})\, x_{a,i}, \qquad J_{ij} = \sum_a x_{a,i}\, \frac{\partial N_a}{\partial \xi_j}.$$

The coordinates $x_{a,i}$ are read many times. Making the contiguous storage explicit enables later code to cache direct pointers to those values instead of repeatedly calling higher-level accessors.

14.3 Node<Dim, Storage> and direct DoF access

14.3.1 Added direct pointer interface

Node already exposed dof_index() as a span, which is a good high-level interface. This pass adds dof_data() for the cases where the caller already knows the number of DOFs and only needs the raw contiguous pointer for PETSc interaction.

This is now used in selected PETSc-heavy paths such as boundary-condition and model bookkeeping code.

14.3.2 Array constructors

Node also gained constructors from std::array<double, Dim>. This reduces the need for std::apply-style reconstruction when a node is materialised from a geometry object such as Vertex<Dim>. In practice, this simplifies the Domain transition code and keeps template instantiation noise lower.

14.4 Cached coordinate pointers inside geometric elements

14.4.1 Problem

After the previous refactor, each concrete geometry family already stored a geometry-only binding to Point/Vertex. That fixed the architectural coupling, but the implementation still fetched coordinates through chains like

$$\texttt{points}[a] \to \texttt{coord()} \to \texttt{array}[i].$$

This is acceptable, but it is still more indirect than necessary inside the innermost geometry loops.

14.4.2 Implemented solution

The concrete families

- LagrangeElement,
- SimplexElement, and
- SerendipityElement

now cache, at bind time, direct pointers to the coordinate storage of each bound point:

$$\texttt{coord_data_[a]} = \texttt{point_a.data()}.$$

The geometric hot path then uses:

- coord_data_[a][i] in coord_array(),
- coord_data_[a][i] in evaluate_jacobian().

This removes a repeated layer of accessor traffic while preserving the semantic Point/Vertex binding.

14.4.3 Why this is safe

The cached coordinate pointers are non-owning views into the bound geometry objects. They are valid under the same condition that already governed the older point/node pointer bindings:

the lifetime and address stability of the bound geometry objects must exceed the lifetime of the bound element geometry.

This is not a new invariant; it is simply made more explicit.

14.5 Residual geometry-via-node usage in beam elements

14.5.1 Observation

Even after the previous phases, `BeamElement::compute_frame()` still read coordinates from `geometry_->node_p(0)` and `geometry_->node_p(1)`. That meant the beam frame computation still depended on analysis nodes even though it is purely geometric.

14.5.2 Implemented change

The beam frame is now built from:

`geometry_->point_p(0).coord_ref()`, `geometry_->point_p(1).coord_ref()`.

Therefore a beam element can compute its frame from geometry-only bindings before nodal DOFs are present.

14.6 Validation and traceability

This pass added or extended tests in three places:

1. `test_point.cpp`: verifies aliasing and contiguous pointer access for `Point`;
2. `test_node_dof.cpp`: verifies array construction and `dof_data()` consistency for `Node`;
3. `test_beam_element.cpp`: verifies that beam frame construction works from geometric vertices alone, without materialising analysis nodes.

These tests complement the earlier `domain_vertex_refactor` checks and make the refactor traceable from ontology down to low-level data access.

14.7 Interpretation

This chapter is intentionally modest in scope. It does not claim a microbenchmark-proven global speedup. What it does provide is a stricter and cleaner data path:

- geometry entities expose their contiguous storage explicitly;
- analysis entities expose their contiguous DOF storage explicitly;
- geometric elements cache the minimal data needed for repeated interpolation and Jacobian evaluation;
- structural elements use geometry where geometry is enough.

That is the right kind of optimisation for this codebase: local, transparent, semantically aligned, and compatible with the broader compile-time design.

Chapter 15

Spatial Sites and Lazy Domain Materialisation

15.1 Scope of this pass

This chapter documents the next incremental step of the `Domain` refactor. The change is deliberately modest in algorithmic scope and strict in architectural intent:

- keep `Domain` geometry-first;
- avoid materialising analysis nodes when only geometry is needed;
- make the different "point-like" entities comparable through a small static concept vocabulary;
- preserve the low-level requirements of HPC kernels: contiguous storage, inlining, and minimal pointer chasing.

The result is not a full removal of `Node` from the upper layers. It is an ontologically cleaner and computationally cheaper boundary between geometry, analysis, and constitutive sampling.

15.2 The problem of overlapping point-like concepts

Finite-element codes naturally generate several entities that look similar because they all carry coordinates. However, they are not the same concept. The overlap becomes especially visible in structural and mixed formulations:

- in Gauss–Lobatto beam integration, an endpoint may be both a nodal station and an integration station;
- in continuum formulations, a constitutive material point is collocated with a quadrature point but is not itself the quadrature rule;
- in mixed formulations, an integration site may also carry field unknowns, but that does not make it identical to a mesh vertex.

Therefore the correct design rule is:

same spatial position does not imply same semantic type.

The implementation should model collocation by *binding* or *decoration*, not by collapsing all roles into a single class.

15.3 A static ontology of spatial sites

15.3.1 Read-only coordinate site: `PointViewT`

This pass introduces a small compile-time concept, `geometry::PointViewT`. A type satisfies it if it exposes:

- a static `dim`,
- `coord()`,
- `coord(i)`,
- `coord_ref()`,
- `data()`.

This concept is intentionally read-only. It captures the minimum common geometry-facing contract needed by algorithms that only inspect coordinates.

15.3.2 Mutable point: `PointT`

`PointT` now refines `PointViewT` by adding coordinate mutation:

- `set_coord(i, value)`,
- `set_coord(array)`.

This split matters because not every site should be mutable. A constitutive wrapper such as `MaterialPoint` should forward coordinates from the integration point it decorates, but it should not pretend to own or mutate the geometry itself.

15.3.3 Entity-by-entity interpretation

With the new interface split, the point-like entities can be classified as follows:

`Point<Dim>` mutable geometric carrier; coordinates only.

`Vertex<Dim>` point plus stable mesh identity.

`Node<Dim, Storage>` point plus analysis identity, DoF storage, and PETSc-related node metadata.

`IntegrationPoint<Dim>` mutable quadrature site; coordinates, weight, and identifier.

`MaterialPoint<Policy>` read-only constitutive decorator bound to an `IntegrationPoint`.

`MaterialSection<Policy>` read-only constitutive decorator for structural sections, also bound to an `IntegrationPoint`.

`NodeSection<Dim>` read-only geometric and DoF-forwarding view over a `Node`, enriched with section frame and section geometry.

The important architectural detail is that only some of these types own coordinates. The others forward them.

15.4 Why forwarding is the right model

15.4.1 Material points and sections

A `MaterialPoint` does not own geometry. It owns constitutive state and binds to the numerical sampling site where that state is evaluated. Therefore it now forwards:

- `coord()`,
- `coord_ref()`,
- `data()`,
- `weight()`.

The same is true for `MaterialSection`. This makes both types valid `PointViewT` sites without duplicating coordinate storage.

15.4.2 Node sections

`NodeSection` already bound a `Node` plus section-specific metadata. This pass makes the forwarding explicit for:

- coordinates (`coord`, `coord_ref`, `data`),
- contiguous DoF storage (`dof_data()`).

This is particularly useful for structural formulations, because a section station is often treated as a special nodal view rather than as a new point owner.

15.5 Lazy node materialisation in `Domain`

15.5.1 Geometric operations should not force analysis state

The earlier `Domain` transition already made `Vertex<Dim>` the canonical geometric storage. However, there were still two remaining issues:

1. `create_boundary_from_plane()` still forced node-cache materialisation even when only geometry was needed;
2. the non-const `node_p(id)` path had an incorrect guard and could skip node-index construction.

15.5.2 Implemented fix

This pass changes the behaviour as follows:

- `create_boundary_from_plane()` is now geometry-first; it binds face elements to vertices immediately and binds nodes only if the node cache already exists;
- `node_p(id)` now reliably builds the node index on demand;
- `Domain` exposes three explicit traceability predicates: `has_materialized_nodes()`, `has_vertex_index()` `has_node_index()`.

The first predicate is important both for debugging and for architectural tests: it allows the test suite to assert that a geometry-only workflow truly stays geometry-only.

15.5.3 Why the node cache still exists

From a purely ontological point of view, one might want to remove `Node` duplication entirely and replace it with a thinner attachment structure. That direction is still plausible, but it should not be taken lightly.

The current element kernels, structural sections, PETSc section setup, and element-level DoF caches still benefit from a stable `Node&` view with:

- inline coordinate access,
- contiguous DoF access,
- zero-cost SBO-backed DoF storage policies,
- no proxy objects in the inner assembly path.

Keeping the node cache while making it lazy is therefore the pragmatic design point for this stage of the refactor.

15.6 Data-layout implications for HPC

15.6.1 Owned storage vs forwarded storage

The implementation now follows a simple rule:

- `Point`, `Vertex`, `Node`, and `IntegrationPoint` own contiguous coordinate storage;
- `MaterialPoint`, `MaterialSection`, and `NodeSection` do not own coordinates and only forward views.

This keeps the hot data local and prevents hidden copies when decorating an existing site with extra semantics.

15.6.2 Why no generic runtime interface was added

An obvious alternative would be an inheritance hierarchy such as:

$$\texttt{ISpatialSite} \leftarrow \{\texttt{Vertex}, \texttt{Node}, \texttt{IntegrationPoint}, \dots\}$$

with virtual coordinate access.

That was intentionally rejected. The reasons are both architectural and computational:

- the library already has enough runtime polymorphism at the `ElementGeometry` and `Material` wrappers;
- coordinate access is too frequent to justify another virtual boundary;
- the overlap between site types is semantic, not behavioural in the object-oriented sense.

Compile-time concepts are the better fit here: they make the common contract explicit without imposing a vtable or pointer-indirection tax.

15.7 What this means for overlapping formulations

The new model clarifies how to think about several important special cases:

Gauss–Lobatto beam integration an endpoint may be represented by a `NodeSection` and by a `MaterialSection` bound to an `IntegrationPoint` at the same coordinate. The location is shared; the role is not.

Continuum material points a `MaterialPoint` is bound to an `IntegrationPoint`. It is not itself the quadrature rule and not a mesh node.

Mixed or assumed-strain formulations if a sampling site later carries unknowns, that should be modelled as an additional attachment or dedicated type, not by erasing the distinction between integration and nodal roles.

This is the key conceptual payoff of the refactor: the code can represent collocation without ontological collapse.

15.8 Validation

This pass was validated by extending the test suite in two directions:

1. `domain_vertex_refactor` now verifies that geometry-only element binding and boundary creation do not materialise the node cache;
2. `integration_sites` verifies that `IntegrationPoint`, `MaterialPoint`, `MaterialSection`, and `NodeSection` satisfy the new static geometry-view contract.

This complements the earlier low-level tests on contiguous access and keeps the refactor traceable from architecture to implementation.

15.9 Interpretation

The central lesson of this step is that the library does not need fewer concepts; it needs cleaner boundaries between them. The right optimisation was not to merge `Point`, `Node`, `IntegrationPoint`, and `Section`-like entities. The right optimisation was:

- to expose their common geometric contract statically,
- to preserve ownership only where ownership is real,
- to keep `Domain` geometry-first for as long as possible,
- to materialise analysis state only when the solver path requires it.

That is a better match for both SOLID-style concept clarity and HPC-oriented implementation discipline.

Chapter 16

Adaptive Nodal Projection for Material Fields

This chapter refines the earlier diagnosis of the TET10 post-processing artefact and documents the resulting architectural change in `VTKModelExporter`. The key conclusion is that *negative quadrature weights are not the only failure mode.* For quadratic simplex elements, the row-sum lumped nodal measures can be non-positive even under all-positive quadrature. Therefore the exporter now chooses the projection operator adaptively.

16.1 Why the Previous Diagnosis Was Incomplete

chapter 3 correctly identified a catastrophic failure mode: if a simplex rule contains negative weights, then the signed support of the quadratic simplex basis can drive the lumped L^2 denominator to zero or to the wrong sign.

However, the ParaView artefact observed for the current TET10 path persisted even after switching to all-positive quadrature rules such as:

- the 4-point Hammer–Marlowe–Stroud rule (`SimplexIntegrator<3,2>`), and
- the 27-point Stroud conical product rule (`ConicalProductIntegrator<3,3>`).

That observation immediately rules out "negative quadrature weights" as the sole root cause. The actual issue lies in the *row-sum lumping itself.*

16.2 Exact Row-Sum Lumping for TET10

For the quadratic tetrahedron, the vertex and edge shape functions are:

$$N_i^{(v)} = \lambda_i(2\lambda_i - 1), \tag{16.1}$$

$$N_{ij}^{(e)} = 4\lambda_i\lambda_j, \tag{16.2}$$

where λ_i are barycentric coordinates.

The lumped nodal measure attached to node I is

$$m_I^{(\ell)} = \int_T N_I \,\mathrm{d}V. \tag{16.3}$$

Using the standard simplex moment formula

$$\int_T \lambda_1^{a_1}\lambda_2^{a_2}\lambda_3^{a_3}\lambda_4^{a_4}\,\mathrm{d}V = |T|\,\frac{a_1!a_2!a_3!a_4!\,3!}{(a_1+a_2+a_3+a_4+3)!}, \tag{16.4}$$

we obtain:

16.2.1 Vertex nodes

$$\int_T N_i^{(v)}\,\mathrm{d}V = \int_T \left(2\lambda_i^2 - \lambda_i\right)\,\mathrm{d}V \tag{16.5}$$

$$= 2\frac{|T|}{10} - \frac{|T|}{4} \tag{16.6}$$

$$= -\frac{|T|}{20}. \tag{16.7}$$

16.2.2 Edge-midpoint nodes

$$\int_T N_{ij}^{(e)}\,\mathrm{d}V = 4\int_T \lambda_i\lambda_j\,\mathrm{d}V \tag{16.8}$$

$$= 4\frac{|T|}{20} \tag{16.9}$$

$$= \frac{|T|}{5}. \tag{16.10}$$

Therefore, for *every* non-degenerate TET10:

$$m_i^{(\ell)} = -\frac{|T|}{20} < 0 \qquad \text{for the four vertex nodes,} \tag{16.11}$$

and

$$m_{ij}^{(\ell)} = \frac{|T|}{5} > 0 \qquad \text{for the six edge nodes.} \tag{16.12}$$

This is an exact statement. It does not depend on the particular positive quadrature rule, provided the rule is at least degree-2 exact.

16.3 Consequence: Lumped L^2 Is Not a Nodal Average

For a nodal projection to behave as an average, the local weights attached to each node must be strictly positive. TET10 violates that requirement at its vertex nodes by construction.

This has two implications:

- The projected nodal value is no longer an average of nearby Gauss-point data.
- The operator behaves like a signed extrapolation, which can overshoot even when the Gauss-point field is smooth and monotone.

This explains the visual symptom seen in ParaView:

- the displacement field is correct, because it is a primary nodal field;
- the Gauss-point strains are correct, because they come directly from the element kinematics;
- the *projected* nodal strains are oscillatory for TET10, because the transfer operator is not a proper averaging operator.

Hence the issue is fundamentally a *post-processing transfer* problem, not a stiffness integration problem.

16.4 Why HEX27 Remains Safe

For the quadratic tensor-product hexahedron, the shape functions are products of one-dimensional quadratic Lagrange polynomials on $[-1, 1]$. Their lumped nodal measures remain strictly positive under the standard $3 \times 3 \times 3$ Gauss–Legendre rule used in `fall_n`. Therefore the row-sum lumped projection still behaves as a genuine nodal average and produces smooth fields.

This distinction is important:

Family	Local lumped nodal weights	Safe to use lumped L^2 by default?
HEX27	strictly positive	yes
TET10	mixed sign	no

16.5 Adaptive Policy Implemented in `VTKModelExporter`

The exporter now exposes:

```
enum class MaterialFieldProjection {
    Auto,
    LumpedL2,
    PolynomialPatchRecovery,
    VolumeWeightedAveraging
};
```

The production path is `Auto`. Its algorithm is:

1. For each element geometry, compute the local lumped nodal weights

$$m_i^{(\ell)} \approx \sum_g N_i(\boldsymbol{\xi}_g)\, w_g\, |J_g|.$$

2. If *all* local weights are strictly positive for *all* elements, use lumped L^2 projection.
3. Otherwise, use polynomial patch recovery (`patch_recover_to_nodes()`).
4. Retain volume-weighted averaging only as an explicit low-order fallback path.

This choice is conservative and appropriate for production visualisation:

- it preserves the smoother lumped L^2 path for element families where it is mathematically sound;
- it automatically protects higher-order simplex elements without forcing the user to know the details of the underlying quadrature rule;
- it avoids the excessive diffusion of a pure volume-average fallback by using a real local reconstruction on unsafe patches.

Complexity. The detection pass costs $O(n_{\text{elem}}\, n_{\text{gp}}\, n_{\text{node}})$, which is of the same order as the projection itself and negligible compared with an FE solve. Because this happens only during post-processing, the architectural priority is robustness and traceability, not micro-optimisation.

16.6 Code-Level Traceability

The implementation is localised in:

- `src/post-processing/VTK/VTKModelExporter.hh` — `MaterialFieldProjection`, `lumped_projection` `has_strictly_positive_lumped_projection_weights()`, `patch_recover_to_nodes()`, and adaptive dispatch in `compute_material_fields()`.
- `main.cpp` — unchanged call site: `compute_material_fields()`, but its semantics are now adaptive rather than hard-coded lumped L^2.
- `tests/test_simplex.cpp` — regression tests that prove:
  - TET10 keeps negative vertex lump weights under positive quadrature;
  - HEX27 keeps strictly positive lump weights.
  - The new patch-recovery path reproduces exact linear and quadratic fields on TET10 patches.

16.7 Verification

Two levels of verification were added:

1. **Mathematical regression:** the simplex test suite now checks the exact TET10 lumped nodal weights under the 4-point positive HMS rule, obtaining $-\frac{1}{120}$ at the four vertices and $\frac{1}{30}$ at the six edge nodes of the reference tetrahedron.
2. **Implementation regression:** the same suite verifies that the 27-point Stroud conical product rule also yields negative TET10 vertex lumps despite its strictly positive quadrature weights, while HEX27 remains strictly positive. It also verifies exact linear recovery on a single TET10 and exact quadratic recovery on a multi-element TET10 patch.

These checks establish the core architectural point:

> For TET10, the unsafe behaviour is not caused by the constitutive model, nor by the computed Gauss-point strains, nor by the positivity of the stiffness quadrature weights. It is caused by using row-sum lumped nodal projection on a shape-function family whose local lumped nodal measures are signed.

Chapter 17

Polynomial Patch Recovery for TET10 Visualisation

This chapter documents the second-stage fix for the TET10 visualisation path. The first fix — avoiding unsafe lumped L^2 projection — was necessary but not sufficient. Replacing signed lumping with a plain volume-weighted average removed the gross artefact, but it also produced a field that was too diffusive for smooth contour rendering. The proper solution is a local polynomial reconstruction over nodal patches.

17.1 Why Averaging Was Not Enough

The TET10 strain field in the cantilever benchmark is smooth at the level of Gauss-point samples: the displacement solution is correct, the kinematics are correct, and the constitutive update is correct. The visual defect appears only after transfer to the nodes.

Volume-weighted averaging fixes the sign problem but throws away too much information:

- it compresses each element to a single average value before nodal transfer;
- it cannot reproduce linear or quadratic intra-element variation;
- it therefore damps gradients that should remain visible in the final contour plot.

For higher-order simplex elements this is the wrong trade-off. The data are already rich enough to support a better local fit, so a production-quality visualisation path should use them.

17.2 Recovered Quantity and Patch Topology

Let node I have coordinate $\mathbf{x}_I$. Its patch is the set of all elements incident on that node:

$$\mathcal{P}_I = \{e \in \mathcal{E} \mid I \in e\}. \tag{17.1}$$

The exporter gathers every Gauss point of every element in $\mathcal{P}_I$. Each observation contributes:

$$(\mathbf{x}_g,\ \sigma_g,\ \omega_g), \tag{17.2}$$

where $\mathbf{x}_g$ is the physical coordinate of the sample, σ_g is the material field component value, and $\omega_g = |w_g J_g|$ is the positive sample weight.

Important distinction. The recovery patch is a *post-processing construct*. It is not part of the FE approximation space, does not add new unknowns, and does not alter the element stiffness formulation. It exists solely to reconstruct a nodal field for rendering.

17.3 Centered and Scaled Basis

To reduce conditioning problems and keep the constant coefficient equal to the recovered nodal value, the polynomial basis is centered at the node:

$$\hat{\mathbf{x}}_g = \frac{\mathbf{x}_g - \mathbf{x}_I}{h_I}, \qquad h_I = \max_{g \in \mathcal{P}_I} \|\mathbf{x}_g - \mathbf{x}_I\|_2. \tag{17.3}$$

For the 3D case, the quadratic basis is

$$\mathbf{p}(\hat{\mathbf{x}}) = \begin{bmatrix} 1 & \hat{x} & \hat{y} & \hat{z} & \hat{x}^2 & \hat{x}\hat{y} & \hat{x}\hat{z} & \hat{y}^2 & \hat{y}\hat{z} & \hat{z}^2 \end{bmatrix}^{\mathsf{T}}. \tag{17.4}$$

With this choice,

$$\tilde{\sigma}_I = p_I(\mathbf{x}_I) = a_{I,0}, \tag{17.5}$$

because all non-constant basis terms vanish at the origin of the local patch coordinates.

17.4 Least-Squares System

For one field component, the recovery solves

$$\mathbf{a}_I = \arg\min_{\mathbf{a}} \sum_{g \in \mathcal{P}_I} \omega_g \left(\mathbf{p}(\hat{\mathbf{x}}_g)^{\mathsf{T}} \mathbf{a} - \sigma_g \right)^2. \tag{17.6}$$

The implementation accumulates the normal equations

$$\mathbf{N}_I = \sum_g \omega_g \, \mathbf{p}_g \mathbf{p}_g^{\mathsf{T}}, \qquad \mathbf{r}_I = \sum_g \omega_g \, \mathbf{p}_g \sigma_g, \tag{17.7}$$

and solves

$$\mathbf{N}_I \mathbf{a}_I = \mathbf{r}_I. \tag{17.8}$$

Why use $|w_g J_g|$? In numerical integration the sign of w_g matters because it contributes to an integral approximation. In patch recovery the role of the weight is different: it measures the local influence of a sample. A negative quadrature weight should therefore *not* negate the influence of a perfectly valid sample in the least-squares fit. Using $|w_g J_g|$ is the correct semantics for recovery.

17.5 Rank-Adaptive Fallback

Not every nodal patch can support a full quadratic fit:

- a boundary node may see too few neighbouring elements;
- a coarse mesh may provide too few independent Gauss points;
- a highly directional patch may lead to rank deficiency.

The exporter therefore uses the following hierarchy:

1. quadratic fit if the patch has enough samples and full rank;
2. linear fit if the quadratic system is rank-deficient;
3. weighted average if even the linear fit is not admissible.

This hierarchy matters in practice. A single TET10 contributes four HMS Gauss points, which is not enough for a quadratic 3D fit (ten coefficients) but is exactly enough for a linear fit (four coefficients). The new regression tests check this explicitly.

17.6 Implementation Choices and HPC Considerations

The implementation in `VTKModelExporter.hh` was written to avoid contaminating the assembly kernels with post-processing concerns:

- all recovery logic is isolated in the exporter;
- the FE solve path, material update path, and element geometry kernels are unchanged;
- the recovery loop uses geometry already bound in the elements, so no new ownership or pointer chains were introduced into the hot path;
- the least-squares accumulation uses fixed-size small Eigen matrices for the normal equations, with only a tiny dynamic solve on the active block size.

This is the right architectural compromise for `fall_n`:

- **compile-time and runtime discipline** are preserved in the FE kernels;
- **visualisation quality** is upgraded where it belongs, in post-processing;
- **traceability** remains high because the policy is explicit in `MaterialFieldProjection`.

17.7 Regression Coverage

Two new tests were added in `tests/test_simplex.cpp`:

1. `test_tet10_patch_recovery_recovers_linear_field_on_single_element()` verifies that the fallback from quadratic to linear fit works and is exact for a linear field on one straight-sided TET10.
2. `test_tet10_patch_recovery_recovers_quadratic_field_on_patch()` verifies exact recovery of a quadratic scalar field at a shared patch node using multiple TET10 elements and real simplex quadrature points.

These tests are more valuable than a screenshot-based validation because they check the transfer operator directly, independently of ParaView rendering settings.

17.8 Architectural Outcome

The final picture is now clean:

- **HEX27** stays on lumped L^2 because its local nodal lumping is positive and well behaved.
- **TET10** automatically switches to polynomial patch recovery because signed lumping makes row-sum L^2 unsuitable as a nodal average.
- **Volume averaging** remains available for debugging and for the rare case where a deliberately diffusive fallback is preferred.

This resolves the remaining mismatch between mathematically correct Gauss-point fields and visually acceptable nodal contours without degrading the performance-oriented architecture of the main finite element code.

Chapter 18

Explicit Tensor Filters for VTK / ParaView

The current VTK contract in `fall_n` is intentionally simple:

- `*_mesh.vtu` stores the continuum mesh and nodal fields;
- `*_gauss.vtu` stores one point per material point and the quadrature fields `qp_*`;
- both datasets can carry `displacement`, and the Gauss cloud receives the interpolated material-point displacement explicitly.

Within that split workflow, the remaining post-processing problem is not data transport but *field semantics*. A raw array named simply `stress` or `strain` is a weak contract: it hides both the carrier and the tensor convention. This chapter documents the cleaned-up naming and the explicit scalar filters now emitted by the exporter.

18.1 Naming Contract

The exporter distinguishes both *location* and *representation*:

- `nodal_*`: values projected onto mesh nodes;
- `qp_*`: values evaluated or stored at material points;
- `_voigt`: raw symmetric-tensor storage in the library's Voigt convention;
- scalar suffixes such as `_xx`, `_von_mises`, or `_equivalent_strain`: explicit post-processing views.

Examples:

- `qp_stress_voigt`, `qp_stress_xx`, `qp_stress_von_mises`;
- `nodal_strain_voigt`, `nodal_strain_xx`, `nodal_strain_equivalent_strain`;
- `qp_plastic_strain_voigt`, `nodal_plastic_strain_equivalent_strain`.

This is the main usability improvement. The user no longer has to infer whether a field is nodal or quadrature-based, and the exporter no longer asks ParaView to reinterpret unnamed raw tensors on the fly.

18.2 Stress Filters

For a symmetric stress tensor $\boldsymbol{\sigma}$, the exporter emits:

- component scalars $\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{yz}, \sigma_{xz}, \sigma_{xy}$;
- trace
$$\mathrm{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz};$$
- mean / hydrostatic stress
$$\sigma_m = \sigma_h = \frac{1}{d}\,\mathrm{tr}(\boldsymbol{\sigma});$$

- pressure
$$p = -\sigma_m;$$
- deviatoric norm
$$\|\operatorname{dev}\boldsymbol{\sigma}\| = \sqrt{\operatorname{dev}\boldsymbol{\sigma} : \operatorname{dev}\boldsymbol{\sigma}};$$
- second deviatoric invariant
$$J_2 = \frac{1}{2}\operatorname{dev}\boldsymbol{\sigma} : \operatorname{dev}\boldsymbol{\sigma};$$
- von Mises / Beltrami–Haigh equivalent stress
$$\sigma_{\mathrm{vm}} = \sigma_{\mathrm{BH}} = \sqrt{\frac{3}{2}\operatorname{dev}\boldsymbol{\sigma} : \operatorname{dev}\boldsymbol{\sigma}};$$
- triaxiality
$$\eta = \frac{\sigma_m}{\sigma_{\mathrm{vm}}};$$
- octahedral shear stress
$$\tau_{\mathrm{oct}} = \sqrt{\frac{2}{3}J_2}.$$

The exporter writes both `von_mises` and `beltrami_haigh`. In this context they are numerically the same J_2-based equivalent stress, but exposing both names removes friction for users who search for one label or the other in ParaView.

18.3 Strain Filters

The strain arrays are stored in engineering Voigt notation:

$$\{\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{yz}, \gamma_{xz}, \gamma_{xy}\}, \qquad \gamma_{ij} = 2\varepsilon_{ij}.$$

The exporter therefore converts the off-diagonal entries back to tensor shear components before evaluating invariants. The derived strain filters are:

- component scalars in the stored engineering convention;
- trace / volumetric strain
$$\varepsilon_v = \operatorname{tr}(\boldsymbol{\varepsilon});$$
- deviatoric norm
$$\|\operatorname{dev}\boldsymbol{\varepsilon}\| = \sqrt{\operatorname{dev}\boldsymbol{\varepsilon} : \operatorname{dev}\boldsymbol{\varepsilon}};$$
- equivalent strain
$$\varepsilon_{\mathrm{eq}} = \sqrt{\frac{2}{3}\operatorname{dev}\boldsymbol{\varepsilon} : \operatorname{dev}\boldsymbol{\varepsilon}}.$$

The same reconstruction is applied to plastic-strain-like quantities, which is why arrays such as `qp_plastic_strain_xx` and `nodal_plastic_strain_equivalent_strain` are available without any ParaView-side calculator.

18.4 Why the Exporter Owns These Scalars

There are two architectural reasons to compute the scalar filters inside `fall_n`:

1. the library owns the tensor convention and therefore should own the conversion rules;
2. explicit scalar arrays are stable extension points for downstream scripts, regression tests, dashboards, and future post-processing modules.

This is compatible with the HPC direction of the library. The additional work is restricted to post-processing, while the FE hot path remains unchanged.

18.5 Recommended ParaView Usage

For smooth contours on the continuum mesh:

1. load `*_mesh.vtu`;
2. if desired, apply `Warp By Vector` with `displacement`;
3. colour by `nodal_stress_von_mises`, `nodal_strain_xx`, or any other explicit nodal scalar.

For material-point inspection:

1. load `*_gauss.vtu`;
2. apply `Warp By Vector` directly using `displacement`;
3. colour by `qp_stress_von_mises`, `qp_stress_xx`, `qp_strain_equivalent_strain`, or similar explicit scalars;
4. use `Points` or `Point Gaussian` representation as appropriate.

The raw `_voigt` arrays remain in the files for traceability and advanced tooling, but they are no longer the primary user-facing representation.

Chapter 19

VTK Export Cleanup

This chapter documents the cleanup pass that reduced the VTK layer to the smallest operational contract that still preserves extension points.

19.1 Retained Responsibilities

After the cleanup, the active VTK surface in `fall_n` is:

- `VTKCellTraits.hh`: mapping from `fall_n` topology to VTK cell ids and local-node permutations;
- `VTKTensorFieldDerivatives.hh`: deterministic tensor-to-scalar post-processing utilities;
- `VTKModelExporter.hh`: split export of the continuum mesh and the Gauss cloud;
- `PVDWriter.hh`: time-series collection writer.

This is enough for the current library needs:

- clean nodal contour plots on the mesh;
- faithful material-point inspection on a warpable Gauss cloud;
- explicit scalar invariants for ParaView;
- time-series support through `.pvd`.

19.2 Removed Complexity

The cleanup explicitly removed three categories of dead or unstable implementation:

1. embedded native quadrature export as part of the main exporter API;
2. the mixed single-file visualization mesh that combined continuum cells and Gauss glyphs in one `.vtu`;
3. broad VTK rendering and GUI dependencies from the common include path.

These paths were not removed because they were impossible. They were removed because they increased API surface, state management, and cognitive load without yielding a robust everyday workflow in ParaView.

19.3 Why the Split Export Is the Right Current Boundary

The split contract is not merely a fallback; it is the correct current boundary between FE post-processing and VTK:

- the continuum mesh remains semantically clean;
- the Gauss cloud carries only material-point quantities;
- `Warp By Vector` works directly on both datasets;
- ParaView is not asked to reconstruct semantics from ambiguous metadata;
- the exporter state is easy to test and reason about.

This matters architecturally. A post-processing layer should not be clever just because VTK allows it; it should be explicit enough that future extensions do not inherit accidental complexity.

19.4 Compile-Time and Dependency Impact

The cleanup also reduced compilation and linkage overhead:

- `header_files.hh` no longer pulls the broad `VTKheaders.hh` compatibility umbrella;
- the compatibility umbrella itself was reduced to minimal data-model and IO includes for legacy code;
- the CMake VTK dependency set now targets only the modules needed by the active exporter and tests.

This follows the library's general HPC-oriented guideline: keep heavyweight and GUI-oriented dependencies out of the transitive include path unless they are strictly required.

19.5 Retained Extension Points

The cleanup did *not* freeze the VTK layer. The intended extension points remain:

- adding new explicit scalar filters from tensor data;
- adding new Gauss-cloud fields as constitutive models evolve;
- extending `VTKCellTraits` as new geometries are introduced;
- adding alternate writers as isolated adapters rather than as branches inside the main exporter.

That is the key design rule going forward: extend through small orthogonal adapters, not by re-expanding `VTKModelExporter` into a multipurpose state machine.

Chapter 20

Structural Scale Reconstruction and Multi-Block VTK Export

20.1 Motivation

The continuum exporter described in earlier chapters is not sufficient for structural finite elements. Beams and shells do not store a full three-dimensional stress or strain field at every spatial point. Instead, their formulation *degenerates* the continuum kinematics into a smaller set of generalized measures:

- beam sections carry axial strain, curvature, shear strains, and torsion rate;
- shell sections carry membrane strains, curvatures, and transverse shears.

For the present work, reconstructing the lost three-dimensional field is not a mere visualisation problem. The same machinery will later be used for *scale reconstruction*: transferring a reduced structural solution into an initial condition for a continuum discretisation. For that reason, the reconstruction logic must live in its own compile-time module rather than be buried inside a VTK writer.

20.2 Architectural Split

The implementation introduced in this phase is organised in three layers:

1. **Constitutive snapshot layer**. The type-erased `Material<Policy>` now exports a lightweight `SectionConstitutiveSnapshot`. This is the structural analogue of the previously introduced `InternalFieldSnapshot`. It carries only the constitutive data needed to reconstruct a section: beam stiffness parameters $(E, G, A, I_y, I_z, J, k_y, k_z)$, shell thickness parameters (E, ν, G, t, κ), and, when applicable, the current fiber field.
2. **Reduction / reconstruction layer**. The new `StructuralReductionPolicy<ElementT>` specialisations in `src/reconstruction/StructuralFieldReconstruction.hh` encode the formulation-specific mapping from reduced kinematics to reconstructed fields. These policies are specialised for:
   - `BeamElement<TimoshenkoBeam3D,3>`,
   - `TimoshenkoBeamN<N>`,
   - `ShellElement<MindlinReissnerShell3D>`.
3. **Carrier / export layer**. The new `StructuralVTMExporter` builds a `.vtm` multi-block file from the reconstruction layer. It does not implement beam or shell theory; it only requests samples from the reduction policy and maps them onto VTK carriers.

This split is deliberate. The exporter is allowed to disappear or to be replaced by another post-processing backend; the reconstruction module should remain valid because it expresses the structural theory itself.

20.3 Spatial Roles and Ontology

The reconstruction work reinforced the ontology introduced during the domain refactor:

- `Point` remains pure geometry.
- `Vertex` remains a geometric mesh entity.
- `Node` remains a carrier of degrees of freedom.
- `IntegrationPoint<dim>` is a spatial sampling site in the *embedding* dimension, not in the topological dimension.
- `MaterialPoint` is a continuum constitutive role layered on an `IntegrationPoint`.
- `MaterialSection` is the structural analogue of `MaterialPoint`: a constitutive role layered on an integration site, but carrying generalized beam or shell measures.
- `NodeSection` is a nodal geometric station, not a constitutive point.

The crucial correction in this phase is that `MaterialSection` now binds to `IntegrationPoint<MaterialPol` i.e. to a point in the embedding space. The previous 1D-only design was semantically too narrow even for beams, because the constitutive site of a beam is spatially embedded in $\mathbb{R}^3$.

20.4 Constitutive Snapshots

The new `SectionConstitutiveSnapshot` is returned by a single extra virtual method on the type-erased material wrapper. This follows the same reasoning as the internal-field export: the runtime indirection is paid only when post-processing asks for it, whereas the reconstruction kernels still operate with fully static element and field types.

20.4.1 Beam Snapshot

For elastic Timoshenko sections the snapshot contains:

$$\{E,\ G,\ A,\ I_y,\ I_z,\ J,\ k_y,\ k_z\}.$$

This is sufficient to reconstruct the standard sectional field:

$$\varepsilon_{xx}(y,z) = \varepsilon_0 - z\,\kappa_y + y\,\kappa_z,$$

$$\sigma_{xx}(y,z) = E\,\varepsilon_{xx}(y,z),$$

$$\tau_{xy} = G\,\gamma_y, \qquad \tau_{xz} = G\,\gamma_z.$$

For higher-fidelity torsion reconstruction one would need a section warping model. The present implementation does *not* invent one. Torsion remains available as a generalized measure, and can later be lifted by a specialised profile/warping policy.

20.4.2 Shell Snapshot

For Mindlin–Reissner shells the snapshot contains:

$$\{E,\ \nu,\ G,\ t,\ \kappa\}.$$

Given the mid-surface generalized strains

$$\{\varepsilon_{11}, \varepsilon_{22}, \gamma_{12}, \kappa_{11}, \kappa_{22}, \kappa_{12}, \gamma_{13}, \gamma_{23}\},$$

the through-thickness reconstruction is:

$$\varepsilon_{11}(z) = \varepsilon_{11}^0 + z\,\kappa_{11}, \qquad \varepsilon_{22}(z) = \varepsilon_{22}^0 + z\,\kappa_{22},$$

$$\gamma_{12}(z) = \gamma_{12}^0 + z\,\kappa_{12}, \qquad \gamma_{13}(z) = \gamma_{13}, \qquad \gamma_{23}(z) = \gamma_{23}.$$

For the present linear elastic shell model, the in-plane stresses are obtained with the plane-stress relation:

$$\sigma_{11} = \frac{E}{1-\nu^2}\left(\varepsilon_{11} + \nu\varepsilon_{22}\right), \qquad \sigma_{22} = \frac{E}{1-\nu^2}\left(\nu\varepsilon_{11} + \varepsilon_{22}\right),$$

$$\tau_{12} = G\,\gamma_{12}, \qquad \tau_{13} = G\,\gamma_{13}, \qquad \tau_{23} = G\,\gamma_{23}.$$

20.4.3 Fiber Snapshot

The previous architecture allowed nonlinear fiber sections in the solver, but the layout and the current fiber state were hidden behind type erasure. This phase adds a structural export path that exposes the *current* fiber field through the snapshot:

$$\{y_i,\ z_i,\ A_i,\ \varepsilon_i,\ \sigma_i\}_{i=1}^{n_f}.$$

This is obtained from the current generalized strain stored at the section and the constitutive response of each fiber material:

$$\varepsilon_i = \varepsilon_0 - z_i\kappa_y + y_i\kappa_z, \qquad \sigma_i = \sigma_i(\varepsilon_i).$$

No ad-hoc extrapolation is introduced: the fiber block reflects the actual constitutive state of the section.

20.5 Reduction Policies

20.5.1 Why Policies and Not VTK-Specific Code

Each structural formulation degenerates the continuum in a different way. A Timoshenko beam, an Euler–Bernoulli beam, an MITC shell, and a future geometrically exact beam do not lose the same kinematic information, and therefore they do not admit the same reconstruction. A generic "visual beam" abstraction would hide this distinction and eventually become architecturally wrong.

The chosen design is therefore:

```
StructuralReductionPolicy<ElementT>
```

with one specialisation per formulation. This respects the origin of the abstraction: the reconstruction belongs to the formulation, not to the renderer.

20.5.2 3D Timoshenko Beam

For `BeamElement<TimoshenkoBeam3D,3>` and `TimoshenkoBeamN<N>`, the reduction policy reconstructs:

- the axis displacement,
- the section rotation vector,
- the generalized section strain,
- the generalized section resultants,
- the reconstructed section field at any local section coordinate (y, z).

The displacement of a reconstructed section point is computed with the standard small-rotation lifting:

$$\mathbf{u}(y, z) = \mathbf{u}_c + \boldsymbol{\theta} \times (0, y, z).$$

This is the same kinematic relation that will later be needed for continuum initialisation from a beam model.

20.5.3 MITC4 Mindlin Shell

For `ShellElement<MindlinReissnerShell3D>` the reduction policy reconstructs:

- the mid-surface displacement,
- the local rotation vector,
- the generalized shell strain at an arbitrary parametric point,
- the generalized shell resultants at a material site,
- the through-thickness field at an arbitrary signed thickness position.

The displaced position of a point offset by z from the mid-surface is:

$$\mathbf{u}(z) = \mathbf{u}_m + \boldsymbol{\theta} \times (0, 0, z).$$

Again, this is not a visual trick; it is the actual lifting implied by the Mindlin kinematics used by the element.

20.6 Compile-Time and HPC Considerations

The new module intentionally favours compile-time structure over generic runtime indirection:

- the reconstruction layer is specialised by concrete element type;
- element-local vectors remain fixed-size Eigen objects;
- no new virtual dispatch appears inside the hot reconstruction formulas;
- the only extra virtual call is the constitutive section snapshot, used outside the solver kernel;
- the element DoF index caches were made `mutable`, so the exporter and reconstruction layer can extract element-local states in `const` traversals without breaking value semantics or reintroducing non-const APIs.

This keeps the intended performance hierarchy intact:

1. solver kernels remain governed by the concrete element type and Eigen stack objects;
2. post-processing may pay a small dynamic cost at the boundary to the type-erased material wrapper;
3. VTK-specific costs are isolated in the exporter only.

20.7 Section Profiles

To separate structural theory from rendering carriers, a second layer of abstraction is introduced: `SectionProfile`.

20.7.1 Beam Profiles

For beams, the exporter currently provides:

- `RectangularSectionProfile<SamplesPerEdge>`,
- `CircularSectionProfile<Segments>`.

These profiles do not define a constitutive law. They only provide the local boundary points used to build the visualization carriers. The profile points are generated from `consteval` reference rings and scaled at runtime by the actual section dimensions. This keeps the topology static and the per-element work minimal. The VTK exporter constrains its beam and shell carrier types with dedicated compile-time concepts: `BeamSectionProfileLike` and `ShellThicknessProfileLike`. This keeps the carrier contract explicit and prevents the exporter from turning into an untyped runtime adapter.

20.7.2 Shell Thickness Profile

For shells, the relevant geometric carrier is the thickness direction. The new `ShellThicknessProfile<Samples>` provides compile-time sample locations on the interval $[-\frac{1}{2}, \frac{1}{2}]$. The exporter uses this both for the section block and for any future scale transfer through the thickness.

20.8 The Multi-Block `.vtm` Contract

The exporter writes a ParaView/VTK multiblock file with the following semantic blocks:

1. `axis`: reduced carrier of the structural model. For beams this is the centerline polyline; for shells it is the mid-surface wireframe.
2. `surface`: a display carrier built from the section profile (beams) or the top/bottom shell surfaces.
3. `sections`: section stations. For beams this contains section rings; for shells this contains through-thickness section lines.
4. `material_sites`: actual constitutive sites.
5. `fibers`: actual fibers when the section model provides them.

Every geometric carrier that is meant to deform in ParaView carries a `displacement` vector in `PointData`. Thus the standard `Warp By Vector` filter works directly, without any native VTK quadrature metadata.

20.8.1 Executable Examples

The main executable now writes two structural demonstration outputs alongside the existing continuum comparison:

- `timoshenko_ref_structural.vtm`: the quadratic `TimoshenkoBeamN<3>` reference beam used in the cantilever comparison. The file exposes the beam axis, swept surface, section rings, and constitutive sites through the same reduced state that is used by the solver.
- `shell_cantilever_structural.vtm`: a small MITC4 cantilever shell patch clamped on one edge and loaded by a distributed nodal tip force. This serves as the first end-to-end example of shell reconstruction and ParaView deformation through `Warp By Vector`.

These examples are intentionally modest. Their role is not to replace a full benchmark suite, but to keep the structural reconstruction path wired into the production executable so regressions become visible immediately.

20.8.2 A Note on Empty Trailing Blocks

The VTK XML multiblock writer may elide a trailing empty block when the last dataset has no geometry. This is a writer/reader behaviour, not a loss of semantic information in the implementation. The exporter therefore keeps the logical block layout fixed, but consumers should not rely on a trailing empty block being materialised on disk when, for example, no fiber section exists in the current model.

20.9 Testing Strategy

Two new tests were introduced:

- `test_structural_reconstruction.cpp` verifies the formulation-level reconstruction:
  - a 3D Timoshenko beam reconstruction is checked against the expected axial-plus-bending field;
  - a flat MITC4 shell reconstruction is checked for linear through-thickness variation.
- `test_structural_vtm_export.cpp` verifies the VTK contract: the axis, surface, sections, and material-sites blocks exist and carry the expected arrays for both a beam and a shell model, while the fiber block is emitted only when it has actual content.

The multiblock writer was also given an auxiliary method, `write_with_local_states()`, so that the export contract can be tested without requiring a PETSc/MPI runtime. This turns out to be useful beyond testing: it decouples the structural carriers from a specific solver backend and makes the exporter equally applicable to offline reconstructed states. In the same spirit, the beam surface and section carriers now use the first and last section snapshots at the element ends instead of blindly reusing the first material site on both sides. This is a small but important correction for future non-uniform beam sections.

20.10 Limitations and Future Work

This phase intentionally does not solve every structural post-processing problem. The most relevant open items are:

- Euler–Bernoulli and geometrically exact beam policies still need their own reduction specialisations.

- Torsional warping reconstruction is not yet implemented.
- The beam section block currently uses end and material stations; a future step should expose exact interpolation nodes for arbitrary higher-order line elements.
- For nonlinear shells with layered constitutive laws, a richer shell section snapshot will be required.
- Fiber blocks currently export the actual fiber field but do not yet generate a finite-area carrier; they are emitted as point sites.

These are acceptable limitations because the key architectural property is now in place: *reconstruction belongs to the structural formulation and is available independently of the visualization backend.*

Chapter 21

Mixed Structural Table Example

21.1 Purpose

This chapter documents a compact structural example that exercises the main architectural decisions developed in the preceding chapters:

- a heterogeneous structural model stored through `StructuralElement`,
- formulation-specific reconstruction through `StructuralReductionPolicy<ElementT>`,
- multi-block VTK export through `StructuralVTMExporter`,
- coexistence of beam and shell elements on a shared geometric support.

The example is intentionally simple enough to inspect by hand, but rich enough to validate the polymorphic structural pipeline end to end.

21.2 Physical Model

The structure resembles a table:

- four corner columns,
- four perimeter beams on the top frame,
- one shell slab spanning the frame.

21.2.1 Geometry

The adopted dimensions are:

$$L = 5.0, \qquad H = 3.0, \qquad t_{\text{slab}} = 0.18.$$

The four column bases are placed at:

$$(0,0,0),\ (L,0,0),\ (L,L,0),\ (0,L,0),$$

and the top frame lies in the plane:

$$z = H.$$

The slab is discretised as a structured 4×4 MITC4 mesh over the square domain $[0,L]\times[0,L]$.

21.2.2 Sections and Materials

The example uses two distinct beam formulations and one shell formulation:

- **Columns**: `BeamElement<TimoshenkoBeam3D,3>` with rectangular section 0.40×0.40, Young's modulus $E_c = 21000$, Poisson ratio $\nu = 0.20$.
- **Perimeter beams**: `TimoshenkoBeamN<2>` with rectangular section 0.60×0.40, Young's modulus $E_b = 24000$, Poisson ratio $\nu = 0.20$.
- **Slab**: `ShellElement<MindlinReissnerShell3D>` with thickness $t = 0.18$, Young's modulus $E_s = 26000$, Poisson ratio $\nu = 0.20$.

The shear modulus is reconstructed from isotropic elasticity:

$$G = \frac{E}{2(1+\nu)}.$$

For rectangular beams the section properties are computed directly from the section dimensions:

$$A = bh, \qquad I_y = \frac{bh^3}{12}, \qquad I_z = \frac{hb^3}{12},$$

and the Saint-Venant torsional constant is approximated by

$$J \approx \frac{b_{\min}^3 h_{\max}}{3}\left(1 - 0.63\frac{b_{\min}}{h_{\max}}\right).$$

21.3 Loads and Boundary Conditions

All nodes with $z = 0$ are fully clamped through the structural model interface.

Two load cases are superposed:

1. a uniform slab pressure
$$p_z = -0.12,$$
converted to consistent nodal forces by direct integration of the shell interpolation,
$$\mathbf{f}_I = \int_{\Omega_e} N_I \mathbf{t}\, dA, \qquad \mathbf{t} = (0, 0, p_z)^T;$$
2. a lateral point force applied at the corner node (L, L, H),
$$F_x = 0.20.$$

This combination is deliberately chosen to excite:

- vertical flexure of the slab,
- frame action in the perimeter beams,
- bending plus sway of the full table.

21.4 Architectural Realisation

21.4.1 Why the Domain Contains Only the Slab

The current `Domain` layer is intentionally aligned with a pure `DMPlex` surface/volume topology. The slab therefore lives inside the PETSc mesh, whereas the columns and beams are introduced as external line geometries bound to the same `Node`/`Vertex` storage. This keeps the domain ontologically clean:

- the 2D topological mesh remains a shell/slab mesh,
- the line members remain structural geometries rather than fake 2D entities,
- the mixed structural model is assembled one layer above the domain.

21.4.2 Polymorphic Structural Container

The final element container is

```
std::vector<StructuralElement>,
```

owned by

```
Model<TimoshenkoBeam3D, SmallStrain, 6, SingleElementPolicy<StructuralElement>>.
```

This means that the analysis sees one structural family with a common finite-element interface, while post-processing can still recover the concrete type through the structural type-erasure layer:

- `BeamElement<TimoshenkoBeam3D,3>` for columns,
- `TimoshenkoBeamN<2>` for edge beams,
- `ShellElement<MindlinReissnerShell3D>` for slab cells.

The example therefore validates the intended design point: runtime heterogeneity at the model boundary, formulation-specific compile-time reconstruction inside each policy.

21.5 Quadrature Contract for `TimoshenkoBeamN`

The example uncovered an architectural inconsistency in `TimoshenkoBeamN<N>`. The element uses a reduced constitutive integration with

$$n_{\mathrm{gp}} = N - 1,$$

but its public topology query originally forwarded `geometry_->num_integration_points()`, which depends on the underlying geometry integrator. For `TimoshenkoBeamN<2>` this created an inconsistent state:

- the element stored one constitutive section,
- the exporter believed that the element had two Gauss points when the geometry used `GaussLegendreCellIntegrator<2>`.

The fix adopted here is structural, not cosmetic:

1. `TimoshenkoBeamN<N>::num_integration_points()` now returns the reduced value $N-1$.

2. The constructor now enforces the quadrature contract: the attached geometry must also expose exactly $N - 1$ integration points.
3. The table example therefore assigns `GaussLegendreCellIntegrator<1>` to the perimeter beams and `GaussLegendreCellIntegrator<2>` to the columns.

This is important beyond visualisation. The reconstruction layer and any future scale-transfer operator must reason about the actual constitutive sites of the formulation, not about an unrelated geometric quadrature.

21.6 VTK Output

The structural exporter writes a `.vtm` index together with one `.vtp` sidecar per semantic block:

- `table_frame_structural.vtm`,
- `table_frame_structural_axis.vtp`,
- `table_frame_structural_surface.vtp`,
- `table_frame_structural_sections.vtp`,
- `table_frame_structural_material_sites.vtp`,
- `table_frame_structural_fibers.vtp`.

These blocks have a fixed semantic meaning:

`axis` beam centre-lines and shell boundary carriers;

`surface` reconstructed beam skins and shell top/bottom faces;

`sections` section stations and shell through-thickness lines;

`material_sites` constitutive sampling sites;

`fibers` fiber carriers when a fiber section is present.

The split into `.vtp` sidecars is intentional. It avoids the fragile writer path observed with the monolithic multi-block writer while preserving a semantically clean `.vtm` interface for ParaView.

21.7 Observed Structural Response

For the current parameter set, the example run stored in the repository reported:

$$\max |u| = 1.3651 \times 10^{-2}, \qquad \max |u_x| = 5.30 \times 10^{-4}, \qquad \max |u_z| = 1.3651 \times 10^{-2}.$$

These values are not presented as a benchmark target. Their role here is to confirm that:

- the mixed structural solve completes,
- the reconstructed carriers can be exported without crashing,
- the example exercises both vertical slab deformation and lateral frame sway.

21.8 Code Path

The complete example is implemented in `main.cpp` as `run_structural_table_example()`. The relevant pieces are:

- geometric construction of slab, columns, and beams;
- consistent shell pressure assembly through `apply_uniform_shell_surface_load()`;
- polymorphic structural model assembly through `StructuralElement`;
- multi-block structural export through `export_structural_vtm()`.

This example now serves as the reference mixed structural demonstration for the current architecture.

Chapter 22

Robustifying `DMPlex` for Naturally Mixed Element Families

22.1 Motivation

Up to this point, the `Domain`–`Model` pipeline relied on a very restrictive interpretation of `DMPlex`: an uninterpolated cell–vertex graph where every analysis element was treated as a "cell" and the code assumed that querying height 1 produced the vertices. That assumption is brittle even for standard meshes and becomes conceptually wrong as soon as one wants to mix, in the same discrete domain, elements of different topological dimension such as:

- beams (1D topology embedded in $\mathbb{R}^3$),
- shells (2D topology embedded in $\mathbb{R}^3$),
- continua (3D topology embedded in $\mathbb{R}^3$).

The immediate objective of this refactor is therefore modest but essential: *keep the current non-interpolated DAG for performance and simplicity, while making the `DMPlex` object semantically correct enough to host mixed element families naturally.*

22.2 Design Decision

The adopted strategy is intentionally incremental:

1. Keep the DAG non-interpolated. Cells still connect directly to vertices; explicit edges and faces are not materialised yet.
2. Encode the missing topological semantics using `DMLabel`s.
3. Drive the algebraic layout from the only invariant that really matters for the current solver: analysis unknowns live on depth-0 vertices.

This preserves the low-cost setup path and avoids polluting the hot assembly loops with new runtime indirections. The extra work happens only once during mesh/DMPlex setup.

22.3 Implemented Semantics

Three labels are now attached to the `DMPlex` points generated by `Domain::assemble_sieve()`:

`fall_n.point_role` distinguishes between vertex points and cell points. The current values are:

- 0 for vertices,
- 1 for cells.

`fall_n.topological_dim` stores the intrinsic topological dimension of each point. Vertices receive value 0, while cells receive the topological dimension reported by `ElementGeometry::topological_d`

`fall_n.physical_group_id` stores an integer identifier for the physical group carried by a geometric element, when such group exists.

These labels are cheap, explicit, and stable. More importantly, they separate two different concerns that had previously been conflated:

- the *graph* structure of the mesh;
- the *topological meaning* of each graph point.

22.4 Why `Depth` and Not `Height`

The previous implementation used:

```
DMPlexGetHeightStratum(dm, 1, &vStart, &vEnd);
```

and interpreted that stratum as the vertices. This is only safe for a very specific cell–vertex graph and becomes false once the mesh is interpolated or its topology is enriched.

The robust statement is:

> analysis vertices are the depth-0 points of the `DMPlex`.

Accordingly, `Model::set_sieve_layout()` was rewritten to define the section over the full chart while conceptually placing DOFs only on depth-0 points. Cell points keep zero DOFs.

22.5 Dimension of the `DMPlex`

Another correction concerns the dimension assigned to the `DM`. The old code forced:

```
DMSetDimension(dm, dim);
```

where `dim` is the embedding space dimension of the model, not the intrinsic dimension of the discrete topology.

This is now replaced by:

$$\dim(\texttt{DM}) = \max_{e \in \mathcal{T}_h} \operatorname{topodim}(e), \tag{22.1}$$

that is, the maximum topological dimension among the stored elements. For a mixed beam–shell model embedded in $\mathbb{R}^3$, the resulting `DMPlex` has dimension 2; for a mixed beam–shell–solid model, it has dimension 3.

22.6 Compile-Time and HPC Considerations

This refactor was designed to preserve the original performance intent:

- Geometry, interpolation, and quadrature remain statically typed.
- No new virtual dispatch was introduced inside numerical integration, constitutive update, or element assembly loops.

- The new metadata is written once during `assemble_sieve()` and then consumed through PETSc's own label machinery.
- The DAG itself is still cell–vertex, which keeps setup lighter than a fully interpolated mesh and reduces memory pressure.

In other words, the refactor improves semantic correctness of the mesh layer without paying for it in the hot path.

22.7 Validation

A dedicated regression test was added:

`tests/test_dmplex_mixed_topology.cpp`

It builds a small domain containing:

- one 1D beam geometry,
- one 2D shell geometry,
- shared depth-0 vertices.

The test verifies:

1. the `DM` dimension is the maximum topological dimension;
2. height-0 contains the two cells and depth-0 the four vertices;
3. the three labels above are present and correctly populated;
4. a polymorphic structural `Model` assigns DOFs only to vertices, leaving cell points with zero DOFs.

22.8 Current Scope and Next Step

This chapter documents a necessary intermediate stage, not the final one. Mixed families now coexist naturally in the same `DMPlex` at the semantic level, but the mesh is still not fully interpolated. Therefore:

- the solver can reason correctly about vertices versus cells;
- element families of different intrinsic dimension can share the same `Domain` and `DMPlex`;
- future work may still introduce explicit edges/faces when the library needs richer topological traversal or native mixed-dimensional coupling operators.

This staging is intentional. It respects the architectural principle adopted throughout `fall_n`: only introduce abstractions that correspond to a real numerical need, and keep the compile-time structure of the kernels as static as possible.

Chapter 23

Regular 5-Story Building Example

23.1 Purpose

This chapter documents the *elastic baseline* mixed-building example that preceded the materially nonlinear reinforced-concrete continuation developed in Chapter 24. It remains useful as the simplest reference configuration for understanding the shared `Domain`, mixed 1D/2D topology, and polymorphic structural wrapping without the additional complexity of path-dependent constitutive response.

The example is a regular reinforced-concrete-like building with:

- five stories,
- four grid axes in the x-direction,
- three grid axes in the y-direction,
- frame members modelled with 3D Timoshenko beams,
- slabs modelled with MITC4 / Mindlin–Reissner shells.

The objective is not a code-check design, but a pedagogical and architectural integration example:

- shared geometry and topology in one `Domain`,
- mixed 1D/2D structural families,
- one polymorphic structural solver container,
- one structural `.vtm` export for ParaView.

23.2 Unit System

The example is written consistently in:

$$\text{force} = \text{kN}, \qquad \text{length} = \text{m}, \qquad \text{stress} = \text{kN/m}^2.$$

Therefore:

- Young's moduli are entered in kN/m^2,
- slab loads are entered in kN/m^2,
- nodal lateral forces are entered in kN,
- computed displacements are reported in m.

23.3 Geometry

23.3.1 Plan and height

The plan grid is:

$$x = \{0,\ 6,\ 12,\ 18\}\ \text{m}, \qquad y = \{0,\ 5,\ 10\}\ \text{m}.$$

This gives:

- 3 bays of 6 m in the x-direction,
- 2 bays of 5 m in the y-direction.

The storey height is:

$$h = 3.20\ \text{m},$$

so the floor elevations are:

$$z_k = 3.20\,k, \qquad k = 0, \ldots, 5.$$

The total structural height is:

$$H = 16.0\ \text{m}.$$

23.3.2 Discretisation

All members share the same building nodes:

- $4 \times 3 = 12$ plan nodes per level,
- 6 levels including the base,
- total node count:

$$n_{\text{nodes}} = 12 \times 6 = 72.$$

The element counts are:

- columns:

$$n_c = 5 \times 4 \times 3 = 60,$$

- beams:

$$n_b = 5\,[(4-1) \times 3 + (3-1) \times 4] = 85,$$

- slab panels:

$$n_s = 5 \times (4-1) \times (3-1) = 30.$$

Hence the total number of structural geometric elements is:

$$n_{\text{elem}} = 60 + 85 + 30 = 175.$$

23.4 Sections and Materials

23.4.1 Columns

Columns are modelled as 3D Timoshenko beams with rectangular section:

$$b_c = 0.45\ \text{m}, \qquad h_c = 0.45\ \text{m}.$$

The frame material is idealised as linear elastic with:

$$E_f = 28.0 \times 10^6\ \text{kN/m}^2, \qquad \nu = 0.20.$$

The shear modulus is reconstructed as:

$$G_f = \frac{E_f}{2(1+\nu)}.$$

23.4.2 Beams

Beams are also modelled as 3D Timoshenko beams, with section:

$$b_b = 0.30 \text{ m}, \qquad h_b = 0.60 \text{ m}.$$

The same frame elastic modulus is used:

$$E_b = E_f = 28.0 \times 10^6 \text{ kN/m}^2.$$

For both columns and beams the section properties are computed directly from the rectangular dimensions:

$$A = bh, \qquad I_y = \frac{bh^3}{12}, \qquad I_z = \frac{hb^3}{12}.$$

The torsional constant is approximated with the standard rectangular Saint-Venant estimate:

$$J \approx \frac{b_{\min}^3 h_{\max}}{3}\left(1 - 0.63\frac{b_{\min}}{h_{\max}}\right).$$

The Timoshenko shear factors are taken as:

$$\kappa_y = \kappa_z = \frac{5}{6}.$$

23.4.3 Slabs

Each floor slab panel is modelled by a MITC4 / Mindlin–Reissner shell with thickness:

$$t = 0.15 \text{ m}.$$

The slab elastic modulus is slightly reduced with respect to the frame:

$$E_s = 25.0 \times 10^6 \text{ kN/m}^2, \qquad \nu_s = 0.20.$$

23.5 Loads and Boundary Conditions

23.5.1 Boundary conditions

All base nodes, that is all nodes on the plane $z = 0$, are fully fixed. This is implemented at the model level through:

```
model.fix_z(0.0);
```

which, in the current API, constrains all DOFs of every node lying on that plane.

23.5.2 Gravity loading

Each slab receives a uniform downward pressure:

$$q_g = -7.50 \text{ kN/m}^2.$$

This pressure represents a simplified vertical service load (self-weight + superimposed permanent load + a modest live-load component).

The load is assembled consistently from the shell interpolation:

$$\mathbf{f}_I = \int_{\Omega_e} N_I \, \mathbf{t} \, dA, \qquad \mathbf{t} = (0, 0, q_g)^T.$$

23.5.3 Horizontal loading

To make frame bending visually more evident, one corner node of every floor receives a horizontal force in the global x-direction. The loaded node at storey k is:

$$(x, y, z) = (18,\ 10,\ z_k).$$

The storey force pattern is triangular:

$$F_x^{(k)} = 400\,k \quad \text{kN}, \qquad k = 1, \ldots, 5.$$

Thus the applied forces are:

$$400,\ 800,\ 1200,\ 1600,\ 2000 \text{ kN}.$$

This is intentionally not a perfectly symmetric wind model. The asymmetry is useful here because it activates:

- global building sway,
- perimeter beam bending,
- slab diaphragm action,
- a modest torsional component.

23.6 Architectural Realisation in Code

23.6.1 One shared `Domain`

The first important architectural point is that the example inserts *all* structural geometry into the same `Domain<3>`:

- columns are stored as 1D line geometries,
- beams are stored as 1D line geometries,
- slabs are stored as 2D quadrilateral geometries.

This is exactly the kind of mixed topology that motivated the recent `DMPlex` hardening described in Chapter 22. The resulting `DMPlex` remains non-interpolated, but it is now semantically correct for mixed 1D/2D structural families.

For this model the reported `DMPlex` dimension is:

$$\dim(\texttt{DM}) = 2,$$

because the maximum topological dimension in the domain is the slab topology.

23.6.2 Polymorphic structural container

The solver model is assembled as:

```
Model<TimoshenkoBeam3D, SmallStrain, 6, SingleElementPolicy<StructuralElement>>.
```

This means:

- the element container is

```
std::vector<StructuralElement>,
```

- each concrete geometry is wrapped into the appropriate concrete element type:
  - `BeamElement<TimoshenkoBeam3D,3>` for columns,
  - `BeamElement<TimoshenkoBeam3D,3>` for beams,
  - `ShellElement<MindlinReissnerShell3D>` for slabs;
- the global assembly sees a homogeneous interface, while the structural VTK exporter can still dispatch back to the concrete formulation through `StructuralReductionPolicy<ElementT>`.

23.6.3 Why this example is a good architectural integrator

This one example exercises, simultaneously:

- geometric ownership in `Domain`,
- mixed-topology `DMPlex` assembly,
- analysis-node DOF assignment on shared vertices,
- heterogeneous structural element wrapping,
- shell consistent-load assembly,
- formulation-aware structural reconstruction,
- multi-block VTK output.

In that sense it is a better "main example" than the previous collection of independent benchmarks: it shows how the pieces work together.

23.7 Execution Results

The executed example stored in the repository reported:

$$\max |u| = 9.248 \times 10^{-3} \text{ m},$$

$$\max |u_x| = 8.292 \times 10^{-3} \text{ m}, \qquad \max |u_z| = 2.523 \times 10^{-3} \text{ m}.$$

At the loaded roof corner node, the displacement vector was:

$$u_{\text{roof}} = \begin{bmatrix} 8.292 \times 10^{-3} \\ -3.598 \times 10^{-3} \\ -1.955 \times 10^{-3} \end{bmatrix} \text{m}.$$

The roof drift ratio in the x-direction is therefore:

$$\frac{u_x^{\text{roof}}}{H} = \frac{8.292 \times 10^{-3}}{16.0} = 5.18 \times 10^{-4}.$$

These values are small enough to remain in the linear range, but large enough to make the deformed shape and beam flexure visible in ParaView when a moderate `Warp By Vector` scale factor is used.

23.8 VTK Output

The example writes the following VTK files:

- `regular_5story_building_structural.vtm`,
- `regular_5story_building_structural_axis.vtp`,
- `regular_5story_building_structural_surface.vtp`,
- `regular_5story_building_structural_sections.vtp`,
- `regular_5story_building_structural_material_sites.vtp`,
- `regular_5story_building_structural_fibers.vtp`.

The semantic meaning of the blocks remains the same as in the previous structural exporter chapters:

`axis` line carriers for beam centre-lines and shell edges;

`surface` reconstructed beam skins and shell faces;

`sections` section stations and through-thickness carriers;

`material_sites` constitutive sampling sites;

`fibers` fiber-section carriers (empty in this example).

In ParaView, the intended workflow is:

1. open the `.vtm` file,
2. display the `surface` block,
3. apply `Warp By Vector` using `displacement`,
4. colour by reconstructed resultants or stresses,
5. inspect `sections` and `material_sites` for the formulation-specific internal fields.

23.9 Why the Example Matters

From a software-architecture perspective, this example confirms that the current direction is viable:

- mixed structural families can share one geometric and topological domain,
- `StructuralElement` is sufficient as the polymorphic solver boundary,
- reconstruction remains formulation-specific and therefore reusable for later scale-transfer work,
- the new `main.cpp` now acts as a readable cookbook for building a realistic mixed frame–shell model.

That is exactly the kind of example needed before moving on to the next layer of abstraction, namely a dedicated `DMPlex` policy or builder strategy.

Chapter 24

Nonlinear Reinforced-Concrete 5-Story Building Example

24.1 Purpose and Scope

This chapter documents the current `main.cpp` entry point, which now hosts a materially nonlinear reinforced-concrete building example. The model is the direct continuation of the elastic mixed-building benchmark of Chapter 23, but the frame members are now represented by nonlinear fiber sections while the floor slabs remain elastic MITC4 shells.

The example is intended to validate three architectural goals simultaneously:

- one shared `Domain<3>` and one shared `DMPlex` for columns, beams, and slabs;
- one polymorphic structural element container, mixing beam and shell formulations in the same global solve;
- one constitutive upgrade path in which the frame material law is replaced by a nonlinear fiber discretisation without changing the geometric or topological assembly pipeline.

Equally important, the example is *not* yet a fully nonlinear RC building solver in the broadest sense. The current scope is:

- small-strain structural kinematics;
- nonlinear beam sections through fibers;
- elastic shell sections for slabs;
- static incremental equilibrium with PETSc SNES.

24.2 Unit System

The example switches deliberately to:

$$\text{force} = \text{MN}, \qquad \text{length} = \text{m}, \qquad \text{stress} = \text{MPa} = \text{MN/m}^2.$$

This choice is not cosmetic. The uniaxial constitutive laws currently used in the fiber sections, especially Kent–Park concrete, are calibrated and written numerically in MPa. Using MN–m–MPa preserves unit consistency while avoiding ad hoc rescalings inside the constitutive model. In this system:

- 30 MPa is entered numerically as 30,
- 200 kN is entered as 0.20 MN,
- 7.5 kN/m^2 is entered as 0.0075 MN/m^2.

24.3 Geometry and Topology

24.3.1 Plan and elevations

The plan grid is unchanged from the elastic reference:

$$x = \{0,\ 6,\ 12,\ 18\}\ \text{m}, \qquad y = \{0,\ 5,\ 10\}\ \text{m},$$

with storey height

$$h = 3.20\ \text{m}, \qquad z_k = 3.20\,k, \quad k = 0, \ldots, 5.$$

This yields:

- 72 shared building nodes,
- 60 columns,
- 85 beams,
- 30 shell panels,
- 175 geometric elements in total.

24.3.2 Mixed structural topology

All structural members are inserted into the same `Domain<3>`:

- columns and beams as 1D line geometries,
- slabs as 2D quadrilateral geometries.

The resulting `DMPlex` remains non-interpolated, but its highest topological dimension is now 2 because shell cells are present. The recent `DMPlex` hardening from Chapter 22 is what makes this mixed 1D/2D configuration natural rather than exceptional.

24.4 Constitutive Design

24.4.1 Columns

Columns are modelled as 3D Timoshenko beam elements with section:

$$b_c = h_c = 0.45\ \text{m}.$$

The nonlinear section is built from:

- unconfined cover concrete, modelled with Kent–Park;
- confined core concrete, also modelled with Kent–Park but with a confinement enhancement factor derived from an assumed volumetric tie ratio;
- eight longitudinal reinforcing bars of diameter $\phi = 25$ mm, modelled with Menegotto–Pinto steel.

The material parameters are:

$$f'_c = 35\ \text{MPa}, \qquad E_c = 1000\,f'_c = 35000\ \text{MPa}, \qquad \nu = 0.20,$$

$$E_s = 200000\ \text{MPa}, \qquad f_y = 420\ \text{MPa}, \qquad b_s = 0.01.$$

The cover thickness to fiber centroid is taken as:

$$c_c = 0.05\ \text{m}.$$

The column section is discretised into:

- 84 concrete fibers,
- 8 steel fibers,
- total: 92 fibers per column integration point.

24.4.2 Beams

Beams are also 3D Timoshenko elements, with section:

$$b_b = 0.30 \text{ m}, \qquad h_b = 0.60 \text{ m}.$$

The beam fiber section uses:

- unconfined Kent–Park concrete over the full section;
- six longitudinal reinforcing bars of diameter $\phi = 20$ mm, arranged as three top and three bottom bars;
- the same Menegotto–Pinto steel law as in the columns.

The beam concrete parameters are:

$$f'_c = 30 \text{ MPa}, \qquad E_c = 30000 \text{ MPa}.$$

The beam section discretisation contains:

- 72 concrete fibers,
- 6 steel fibers,
- total: 78 fibers per beam integration point.

24.4.3 Slabs

The slabs remain elastic MITC4 / Mindlin–Reissner shells with thickness:

$$t = 0.15 \text{ m}.$$

The elastic shell properties are:

$$E_s = 28000 \text{ MPa}, \qquad \nu_s = 0.20.$$

This is a deliberate modelling boundary. The current example is a nonlinear *frame* with an elastic slab diaphragm, not yet a fully fiberised RC shell–frame interaction problem.

24.5 Why Fibers Belong in the Constitutive Layer

The crucial architectural decision is that the fiber discretisation is not a visualisation gadget. It is the constitutive relation itself:

`FiberSection3D` → `Material<TimoshenkoBeam3D>` → `BeamElement`.

That means the same reduction policy that maps generalized beam strain $(\varepsilon_0, \kappa_y, \kappa_z, \gamma_y, \gamma_z, \theta')$ to section response is already the one that will later be reusable for:

- post-processing,
- section-force recovery,
- multiscale or scale-bridging initialisation,
- transfer of structural states to continuum models.

The visualisation layer only reads committed constitutive snapshots; it does not define the physics.

24.6 Section Kinematics and Fiber Stress Recovery

For each beam material point, the generalized strain vector is:

$$\mathbf{e} = \begin{bmatrix} \varepsilon_0 & \kappa_y & \kappa_z & \gamma_y & \gamma_z & \theta' \end{bmatrix}^T.$$

The axial strain at a fiber located at local section coordinates (y, z) is reconstructed as:

$$\varepsilon_f(y, z) = \varepsilon_0 - z\,\kappa_y + y\,\kappa_z.$$

The section resultants then follow from numerical integration over the fibers:

$$N = \sum_i \sigma_i A_i, \qquad M_y = \sum_i \sigma_i(-z_i)A_i, \qquad M_z = \sum_i \sigma_i y_i A_i.$$

The shear and torsional terms remain elastic:

$$V_y = \kappa_y GA\,\gamma_y, \qquad V_z = \kappa_z GA\,\gamma_z, \qquad T = GJ\,\theta'.$$

This is why the current example is materially nonlinear but still efficient: only the axial/flexural part of the section is fiber-integrated, while the shear/torsion part remains compact and closed form.

24.7 Global Solution Strategy

The mixed model is solved with `NonlinearAnalysis<..., StructuralPolicy>`, i.e. PETSc SNES on the same polymorphic structural container already used by the linear example. The load vector is applied incrementally:

$$\lambda_k = \frac{k}{12}, \qquad k = 1, \ldots, 12,$$

with automatic step bisection enabled up to depth 6, although the final run converged without exhausting that budget.

The PETSc options used in the executable are:

- `-snes_type newtonls`,
- `-snes_linesearch_type bt`,
- `-snes_rtol 1e-8`,
- `-snes_atol 1e-9`,
- `-snes_max_it 40`,
- `-ksp_type preonly`,
- `-pc_type lu`.

24.7.1 Analysis-layer hardening

In order for this example to compile and solve through the structural type-erased path, `NLAnalysis` was generalised. The solver now uses two assembly routes:

- a fast local-vector route for continuum elements exposing explicit element DOF extraction and local tangent routines;
- a generic `FiniteElement` route for structural type-erased elements, assembling through `compute_internal_forces()` and `inject_tangent_stiffness()`.

This keeps the continuum hot path intact while making the nonlinear solver architecturally consistent with the structural abstraction already present in the codebase.

24.8 Loads and Boundary Conditions

The support conditions are unchanged from the baseline model: all nodes on $z = 0$ are fully fixed.

The slabs carry a uniform gravity pressure:

$$q_g = -0.0075 \text{ MN/m}^2.$$

The lateral load pattern is triangular and applied at the corner $(18, 10, z_k)$:

$$F_x^{(k)} = 0.20\,k \quad \text{MN}, \qquad k = 1, \ldots, 5.$$

This choice keeps the model response moderate while making sway and beam flexure clearly visible.

24.9 Results of the Executed Example

The example was executed successfully in the current workspace. The final incremental solve converged at $\lambda = 1.0$ with 4 SNES iterations in the last step.

The reported response values are:

$$\max \|\mathbf{u}\| = 0.003767 \text{ m}, \qquad \max |u_x| = 0.003314 \text{ m}, \qquad \max |u_z| = 0.001244 \text{ m},$$

and at the roof corner:

$$u_x = 0.003314 \text{ m}, \qquad u_y = -0.001473 \text{ m}, \qquad u_z = -0.001019 \text{ m}.$$

The roof drift ratio is:

$$\frac{u_x^{\text{roof}}}{H} = 2.07 \times 10^{-4}.$$

This is intentionally a mild nonlinear response, not a collapse or pushover limit-state study. The objective here is robustness and architectural traceability, not maximum inelastic excursion.

24.10 VTK Output

The example writes:

`regular_5story_rc_nonlinear_building_structural.vtm`

plus the corresponding block sidecars:

- `_axis.vtp`
- `_surface.vtp`
- `_sections.vtp`
- `_material_sites.vtp`
- `_fibers.vtp`

For nonlinear RC interpretation, the most important block is:

- `fibers`: it contains the actual fiber-level `fiber_strain_xx` and `fiber_stress_xx` values reconstructed from the committed constitutive state.

The surface block remains useful for displacement visualisation and geometric context, but the truly nonlinear stress information lives at the fiber level.

24.11 Regression and Verification

Alongside the full building example, a dedicated regression test was added:

`tests/test_rc_fiber_frame_nonlinear.cpp`

It solves a small RC fiber cantilever with the same `BeamElement + FiberSection + NonlinearAnalysis` stack and checks:

- convergence of the incremental SNES solve,
- finite tip displacement,
- physically correct sign of the vertical response.

During that work a real lifecycle bug was found and fixed in the test harness: PETSc-owning objects must be destroyed before `PetscFinalize()`.

24.12 Current Limits and Next Steps

The example is already useful as a production-quality integration case, but some limits remain explicit:

- the slab is still elastic;
- the structural kinematics are still small-strain;
- beam surface stresses in the generic surface carrier are only a kinematic proxy when no closed-form beam snapshot exists, whereas the fiber block contains the actual nonlinear stress information;
- confined/unconfined concrete zoning is still schematic rather than design-code calibrated.

The next natural architectural step is not in visualisation, but in reconstruction: extend the same reduction-policy logic toward consistent scale-bridging and, eventually, transfer of committed structural states to a continuum discretisation.

Chapter 25

Material Module Architecture Analysis

25.1 Purpose

This chapter provides a focused architectural analysis of the current `materials` module. The goal is not merely to rename classes or to standardise terminology, but to identify the semantic boundaries that must be made explicit if `fall_n` is to evolve toward:

- reinforced concrete modelling at several scales;
- materially nonlinear and dynamic analysis;
- finite-kinematics structural and continuum formulations;
- continuum-scale cracking, damage, localisation, and fracture.

The analysis is guided by three constraints already visible across the code base:

- the local element kernels must remain inexpensive and friendly to compile-time optimisation;
- the library must preserve the current genericity across continuum, beam, and shell formulations;
- the constitutive layer must remain reusable beyond post-processing, because the same reconstruction policies will later be needed for scale transfer and multiscale initialisation.

25.2 Current Architectural Layers

25.2.1 Measure spaces

At the lowest level, the module already contains a useful abstraction:

- `Strain<N>` and `Stress<N>` for continuum Voigt spaces;
- `BeamGeneralizedStrain<N,Dim>` and `BeamSectionForces<N>`;
- `ShellGeneralizedStrain<N,Dim>` and `ShellResultants<N>`.

The concepts in `ConstitutiveRelation.hh` correctly recognise that a constitutive relation should fundamentally operate on a pair

$$(\text{kinematic measure},\ \text{tensional conjugate}),$$

independently of whether that pair belongs to a solid, a beam section, or a shell section. This is one of the strongest parts of the current design and should be preserved.

25.2.2 Relation classes

The code currently places under the same umbrella:

- continuum elastic relations such as `ContinuumIsotropicRelation`;
- reduced section relations such as `TimoshenkoBeamSection3D` and `MindlinShellSection`;
- inelastic local models such as `PlasticityRelation`, `KentParkConcrete`, and `MenegottoPintoSteel`;
- reduction-based section models such as `FiberSection3D`.

All of them satisfy the same compile-time concept:

$$\texttt{compute_response}(k), \qquad \texttt{tangent}(k),$$

with optional inelastic extensions through

$$\texttt{update}(k), \qquad \texttt{internal_state}().$$

This common contract is practical, but it already reveals a semantic overloading: some of these types are truly *material laws*, while others are *section laws* or even *constitutive reductions*.

25.2.3 Material instance and type erasure

The next layer is `MaterialInstance<Relation,StatePolicy>`, which wraps:

- a constitutive relation object;
- a per-point kinematic state container;
- access to the underlying relation through `constitutive_relation()`.

This is then lifted to the type-erased `Material<Policy>` wrapper, which stores:

- the concrete `MaterialInstance`;
- an update strategy (`ElasticUpdate` or `InelasticUpdate`);
- export helpers such as `internal_field_snapshot()` and `section_snapshot()`.

25.2.4 Constitutive sites

Finally, the constitutive object is attached to geometry through:

- `MaterialPoint<Policy>` in continuum elements;
- `MaterialSection<Policy>` in beams and shells.

This layering already captures a valuable idea:

geometry and topology are not constitutive objects;
a constitutive object is bound to a sampling site.

25.3 What is Good and Should Be Preserved

25.3.1 The measure-based generic contract

The most important success of the current module is that the constitutive interface is not hard-wired to Cauchy stress or to infinitesimal continuum strain. Instead, it works on generic conjugate pairs. This is exactly what a library needs if it aims to support:

- continuum solids;
- structural sections;
- future higher-order or enriched formulations;
- multiscale reductions and reconstructions.

25.3.2 Static polymorphism at the relation level

Concrete constitutive classes are template-friendly and concept-constrained. This gives:

- strong compile-time checking;
- direct inlining opportunities inside element kernels;
- no virtual dispatch when the full type is known.

This direction is correct and should remain the default in the hot path.

25.3.3 Separation between geometry and constitutive sites

The distinction between `IntegrationPoint` and `MaterialPoint` / `MaterialSection` is also correct in spirit. It is consistent with the earlier ontology work in the `Domain` refactor:

- the integration point is a numerical sampling site;
- the constitutive point is a physical role attached to that site.

25.3.4 The reduction-based path for structural elements

The existence of `FiberSection3D` is strategically important. It proves that the same element pipeline can host:

- a linear reduced section law;
- a nonlinear reduced section law reconstructed from uniaxial constituents.

This is exactly the kind of reduction/reconstruction mechanism that will later be needed for layered shells, smeared reinforcement, and scale-bridging between frame and continuum descriptions.

25.4 Observed Semantic Inconsistencies

25.4.1 The word `Material` currently means too many things

Today the code uses the word *material* for at least four different concepts:

1. a physical substance or phase (concrete, steel, confined concrete);
2. a constitutive law (`KentParkConcrete`, `MenegottoPintoSteel`);

3. a reduced section law (`TimoshenkoBeamSection3D`, `FiberSection3D`);
4. a type-erased operational wrapper (`Material<Policy>`).

This is the single most important semantic problem in the current module. Architecturally, these should not be collapsed into one noun.

25.4.2 `MaterialPolicy` is not a material

`SolidMaterial<6>`, `BeamMaterial<6,3>`, and `ShellMaterial<8,3>` do not describe a physical material. They define a *constitutive space*:

$$\mathcal{S} = (\text{kinematic type},\ \text{conjugate type},\ \text{dimension}).$$

So the name `MaterialPolicy` is semantically misleading. The type is really a *measure-space policy* or a *constitutive space descriptor*. The current name propagates confusion upward into `Material`, `MaterialPoint`, and `MaterialSection`.

25.4.3 `MaterialInstance` is not just a material instance

`MaterialInstance<Relation,StatePolicy>` currently bundles:

- a constitutive relation;
- kinematic state storage;
- direct access to the relation;
- optional internal-state access for inelastic models.

Semantically, this object is much closer to a *stateful constitutive point model* than to a material instance in the physical sense. In other words, it already lives at the Gauss-point level, not at the material-definition level.

25.4.4 Inelastic relations already contain the integration algorithm

The intended design suggests a clean split:

$$\text{constitutive law} \quad + \quad \text{integration strategy}.$$

However, for nonlinear materials this separation is only nominal at present. The current `InelasticUpdate` strategy simply delegates back to the material relation, while the actual algorithm lives inside:

- `PlasticityRelation`;
- `KentParkConcrete`;
- `MenegottoPintoSteel`;
- `FiberSection`.

Therefore the current strategy layer is semantically strong for elastic materials, but weak for inelastic ones. The empty `FullImplicitBackwardEuler.hh` placeholder makes this explicit.

25.4.5 The constitutive relation and its committed state are fused

For inelastic models, the relation object itself stores internal variables. This means that:

- the constitutive law is not purely a law;
- the law is not shareable as an immutable flyweight;
- the same class simultaneously represents parameters, algorithmic cache, and committed history.

This fusion is convenient for a first implementation, but it becomes a real limitation for:

- thread safety;
- state-block vectorisation;
- fine-grained checkpointing;
- large-scale nonlinear dynamics;
- constitutive substepping and rollback.

25.4.6 The kinematic state is duplicated and over-recorded

`MaterialState` stores the current kinematic measure, and for inelastic materials `MemoryState` stores the *entire history* of that measure. At the same time, the inelastic relations store their own internal variables.

This produces two problems:

- semantic duplication: committed constitutive history already exists in the relation state;
- memory amplification: the default inelastic path records every past strain state whether or not the algorithm requires it.

For nonlinear dynamics or long static load histories this is not sustainable. The default site state should be:

- current trial state,
- committed state,
- optionally converged last-step state,

not an ever-growing vector of all past strains.

25.4.7 The flyweight story is currently contradictory

The comments in `MaterialInstance` describe the relation as a shared flyweight. However, the copy constructor performs a deep copy of the relation. Because elements and Gauss-point carriers are usually created by copying a `Material<Policy>`, the practical behaviour is:

- elastic relations are often duplicated per point;
- inelastic relations are also duplicated per point;
- the claimed flyweight benefit is therefore much weaker than it first appears.

This is not just a micro-optimisation issue. It means the current naming and commentary no longer match the real ownership semantics.

25.4.8 Section laws are currently mixed with material laws

`TimoshenkoBeamSection3D`, `MindlinShellSection`, and `FiberSection3D` live inside the materials module and satisfy the same top-level concepts as bulk material laws. That is convenient, but it masks an important distinction:

- a bulk constitutive law maps local continuum measures to continuum stresses;
- a section constitutive law maps reduced section measures to section resultants;
- a fiber section is in fact a *reduction policy plus numerical integration over constituents.*

This distinction matters if the same section reconstruction is later reused for scale transfer to a continuum model.

25.4.9 Post-processing adapters are embedded in the core wrapper

`Material<Policy>` currently knows how to export:

- `InternalFieldSnapshot`;
- `SectionConstitutiveSnapshot`;
- fiber field snapshots.

This makes the wrapper convenient for VTK export, but it also means the central constitutive wrapper now depends on post-processing concerns. The core constitutive path should not grow indefinitely through `if constexpr`-based export detection.

25.4.10 Finite-strain constitutive models live outside the material module

Hyperelastic relations currently live in `src/continuum`. This is architecturally understandable, because finite-strain kinematics and tensor operations are housed there, but it also fragments the constitutive story:

- small-strain and section laws are described in `materials`;
- finite-strain laws are described in `continuum`.

This split becomes problematic when the target is large-displacement, inelastic, reinforced-concrete analysis. At that point the constitutive module and the kinematic module must meet through a cleaner interface.

25.5 Performance Implications

25.5.1 Per-fiber type erasure is expensive

The current `Fiber` type stores:

```
Material<UniaxialMaterial>
```

per fiber. This gives maximal heterogeneity and a pleasant API, but it is a poor default for large reinforced-concrete simulations because:

- each fiber carries a heap-owning type-erased wrapper;
- copies of fiber sections duplicate that structure repeatedly;
- the hottest loop in `FiberSection::integrate_fibers()` pays dynamic-dispatch and scattered-memory costs.

For a few dozen fibers this is fine. For layered shells, detailed RC walls, or multiscale transfer problems it will quickly become one of the dominant costs.

25.5.2 The default inelastic storage is not HPC-friendly

`MemoryState<std::vector<S>>` as the default carrier for inelastic materials is not suitable as the long-term production layout. It implies:

- heap growth per point;
- no fixed-size state block;
- poor cache locality;
- no easy vectorisation across sites.

For HPC-oriented nonlinear analysis the natural constitutive storage unit is a small fixed record:

$$\text{trial state} \oplus \text{committed state} \oplus \text{internal variables}.$$

25.5.3 The hot path should remain statically typed

The present design already suggests the right rule:

runtime polymorphism should live at heterogeneous container boundaries,
not inside the innermost constitutive loops.

Therefore:

- element-level heterogeneous containers may continue to use type erasure;
- Gauss-point kernels, return mapping, and fiber loops should preferably operate on fully typed laws and fully typed state blocks.

25.6 A Cleaner Separation: Material vs Constitutive Relation

25.6.1 Recommended ontology

The recommended semantic split is:

1. **Material data**: immutable physical parameters of a phase, mixture, or section family.
2. **Constitutive law**: the mathematical law that defines stress, tangent, and evolution equations for a given measure space.
3. **Integration algorithm**: the numerical procedure that advances the law in time or load.
4. **Constitutive state**: committed and trial internal variables at one site.
5. **Constitutive site**: the object bound to an integration point or section point, owning its state.
6. **Reduction policy**: the map from lower-scale fields to reduced section measures, and conversely the reconstruction back to a richer field when needed.

25.6.2 What should be called "material"

In this ontology, *material* should denote the physical object or parameter block, not the entire runtime wrapper. Examples:

- `ConcreteMaterialData`;
- `SteelMaterialData`;
- `IsotropicElasticData`;
- `RCSectionData`.

25.6.3 What should be called "constitutive law"

A constitutive law should be a purely local mathematical object:

$$(\text{state},\ \text{kinematics},\ \text{context}) \mapsto (\text{response},\ \text{algorithmic tangent},\ \text{new state}).$$

Its contract should not imply geometry, meshing, or ownership of an integration point.

25.6.4 What should be called "reduction policy"

Structural section models are not materials in the same sense as steel or concrete. They are reductions of a lower-scale description. Therefore `FiberSection3D` is better understood as:

a beam-section reduction policy together with a constitutive quadrature over constituent fibers.

The same idea generalises naturally to:

- layered shells;
- smeared reinforcement layers;
- shell-through-thickness integration of concrete damage/plasticity;
- scale transfer from beam/shell models to continuum initial fields.

25.7 A More Scalable Constitutive Contract

25.7.1 Constitutive context

The current strategy interface only sees the kinematic measure k. For the next generation of constitutive models, the local update will need a richer context:

$$\mathcal{C}_{n+1} = \{\Delta t,\ t_{n+1},\ \text{load increment},\ \text{temperature},\ \text{kinematic frame},\ \ell_{\text{int}},\ \text{regularization data}\}.$$

This is essential for:

- rate dependence and viscoplasticity;
- nonlinear dynamics;
- finite-strain constitutive updates;
- nonlocal damage or gradient regularisation;
- fracture-energy based softening.

25.7.2 Suggested compile-time split

An architecture compatible with the current design philosophy would be:

`ConstitutiveSpace`	measure types only,
`ConstitutiveLaw`	physics only,
`IntegrationAlgorithm`	numerical update only,
`ConstitutiveState`	small fixed state blocks,
`ConstitutiveSite<Law,Algorithm,Storage>`	site-level wrapper.

This preserves static polymorphism because all of these remain templates or concept-constrained value types.

25.8 Implications for Reinforced Concrete Across Scales

25.8.1 Uniaxial constituent scale

At the constituent scale the current library already has a credible basis:

- `KentParkConcrete` for confined and unconfined concrete;
- `MenegottoPintoSteel` for reinforcing steel;
- `FiberSection3D` for reduced beam sections.

This scale is the correct place for:

- section-level hysteresis;
- bar yielding and cyclic effects;
- confined versus cover concrete distinction;
- section-force reconstruction.

25.8.2 Layered shells

The next natural scale extension is not yet 3D fracture, but layered shell constitutive models:

- concrete layers through thickness;
- membrane reinforcement layers;
- optional shear layers;
- section resultants obtained by through-thickness integration.

Architecturally this should reuse the same ideas as `FiberSection3D`, but on a shell reduction space rather than a beam section space.

25.8.3 Continuum RC with cracking and fracture

At the continuum scale, merely adding local softening to a stress-strain law is not enough. Once cracking localises, the formulation becomes mesh-sensitive unless an objective regularisation is introduced. The material architecture must therefore distinguish:

- **local constitutive law**: plasticity, damage, plastic-damage, orthotropic cracking, cohesive traction-separation;
- **regularisation policy**: crack-band scaling, nonlocal averaging, gradient enhancement, phase-field, or interface insertion.

This distinction is crucial. Fracture is not only a constitutive issue; it is also a localisation-regularisation issue.

25.8.4 Large displacements and dynamics

For large-displacement RC, the constitutive layer should not be tied to one specific small-strain measure. Instead, the local law should operate on the appropriate conjugate space supplied by the kinematic formulation:

- infinitesimal strain / Cauchy-like stress for small-strain models;
- Green–Lagrange / second Piola–Kirchhoff for total Lagrangian models;
- current-frame measures for updated-Lagrangian or corotational models.

The constitutive law and the kinematic policy must meet through an explicit measure-space contract, not through ad hoc branches.

25.9 Suggested Refactor Path

25.9.1 Stage 1: semantic cleanup

The first stage should be almost entirely architectural and low-risk:

- rename `MaterialPolicy` toward `ConstitutiveSpacePolicy` or similar;
- document that `Material<Policy>` is an operational wrapper, not a physical material;
- split bulk laws, section laws, and reduction policies into clearly named submodules.

25.9.2 Stage 2: state redesign

The second stage should replace `MemoryState` as the default inelastic carrier. The target storage is:

- fixed-size committed state;
- fixed-size trial state;
- optional recorder or history logger outside the hot path.

25.9.3 Stage 3: genuine algorithm-law separation

The third stage should make the integration strategy real for nonlinear laws:

- `BackwardEuler`;
- substepping decorators;
- line-search or local damping wrappers;
- possibly explicit and operator-split strategies for specific models.

At that point the nonlinear relation becomes a law again, not a law plus its only admissible update algorithm.

25.9.4 Stage 4: typed hot path, erased boundary

The fourth stage should move the runtime-erased wrapper outward:

- typed laws and typed states inside Gauss-point loops;
- erased wrappers only where heterogeneous containers are genuinely required.

For fiber sections, the natural next optimisation is:

- homogeneous fiber blocks for concrete patches and bar groups;
- structure-of-arrays storage for fiber coordinates, areas, and state blocks;
- optional erased fallback only for rare heterogeneous mixtures.

25.9.5 Stage 5: RC multiscale and fracture

Only after the semantic and state cleanup is stable does it make sense to add:

- layered RC shell sections;
- continuum plastic-damage or smeared cracking models;
- fracture-energy regularisation;
- cohesive interfaces or phase-field fracture.

Without the earlier cleanup, those features would be forced into already overloaded types and the module would become harder to maintain, not easier.

25.10 Non-owning Type Erasure in `Material.hh`

25.10.1 Viability assessment

The non-owning type-erasure pattern is viable in the materials module, but not as a wholesale replacement of the current owning wrapper. It is viable precisely because the borrowed erased model stores only:

- a pointer to the concrete constitutive object;
- a pointer to the integration strategy;
- one virtual-table pointer.

Therefore the non-owning erased model has a stable footprint of three machine pointers and fits naturally into a fixed local buffer. This is fundamentally different from the owning case, where the concrete constitutive objects are often too large and too heterogeneous for small-buffer optimisation to pay off.

25.10.2 What problem it solves

The borrowed view solves a specific architectural problem:

how to pass constitutive objects through generic code without cloning or re-owning them.

That is useful for:

- algorithms that only need a temporary constitutive handle;
- post-processing and reconstruction utilities;
- adapters layered on top of `MaterialPoint` and `MaterialSection`;
- future heterogeneous orchestration code that should not trigger deep copies accidentally.

25.10.3 What problem it does not solve

The borrowed view does *not* solve constitutive storage by itself. In particular, it should not be misused to pretend that the following problem has disappeared:

each constitutive site still needs well-defined ownership of its state.

This matters especially for reinforced-concrete fiber sections. Each fiber is not merely a material label; it is a constitutive site with its own history. Replacing per-fiber owning storage with a trivial shared borrowed view would collapse independent histories and would therefore be semantically wrong.

25.10.4 Why the strategy belongs in `Material.hh`

The strategy object is the correct injection point for the constitutive integration algorithm because it sits exactly at the operational boundary between:

- the constitutive relation as a law;
- the constitutive site as a stateful runtime object.

This is the right location to host future algorithms such as:

- backward-Euler return mapping;
- substepping;
- line-search or local damping decorators;
- explicit or operator-split updates for dynamics.

By contrast, embedding all integration logic directly inside the relation class fuses law and algorithm and makes it much harder to reuse the same physical law with different numerical update procedures.

25.10.5 Refactor adopted in the current code

The present refactor therefore adopts a hybrid design:

- `Material<Policy>` remains the owning, heap-backed erased wrapper;
- `MaterialConstRef<Policy>` is a borrowed read-only erased view;
- `MaterialRef<Policy>` is a borrowed mutable erased view.

This preserves the current ownership semantics of `MaterialPoint`, `MaterialSection`, and `FiberSection`, while enabling generic algorithms to work with borrowed constitutive handles when ownership transfer is not intended.

25.10.6 Architectural consequence

The key consequence is that the module now has a clearer boundary:

- ownership and lifetime remain explicit;
- the hot constitutive storage is not forced into an unsuitable small-buffer abstraction;
- the library gains a lightweight non-owning polymorphic layer exactly where it is semantically valid.

25.11 Final Opinion

The current materials module is already much better than a monolithic "stress-strain class" design. It has:

- strong measure-based abstractions;
- useful concept constraints;
- a credible path for structural nonlinear RC through fibers;
- a polymorphic envelope that works across solids, beams, and shells.

But the module is now at the point where its first successful generalisation is becoming its main semantic limitation. The primary issue is no longer whether the constitutive code works. It does. The primary issue is that the present API still uses the word "material" for too many different ontological objects.

The recommended direction is therefore not a rewrite, but a disciplined separation:

material data → constitutive law → integration algorithm → stateful constitutive site → reduction / reconstruction policy.

If this separation is carried through while preserving the existing compile-time contracts, `fall_n` will have a robust basis for:

- nonlinear reinforced concrete frame analysis;
- layered and reduced-order structural models;
- finite-strain and dynamic constitutive updates;
- objective continuum cracking and fracture formulations.

That is the right architectural target.

Chapter 26

Material Module Semantic Cleanup

26.1 Purpose

This chapter documents the first low-risk semantic cleanup applied to the materials module after the architectural analysis of Chapter 25. The goal of this phase is not to replace the existing API, but to make the ontology explicit while preserving the current executable code and tests.

The cleanup follows three rules:

- no behavioural change in the constitutive hot path;
- no forced migration of existing code;
- new names must coexist with legacy names until the next refactor stage.

26.2 What Was Changed

26.2.1 Constitutive-space aliases

The file `MaterialPolicy.hh` now exposes semantically explicit aliases:

- `ContinuumConstitutiveSpace<N>`;
- `BeamConstitutiveSpace<N,Dim>`;
- `ShellConstitutiveSpace<N,Dim>`.

Alongside them, named aliases now exist for the most common spaces:

- `ThreeDimensionalConstitutiveSpace`;
- `TimoshenkoBeamConstitutiveSpace3D`;
- `MindlinReissnerShellConstitutiveSpace3D`;
- and their lower-dimensional counterparts.

The legacy names `ThreeDimensionalMaterial`, `TimoshenkoBeam3D`, etc. remain available as compatibility aliases.

26.2.2 Stateful constitutive-site aliases

The file `LinealElasticMaterial.hh` now exposes:

- `ConstitutiveSite<Relation,StatePolicy>`;
- `ElasticConstitutiveSite<Relation>`;
- `InelasticConstitutiveSite<Relation>`.

These are semantic aliases for the existing `MaterialInstance` class. The old name is intentionally preserved, but the new aliases make the object's meaning explicit: it is already a *stateful constitutive site*, not a mere material definition.

26.2.3 Erased constitutive handles

The file `Material.hh` now exposes the following semantic aliases:

- `ConstitutiveHandle<Space>` for the owning erased wrapper;
- `ConstitutiveConstHandleRef<Space>` for the borrowed const view;
- `ConstitutiveHandleRef<Space>` for the borrowed mutable view.

This makes the role of `Material<Policy>` explicit without breaking the existing API.

26.2.4 Constitutive-site accessors

`MaterialPoint` and `MaterialSection` now expose both naming families:

- legacy accessors: `material_cref()`, `material_ref()`;
- semantic accessors: `constitutive_cref()`, `constitutive_ref()`.

Additional site aliases were introduced:

- `ContinuumConstitutiveSite<Space>`;
- `SectionConstitutiveSite<Space,PointDim>`.

26.3 Ownership Semantics Clarified

One important contradiction was removed from the documentation of `MaterialInstance`. The previous text suggested flyweight-style sharing as the default interpretation of the relation handle. That was not true of the actual copy semantics.

The code already behaved as follows:

- ordinary copy construction deep-clones the relation;
- explicit sharing occurs only when the user passes an external `std::shared_ptr<Relation>`.

The comments now match that behaviour. This is important because implicit sharing between constitutive sites would be semantically dangerous in nonlinear analysis.

26.4 Why This Stage Matters

The cleanup may look cosmetic at first sight, but it removes a real source of architectural ambiguity. Before this change, the same word "material" could refer to all of the following:

- a constitutive space;
- a physical material definition;
- a stateful constitutive site;
- an owning erased runtime handle.

After this stage, the code base can distinguish these meanings explicitly without forcing a flag day migration.

26.5 What Was Intentionally Not Changed

This cleanup does *not* yet:

- separate constitutive law and constitutive state into distinct runtime objects;
- remove `MemoryState` from the default inelastic path;
- move `FiberSection` out of the materials namespace hierarchy;
- change the hot-loop data layout of fiber sections or continuum points.

Those changes belong to the next refactor stage and would carry more risk.

26.6 Immediate Benefits

The immediate benefits of this stage are:

- clearer architectural vocabulary in code and tests;
- more honest documentation of ownership and copying;
- a smoother path toward the next law/state/algorithm split;
- better traceability for future reinforced-concrete, multiscale, and fracture work.

26.7 Next Step

The next meaningful refactor should preserve these aliases and attack the real state-management issue:

$$\textbf{constitutive law} \oplus \textbf{integration algorithm} \oplus \textbf{constitutive state}$$

Only after that separation is explicit will it be reasonable to redesign the storage layout for large-scale reinforced-concrete simulations, nonlinear dynamics, and continuum cracking or fracture models.

Chapter 27

Constitutive State Storage Cleanup

27.1 Motivation

The semantic cleanup of the materials module revealed a persistent mismatch: the constitutive-site abstraction in `fall_n` stored kinematic history by default for inelastic materials, even though the vast majority of the code used `current_state()` only as the *last committed kinematic state*. This had three undesirable consequences:

- the default inelastic path allocated and grew storage that was not needed by most algorithms;
- the API blurred the distinction between "current committed value", "full history", and "trial state";
- the architecture was poorly prepared for generic constitutive integration algorithms requiring trial/accept/reject workflows.

This chapter introduces an explicit state-storage vocabulary while preserving backward compatibility and the static, header-only nature of the hot path.

27.2 Storage Roles

The cleanup distinguishes three different storage roles.

27.2.1 CommittedState

`CommittedState<S>` stores a single value of type `S`. Every `update()` overwrites the previously stored state. This is the correct default for:

- elastic materials;
- inelastic constitutive sites whose irreversible memory already lives inside the constitutive law;
- post-processing paths that only need the last accepted strain-like measure at a material site or section site.

Architecturally this is just the existing `ElasticState<S>` under a new semantic alias. The implementation remains trivial and fully inlineable.

27.2.2 HistoryState

`HistoryState<S>` is the explicit opt-in path for storing a full kinematic history. It is semantically equivalent to the legacy `MemoryState<S>`, but the key clarification is that the policy now owns a *storage backend*, not merely a hard-coded `std::vector`. Each `update()` appends to the supplied storage, and `current_value()` returns the last entry.

Two storage backends are now provided explicitly:

- `DynamicHistoryStorage<S>` for unbounded growth;
- `CircularHistoryStorage<S, N>` for a fixed-size sliding window known at compile time.

This distinction matters because some constitutive models genuinely need history, but not *all* of it. For such cases, a circular buffer provides deterministic memory, cache-friendly locality, and zero heap growth after construction.

This path remains useful for:

- debugging and validation;
- offline material diagnostics;
- experimental algorithms that truly need access to all previous committed kinematic states.

The important change is that `HistoryState` is no longer the *implicit* inelastic default, but remains available with whichever storage backend is appropriate for the algorithm.

27.2.3 TrialCommittedState

`TrialCommittedState<S>` introduces a minimal two-level storage:

- a committed state;
- an optional staged trial state.

The policy exposes:

- `update()` to stage a trial value;
- `commit_trial()` to accept it;
- `revert_trial()` to discard it;
- `current_value()` to see the trial state if present, otherwise the committed one.

This policy is not yet the production default. Its role is preparatory: it provides the storage semantics needed by future generic integration strategies without forcing those algorithms to be encoded inside each constitutive law.

27.3 MaterialState as a Storage Adapter

`MaterialState` now depends on the more accurate concept `StateStoragePolicyFor`. The legacy name `MemoryPolicyFor` is preserved as an alias so existing code remains valid during the transition.

Besides `current_value()` and `update()`, `MaterialState` now conditionally forwards:

- `commit_trial()`;

- `revert_trial()`;
- `has_trial_value()`;
- `committed_value()`;
- `trial_value()`;
- `size()` and indexed logical access when the storage backend is history-bearing.

This conditional forwarding is implemented with `requires`-based introspection. No virtual dispatch is introduced, and no overhead is paid for policies that do not support staged trial states.

27.4 Change in the Default Inelastic Path

The most important semantic change is in `MaterialInstance<Relation, StatePolicy>`:

- `InelasticConstitutiveSite<R>` now defaults to `CommittedState`;
- explicit history tracking remains available through `HistoryTrackingConstitutiveSite<R>` and `HistoryTrackedInelasticMaterial<R>`;
- bounded history tracking is available through `CircularHistoryConstitutiveSite<R, N>`, which binds the site to a fixed compile-time circular buffer;
- `TrialConstitutiveSite<R>` exposes the staged-storage path for future algorithmic work.

This is semantically more honest. In the current architecture, models such as `J2PlasticityRelation`, `KentParkConcrete`, and `MenegottoPintoSteel` store their irreversible internal variables inside the constitutive relation itself. The site-level kinematic state is therefore not the true inelastic memory; it is primarily a local snapshot of the last accepted measure used by post-processing and reconstruction.

27.5 Interaction with Constitutive Updates

The constitutive-site update path was also adjusted so that storage policies with staged trial semantics behave correctly under the existing API:

1. the constitutive law updates its own irreversible variables;
2. the site storage receives the kinematic state through `state_.update(k)`;
3. if the storage policy supports `commit_trial()`, the staged value is immediately promoted to committed state.

This keeps current behaviour unchanged for production algorithms while making the storage layer ready for a later refactor in which the constitutive integrator controls trial and commit phases explicitly.

27.6 Why This Matters for Reinforced Concrete Across Scales

For reinforced concrete across continuum, sectional, and frame scales, a clean separation between *constitutive law*, *state storage*, and *integration algorithm* is essential:

- frame fiber sections require many local constitutive sites, so storage waste multiplies quickly;
- dynamic nonlinear analyses require robust and explicit commit/revert semantics;
- large-displacement and finite-strain formulations need a precise notion of trial versus committed state;
- continuum-scale cracking and fracture will eventually require local and nonlocal internal variables, which should not be confused with merely storing a history of kinematic inputs.

This cleanup does not yet solve those higher-level problems. It does, however, remove a semantic obstacle: site-level storage is no longer forced to pretend that "inelastic" automatically means "store all past kinematics".

27.7 Next Refactor Stage

With storage semantics clarified, the next architectural step is now better defined:

1. separate `ConstitutiveLaw` from `ConstitutiveState`;
2. let the constitutive integrator evolve the state explicitly instead of delegating the entire algorithm to the law;
3. make the future constitutive-state object choose not only its semantic role (committed/history/trial) but also its storage backend (dynamic, circular, small-buffer, etc.);
4. reserve non-owning type erasure for borrowed constitutive views rather than for owned site storage;
5. keep the hot path statically typed for homogeneous loops such as fiber integration and section reconstruction.

That next stage is the one that will most directly enable robust nonlinear reinforced concrete, dynamics, and eventually objective crack/fracture modelling at the continuum scale.

Chapter 28

ConstitutiveState as a Semantic Layer

28.1 Why Another Layer Was Needed

After the storage cleanup of the previous chapter, the code still had an architectural ambiguity: `MaterialState` was doing two jobs at once.

- At the low level, it was a generic adapter over a storage policy.
- At the constitutive-site level, it was implicitly standing in for the local state of a material point or section point.

Those roles are related, but they are not identical. A storage adapter is an implementation device. A constitutive state is a modelling concept.

To make that distinction explicit without breaking the existing kernels, this phase introduces `ConstitutiveState<StateStoragePolicy, ...>` as a thin, fully inline semantic wrapper over `MaterialState`.

28.2 Layered View

The state path is now:

`HistoryStorage` / `CircularHistoryStorage` / `ElasticState` / `TrialCommittedState` → `MaterialState` → `ConstitutiveState` → `MaterialInstance`

The meaning of each layer is now cleaner:

- `HistoryStorage` and its variants define how samples are physically stored;
- `MaterialState` adapts that storage to a uniform compile-time contract;
- `ConstitutiveState` says that the stored quantity is the local state of a constitutive site;
- `MaterialInstance` binds that local state to a constitutive law.

28.3 What ConstitutiveState Adds

`ConstitutiveState` does not add runtime overhead or dynamic dispatch. Its value is semantic and architectural:

- it gives `MaterialInstance` an explicit `ConstitutiveStateT` alias;
- it exposes `constitutive_state()` as a first-class accessor;
- it lets the code and the documentation talk about "the constitutive state of the site" instead of "the storage policy wrapper";
- it prepares the next step, where the constitutive law should stop owning all irreversible memory internally.

In other words, the wrapper is intentionally thin, but its name matters. Architecturally, names are part of the model.

28.4 Effect on MaterialInstance

`MaterialInstance<Relation, StatePolicy>` no longer stores `MaterialState<StatePolicy, KinematicT>` directly. It now stores

```
ConstitutiveState<StatePolicy, KinematicT>
```

and exposes it through:

- `current_state()` as the legacy direct view;
- `constitutive_state()` as the new semantic accessor.

This means user code and future algorithms can progressively move from "give me the current strain-like value" toward "give me the constitutive-state object attached to this site".

28.5 Why This Matters for Future Nonlinear Algorithms

The next architectural step should separate:

1. the constitutive law;
2. the constitutive state;
3. the integration algorithm.

That separation is difficult if the local state has no identity of its own. By introducing `ConstitutiveState`, the code now has a clear place where future information can migrate:

- committed kinematic state;
- trial state;
- algorithmic internal variables;
- nonlocal history variables;
- fracture or damage regularisation data;
- time-integration metadata for dynamic constitutive updates.

This does not mean all of that should be stored in one monolithic object. It means the architecture now has a semantically correct anchor for those future choices.

28.6 Static Polymorphism Is Preserved

The introduction of `ConstitutiveState` preserves the compile-time character of the constitutive hot path:

- all storage-policy choices remain template parameters;
- no additional virtual calls are introduced;
- circular-history windows still use fixed-size arrays and compile-time capacity;
- the relation type remains statically known inside homogeneous loops.

This is important for fiber sections and any future continuum crack/fracture model where thousands or millions of local constitutive sites may be updated.

28.7 What This Refactor Still Does Not Do

Even with `ConstitutiveState` in place, the local nonlinear laws in the current code still keep their irreversible memory internally. Therefore:

- `ConstitutiveState` is not yet the sole owner of local history;
- `ConstitutiveIntegrator` still does not evolve an explicit state object passed by argument;
- the design is cleaner, but not yet fully factored.

This is acceptable at this stage. The refactor is intentionally incremental: first make the ontology correct, then move behaviour.

28.8 Immediate Consequence

From this point onward, the recommended interpretation is:

- `MaterialState`: low-level storage adapter;
- `ConstitutiveState`: local state object of a constitutive site;
- `MaterialInstance`: site-level binding of relation + state;
- `Material<Policy>`: erased owning constitutive handle;
- `MaterialPoint` / `MaterialSection`: continuum and structural constitutive sites bound to geometric integration sites.

This vocabulary is much closer to the real mathematical model and therefore a better base for the next refactor stage.

Chapter 29

ConstitutiveIntegrator as the Algorithmic Injection Point

29.1 Motivation

After introducing `ConstitutiveState`, the constitutive subsystem still had one important ambiguity: the algorithmic step used to evaluate and commit a constitutive site was semantically present, but it was still described under the generic name `IntegrationStrategy`. That name was operationally useful, but it was too broad. The relevant object at this level is not any numerical integration procedure; it is the *local constitutive integrator*.

This chapter makes that distinction explicit by introducing `ConstitutiveIntegrator`.

29.2 Local Architectural Split

The constitutive subsystem is now described by four roles:

1. `ConstitutiveRelation`: the physical local law;
2. `ConstitutiveState`: the state attached to the site;
3. `ConstitutiveIntegrator`: the algorithm that queries and commits the site;
4. `Material<Policy>`: the erased runtime handle that injects the chosen integrator into heterogeneous containers.

This is not yet the final fully factored architecture, because many current nonlinear laws still keep irreversible variables internally. However, the public and internal semantics now reflect the intended long-term split:

```
ConstitutiveLaw + ConstitutiveState + ConstitutiveIntegrator
```

29.3 Why the Injection Point Stays in `Material<Policy>`

The local algorithm is injected at the `Material<Policy>` boundary on purpose.

- It preserves the existing type-erased heterogeneous storage model.
- It avoids contaminating each constitutive relation with multiple algorithmic variants prematurely.

- It keeps the hot path statically typed inside each homogeneous material model, while the runtime polymorphism remains confined to the already existing erased handle.
- It naturally matches the user-facing syntax:

```
Material<ThreeDimensionalMaterial>{
    J2PlasticMaterial3D{E, nu, sigma_y, H},
    EmbeddedInelasticConstitutiveIntegrator{}
};
```

- It opens the door to future variants such as substepping, line-search local updates, operator splits, or explicit/implicit constitutive drivers without changing element kernels.

29.4 Current Concrete Integrators

Two semantic integrators are now made explicit:

- `PassThroughIntegrator`: path-independent evaluation. It reads stress and tangent directly from the constitutive relation and keeps `commit()` as a no-op for backward-compatible elastic behaviour.
- `EmbeddedRelationIntegrator`: compatibility bridge for current nonlinear laws. It evaluates response and tangent from the relation, and during `commit()` it advances the relation through its own `update()` method while also updating the external `ConstitutiveState`.

The legacy names remain as aliases:

- `ElasticUpdate` ≡ `PassThroughIntegrator`
- `InelasticUpdate` ≡ `EmbeddedRelationIntegrator`

This keeps the codebase source-compatible while allowing the semantic cleanup to proceed.

29.5 Why This Matters

The key architectural benefit is that the current code can now express the following distinction:

- the constitutive *law* is responsible for the physics;
- the constitutive *integrator* is responsible for the local algorithmic realization of that physics inside a time/load step.

That distinction is essential for future nonlinear concrete and fracture work. For example, the same inelastic law may later be integrated by:

- an embedded monolithic return-mapping;
- a generic backward-Euler driver;
- a substepping wrapper;
- a viscoplastic regularized driver;
- a dynamic operator split coupled to finite-kinematics update rules.

29.6 Custom Integrators

The refactor also validates a more important property than the naming itself: `Material<Policy>` is now an actual injection point for custom local algorithms.

The new regression test introduces a custom integrator that intentionally updates only the external constitutive-state carrier but does not evolve the relation's internal irreversible variables. The result is a predictable change in post-commit behaviour, which proves that the algorithmic policy is not hard-wired to the constitutive relation.

This is exactly the kind of freedom required for future scale-reconstruction pipelines and multi-algorithm benchmarking.

29.7 What Has Not Been Completed Yet

This step is a semantic and architectural split, not a full behavioural one. The following limitations remain:

- many inelastic relations still own their irreversible state internally;
- the integrator does not yet receive an explicit algorithmic state object distinct from the constitutive relation;
- the current bridge integrator still relies on `relation.update(k)` for nonlinear constitutive evolution.

Therefore, the next meaningful refactor should move the local evolution data out of the constitutive relation and into explicit state objects wherever that is mathematically coherent.

29.8 Immediate Architectural Interpretation

After this step, the recommended reading of the subsystem is:

- `ConstitutiveRelation`: local physical constitutive model;
- `ConstitutiveState`: state attached to the site;
- `ConstitutiveIntegrator`: local algorithmic driver;
- `MaterialInstance`: typed constitutive site;
- `Material<Policy>`: erased constitutive handle with injected local integrator.

This is a materially more precise ontology than the previous generic "strategy" language, and it is better aligned with the future goals of inelastic reinforced concrete, dynamics, large displacements, and continuum-scale damage or fracture.

Chapter 30

Explicit Algorithmic State in `PlasticityRelation`

30.1 Why This Step Matters

The previous chapter introduced `ConstitutiveIntegrator` as the algorithmic injection point at the erased constitutive-handle boundary. That was necessary, but not sufficient. As long as the nonlinear constitutive law still owns its irreversible variables internally, the split remains only partial.

This phase takes the first behavioural step beyond semantics: `PlasticityRelation` can now be evaluated and committed against an *explicit external* algorithmic state.

30.2 Old Situation

Originally, `PlasticityRelation` stored a member α_n internally and all core operations were implicitly tied to that member:

$$\boldsymbol{\sigma}_{n+1} = \texttt{compute_response}(\boldsymbol{\varepsilon}_{n+1}), \tag{30.1}$$
$$\mathbf{C}^{ep}_{n+1} = \texttt{tangent}(\boldsymbol{\varepsilon}_{n+1}), \tag{30.2}$$
$$\alpha_{n+1} = \texttt{update}(\boldsymbol{\varepsilon}_{n+1}). \tag{30.3}$$

That design is workable for a local prototype, but architecturally it creates three problems:

- the constitutive law is not clearly separated from the constitutive site;
- the local integration algorithm cannot be cleanly redirected to other state carriers;
- sharing or borrowing the law becomes semantically fragile because the relation still behaves as if it owned the site memory.

30.3 New Explicit-State Contract

`PlasticityRelation` now supports a second, more explicit interface:

```
ConjugateT compute_response(const KinematicT& eps,
                            const InternalVariablesT& alpha) const;

TangentT tangent(const KinematicT& eps,
                 const InternalVariablesT& alpha) const;

```

```
void commit(InternalVariablesT& alpha,
            const KinematicT& eps) const;
```

Mathematically, the relation is now able to realize the return-mapping map

$$(\boldsymbol{\varepsilon}_{n+1}, \alpha_n) \mapsto (\boldsymbol{\sigma}_{n+1}, \mathbf{C}^{ep}_{n+1}, \alpha_{n+1}) \tag{30.4}$$

without requiring α_n to live inside the relation object itself.

30.4 Compatibility Is Preserved

The embedded-state API is still available:

- `compute_response(eps)`
- `tangent(eps)`
- `update(eps)`
- `internal_state()`

Those methods now act as a compatibility layer around the explicit-state core. Therefore:

- existing tests and client code continue to work;
- the new architecture can progressively migrate site storage out of the relation;
- naked relations can still be used in the old stateful way when convenient.

30.5 What Changed Internally

The central change is not the public overloads by themselves, but the internal factoring of the return-mapping kernel. The predictor/corrector path now receives α_n explicitly:

- elastic predictor uses the supplied plastic strain;
- effective trial stress uses the supplied hardening state;
- return-mapping builds α_{n+1} from the supplied α_n, not from an implicitly owned member.

This makes the local nonlinear map much closer to its mathematical form.

30.6 Interaction with `MaterialInstance`

The typed constitutive site `MaterialInstance<Relation, ...>` now uses the explicit-state path automatically when the relation satisfies `ExternallyStateDrivenConstitutiveRelation`. In that case:

- the site stores the algorithmic internal variables;
- `compute_response()` and `tangent()` query the relation with that explicit state;
- `update()` commits the site state through `relation.commit(alpha, eps)`;
- `internal_state()` exposes the site-level algorithmic memory, not the compatibility state embedded inside the relation.

This is the first place where the constitutive site becomes semantically more important than the relation object.

30.7 Interaction with `ConstitutiveIntegrator`

The bridge integrator `EmbeddedRelationIntegrator` now detects the new contract and prefers:

```
relation.commit(site.algorithmic_state(), eps)
```

over the legacy

```
relation.update(eps)
```

when that explicit-state path exists.

Therefore the architectural hierarchy is now more coherent:

- `PlasticityRelation` provides a local nonlinear map with an explicit algorithmic state argument;
- `MaterialInstance` owns the state of the constitutive site;
- `ConstitutiveIntegrator` drives the commit path.

30.8 What the New Tests Prove

The new regression tests verify three important facts:

1. committing an external plasticity state evolves that external state;
2. doing so does *not* mutate the compatibility state owned by the naked relation;
3. a `MaterialInstance` built from `PlasticityRelation` now stores the plastic internal variables at the site level, and subsequent stresses depend on that site state.

That is the first concrete proof that behaviour is starting to move away from law-owned memory and toward site-owned memory.

30.9 Why the Cache Was Not Reused

The embedded-state implementation of `PlasticityRelation` contained a mutable cache keyed effectively by the strain. That cache was valid only under the implicit assumption that a given relation instance corresponded to a single local state.

Once the same relation can be evaluated with arbitrary explicit states, that assumption is no longer valid. Reusing the old cache for the explicit-state path would have been semantically unsafe.

The current explicit-state overloads therefore prioritise correctness over micro-optimisation and compute directly from the supplied state. If needed later, the correct optimisation point is the constitutive site or the local integrator, not a state-agnostic mutable cache inside the law.

30.10 Current Boundary of the Refactor

At this stage, only `PlasticityRelation` has been migrated to the new explicit-state contract. The following models still keep their irreversible memory internally:

- `MenegottoPintoSteel`
- `KentParkConcrete`
- `FiberSection`

This is intentional. The architectural pattern is now proven on one important continuum inelastic law before being propagated to uniaxial hysteretic and section-reduction models.

30.11 Next Logical Step

The next meaningful move is to propagate the same pattern to uniaxial nonlinear laws, especially those used in reinforced-concrete fiber sections. That will make the constitutive-site state explicit not only for continuum plasticity, but also for steel hysteresis, concrete crushing/softening, and eventually section-level nonlinear reductions.

Once that is in place, the path toward dynamic constitutive updates, finite kinematics, nonlocal regularisation, and continuum-scale damage or fracture becomes much clearer.

Bibliography

[1] Satish Balay, Shrirang Abhyankar, Mark F. Adams, et al. PETSc Users Manual (revision 3.24). `https://petsc.org/release/docs/manual/`, 2024. Argonne National Laboratory, ANL-21/39.

[2] Philippe G. Ciarlet and Pierre-Arnaud Raviart. General Lagrange and Hermite interpolation in $\mathbb{R}^n$ with applications to finite element methods. *Archive for Rational Mechanics and Analysis*, 46(3):177–199, 1972.

[3] Pooyan Dadvand, Riccardo Rossi, and Eugenio Oñate. An object-oriented environment for developing finite element codes for multi-disciplinary applications. *Archives of Computational Methods in Engineering*, 17(3):253–297, 2010.

[4] Michael G. Duffy. Quadrature over a pyramid or cube of integrands with a singularity at a vertex. *SIAM Journal on Numerical Analysis*, 19(6):1260–1262, 1982.

[5] Walter Gautschi. *Orthogonal Polynomials: Computation and Approximation.* Numerical Mathematics and Scientific Computation. Oxford University Press, 2004.

[6] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009.

[7] Gene H. Golub and John H. Welsch. Calculation of Gauss quadrature rules. *Mathematics of Computation*, 23(106):221–230, 1969.

[8] A. Grundmann and H. M. Möller. Invariant integration formulas for the n-simplex by combinatorial methods. *SIAM Journal on Numerical Analysis*, 15(2):282–290, 1978.

[9] Preston C. Hammer, O. J. Marlowe, and Arthur H. Stroud. Numerical integration over simplexes and cones. *Mathematical Tables and Other Aids to Computation*, 10(55):130–137, 1956.

[10] Patrick Keast. Moderate-degree tetrahedral quadrature formulas. *Computer Methods in Applied Mechanics and Engineering*, 55(3):339–348, 1986.

[11] Frank McKenna, Gregory L. Fenves, Michael H. Scott, and Boris Jeremić. Open system for earthquake engineering simulation (OpenSees). *Pacific Earthquake Engineering Research Center, University of California, Berkeley*, 2000. `https://opensees.berkeley.edu/`.

[12] Arthur H. Stroud. *Approximate Calculation of Multiple Integrals.* Prentice-Hall Series in Automatic Computation. Prentice-Hall, 1971.

[13] Gábor Szegő. *Orthogonal Polynomials.* Colloquium Publications. American Mathematical Society, 4th edition, 1975.

[14] O. C. Zienkiewicz and J. Z. Zhu. The superconvergent patch recovery and a posteriori error estimates. Part 1: The recovery technique. *International Journal for Numerical Methods in Engineering*, 33(7):1331–1364, 1992.

[15] O. C. Zienkiewicz and J. Z. Zhu. The superconvergent patch recovery and a posteriori error estimates. Part 2: Error estimates and adaptivity. *International Journal for Numerical Methods in Engineering*, 33(7):1365–1382, 1992.